



Fourier Transform

Fourier Transforms



Fourier transform is an extremely powerful mathematical tool that allows to view the signal in the different domain (Frequency)

This allows to easily pick and choose the most important components and remove all the noise.

Image processing

The Fourier transform gives information about what is happening to image in frequencies terms.

- Certain operation are easier to perform in frequency domain
- Image transform helps to design faster algorithms
- Eliminating high frequencies give image blurs
- Eliminating low frequencies gives image edges
- Enhancing high frequencies and keeping the low frequencies leads to image sharpness

Applications in computer vision



Image Filtering:

By manipulating the frequencies in the frequency domain (e.g., removing high frequencies to blur an image or enhancing high frequencies to sharpen it), you can achieve various filtering effects.

Noise Reduction:

High-frequency noise can be **identified and suppressed** in the frequency domain.

Image Compression:

By discarding the **less significant high-frequency components**, the image can be compressed.

Blur Detection:

A blurry image will have a lower concentration of high-frequency components compared to a sharp image, which can be detected by **analyzing the magnitude of the FFT**.

Feature Extraction:

The Fourier transform can **highlight periodic patterns and texture**, which can be used as **features for tasks like shape recognition**.

Jean Baptiste Josef Fourier



(1768-1830)



$$f(x) = \frac{1}{2} a_0 + \sum_{n=1}^{\infty} a_n \cos nx + \sum_{n=1}^{\infty} b_n \sin nx$$

where,

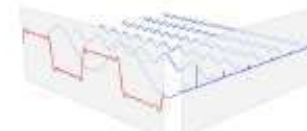
- $a_0 = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) dx$
- $a_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos nx dx$
- $b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin nx dx$
- $n = 1, 2, 3, \dots$

French mathematician, known also as an Egyptologist and administrator, who exerted strong influence on mathematical physics

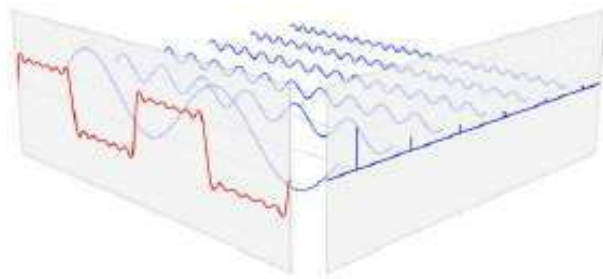
He showed how the conduction of heat in solid bodies may be analyzed in terms of infinite mathematical series now called by his name, the Fourier series

Any Periodic Function can be rewritten as a Weighted Sum of Infinite Sinusoids of different frequencies.

Egyptologist: study all the aspects of ancient Egypt that they can, across periods from about 7.000 BC to the early middle aas.



Basic building block:



infinite sum of sine waves

$$A \sin(\omega x + \phi)$$

Diagram illustrating the components of the sine wave equation $A \sin(\omega x + \phi)$:

- A : amplitude
- $\sin$: sinusoid
- ω : angular frequency
- x : variable
- ϕ : phase

Fourier's claim: Add enough of these to get any periodic signal you want!

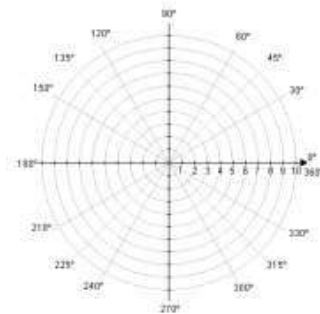
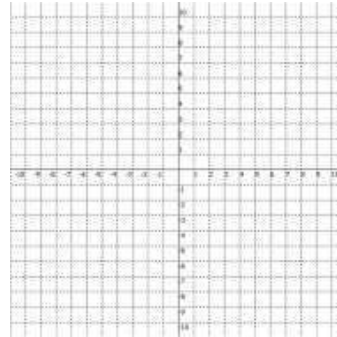
Complex number Basics

Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

what's this? what's this?



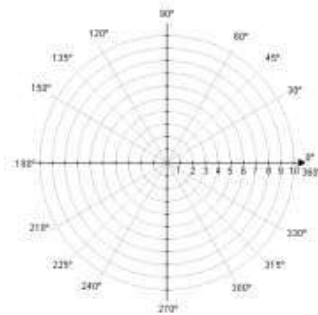
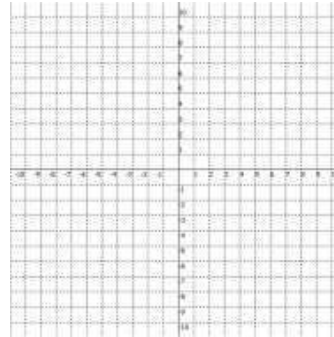


Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

real imaginary





Complex numbers have two parts:

rectangular
coordinates

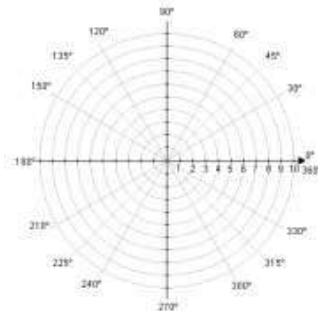
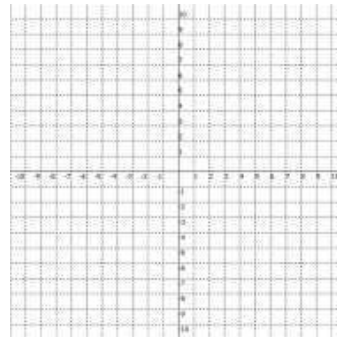
$$R + jI$$

real imaginary

polar
coordinates

$$r(\cos \theta + j \sin \theta)$$

how do we compute these?



polar transform

Complex numbers

Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

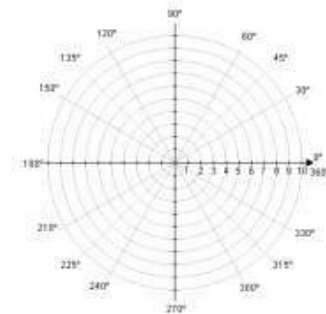
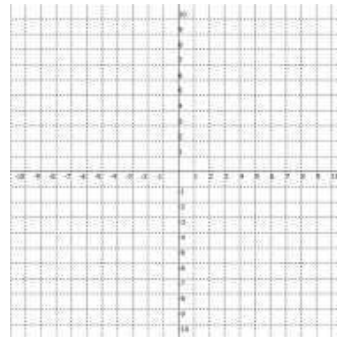
real imaginary

polar
coordinates

$$r(\cos \theta + j \sin \theta)$$

polar transform

$$\theta = \tan^{-1}\left(\frac{I}{R}\right) \quad r = \sqrt{R^2 + I^2}$$



polar transform



Complex numbers: Rectangular to Polar transform



Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

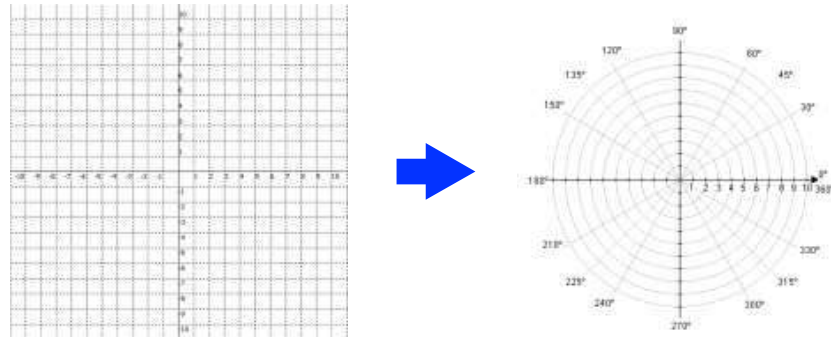
real imaginary

polar
coordinates

$$r(\cos \theta + j \sin \theta)$$

polar transform

$$\theta = \tan^{-1}\left(\frac{I}{R}\right) \quad r = \sqrt{R^2 + I^2}$$



polar transform

How do you write
these in exponential
form?

Complex numbers: Exponential form

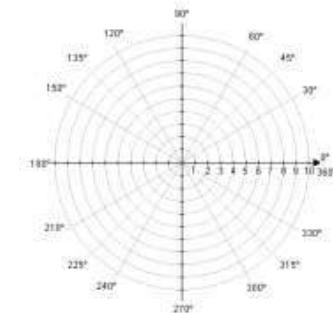
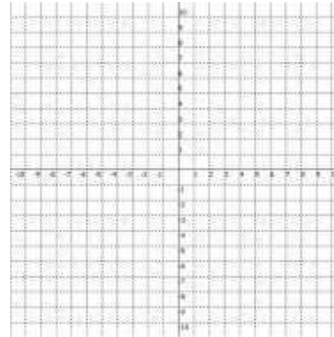


Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

real imaginary



polar
coordinates

$$r(\cos \theta + j \sin \theta)$$

polar transform

$$\theta = \tan^{-1}\left(\frac{I}{R}\right) \quad r = \sqrt{R^2 + I^2}$$

or
equivalently

$$re^{j\theta}$$

how did we get this?

exponential
form

Complex numbers: Exponential form

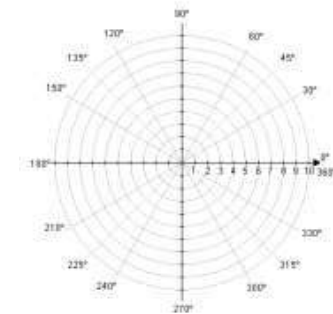
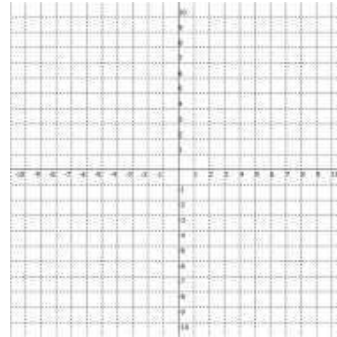


Complex numbers have two parts:

rectangular
coordinates

$$R + jI$$

real imaginary



polar
coordinates

$$r(\cos \theta + j \sin \theta)$$

polar transform

$$\theta = \tan^{-1}\left(\frac{I}{R}\right) \quad r = \sqrt{R^2 + I^2}$$

or
equivalently

$$re^{j\theta}$$

Euler's formula

$$e^{j\theta} = \cos \theta + j \sin \theta$$

exponential
form

This will help us understand the *Fourier transform equations*

Fourier transform



Fourier transform

inverse Fourier transform

continuous

$$F(k) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi kx} dx$$

$$f(x) = \int_{-\infty}^{\infty} F(k) e^{j2\pi kx} dk$$

discrete

$$F(k) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

$k = 0, 1, 2, \dots, N-1$

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

$x = 0, 1, 2, \dots, N-1$

Where is the connection to the 'summation of sine waves' idea?

Fourier transform

Fourier transform

inverse Fourier transform

continuous

$$F(k) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi kx} dx$$

$$f(x) = \int_{-\infty}^{\infty} F(k) e^{j2\pi kx} dk$$

discrete

$$F(k) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

$k = 0, 1, 2, \dots, N-1$

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

$x = 0, 1, 2, \dots, N-1$

Where is the connection to the ‘summation of sine waves’ idea?

Fourier transform



Where is the connection to the ‘summation of sine waves’ idea?

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

Euler's formula

$$e^{j\theta} = \cos \theta + j \sin \theta$$

sum over frequencies

$$f(x) = \sum_{k=0}^{N-1} F(k) \left\{ \cos(2\pi kx) + j \sin(2\pi kx) \right\}$$

scaling parameter

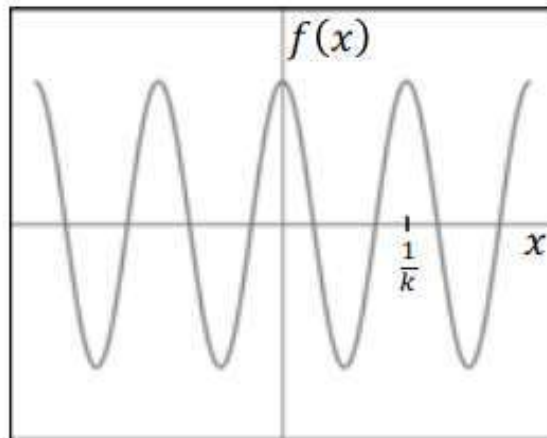
wave components

$$\sin(x) = \frac{e^{ix} - e^{-ix}}{2i}$$

Fourier transform examples:

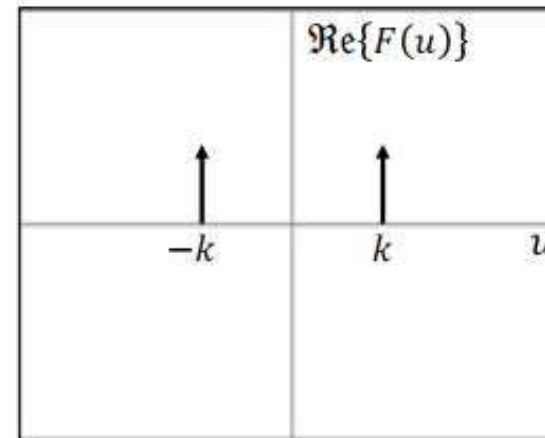


Signal $f(x)$



$$f(x) = \cos 2\pi kx$$

Fourier Transform $F(u)$



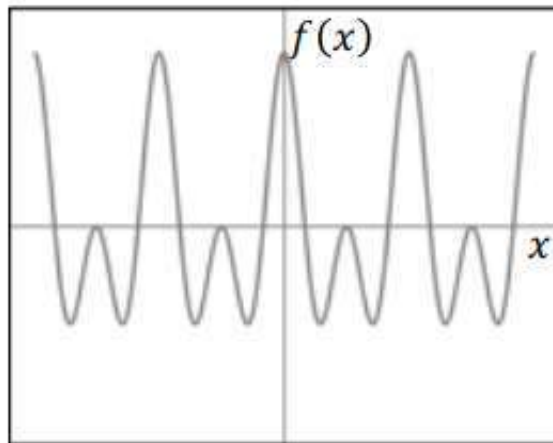
$$\cos(x) = \frac{e^{ix} + e^{-ix}}{2}$$

$$F(u) = \frac{1}{2}[\delta(u + k) + \delta(u - k)]$$

Fourier transform examples:

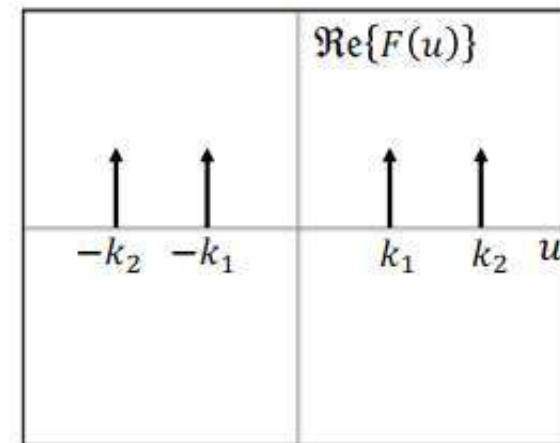


Signal $f(x)$



$$f(x) = \cos 2\pi k_1 x + \cos 2\pi k_2 x$$

Fourier Transform $F(u)$

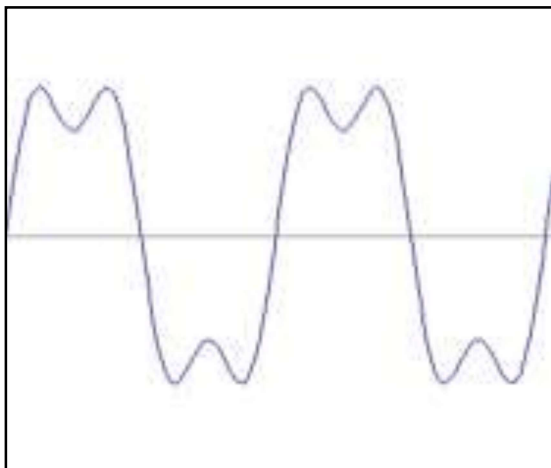


$$F(u) = \frac{1}{2} [\delta(u + k_1) + \delta(u - k_1) + \delta(u + k_2) + \delta(u - k_2)]$$

Square wave generation:



How would you generate this function?



=

?

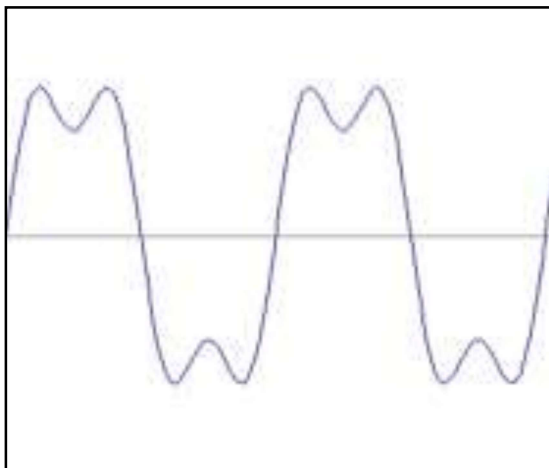
+

?

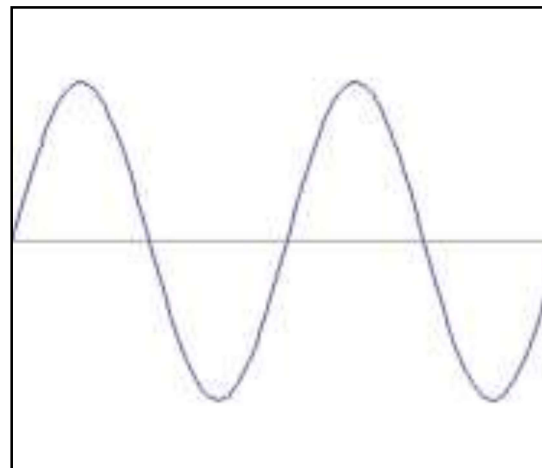
Square wave generation:



How would you generate this function?



=



$\sin(2\pi x)$

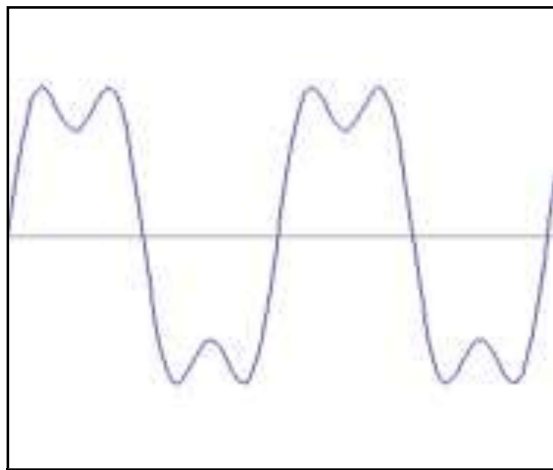
+

?

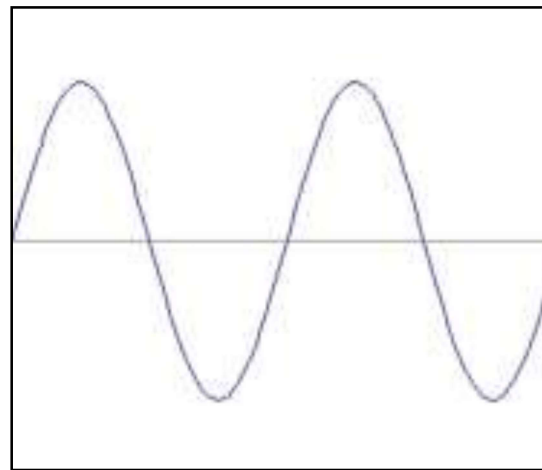
Square wave generation:



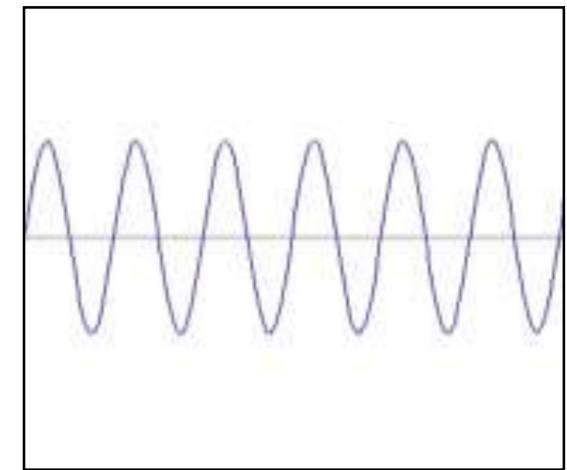
How would you generate this function?



=



+



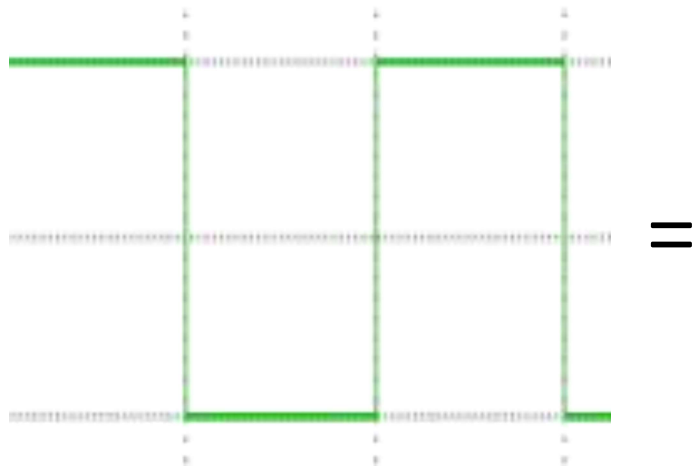
$$f(x) = \sin(2\pi x) + \frac{1}{3} \sin(2\pi 3x)$$

$$\sin(2\pi x)$$

$$\frac{1}{3} \sin(2\pi 3x)$$

Generating square wave:

How would you generate this function?



=

?

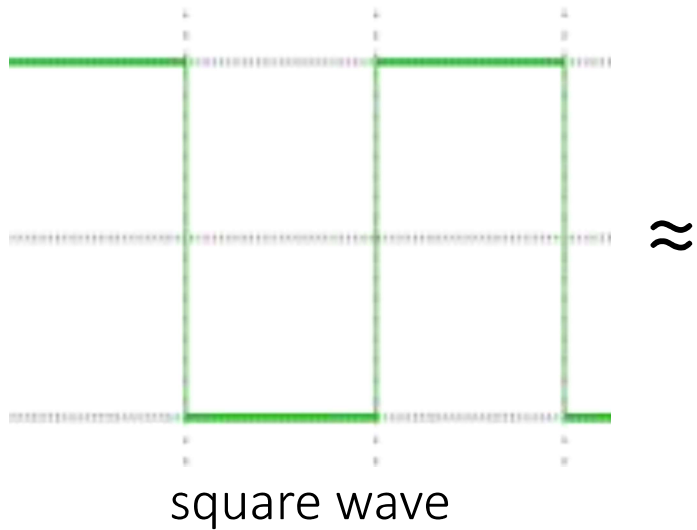
+

?

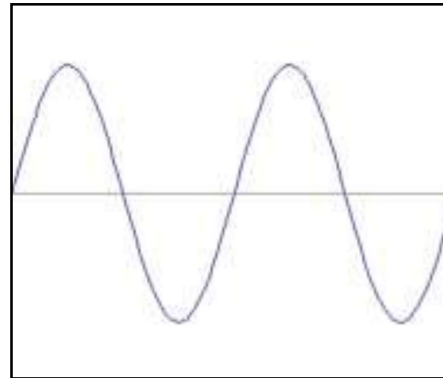
square wave

Generating square wave:

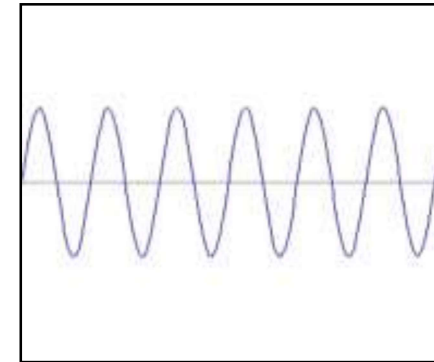
How would you generate this function?



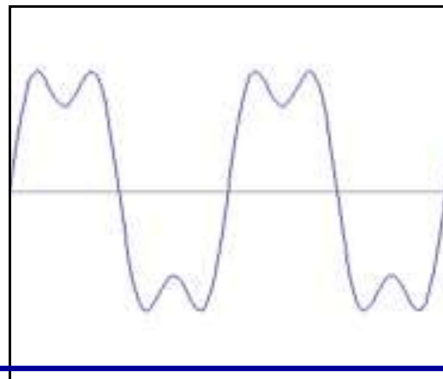
$\approx$



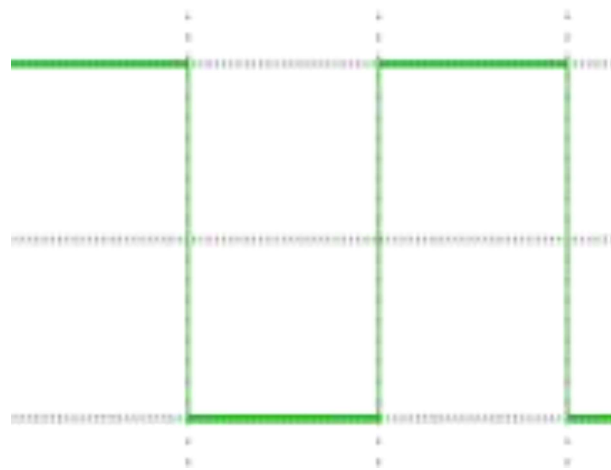
+



$=$

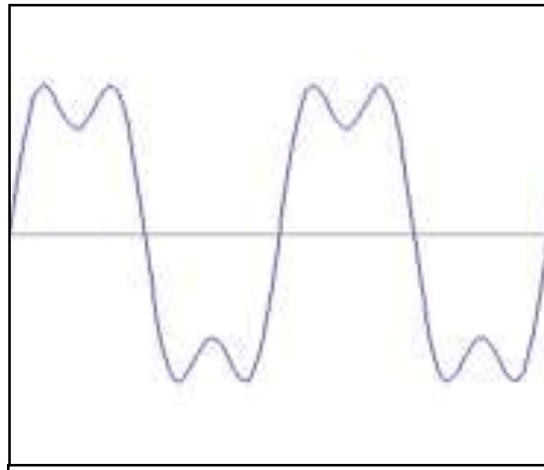


Generating square wave:

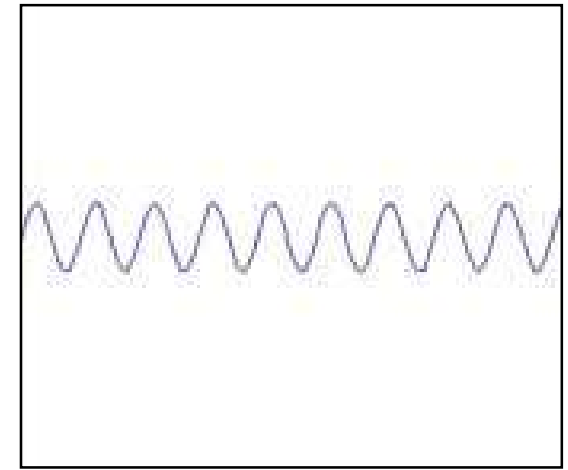


square wave

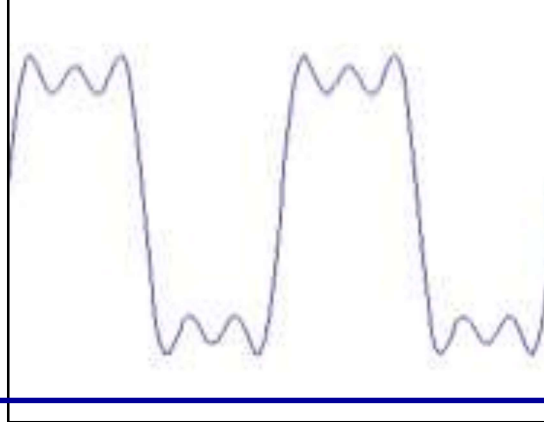
$\approx$



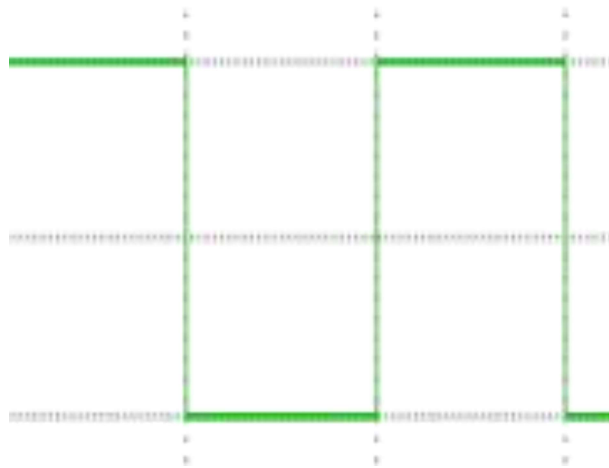
+



$=$

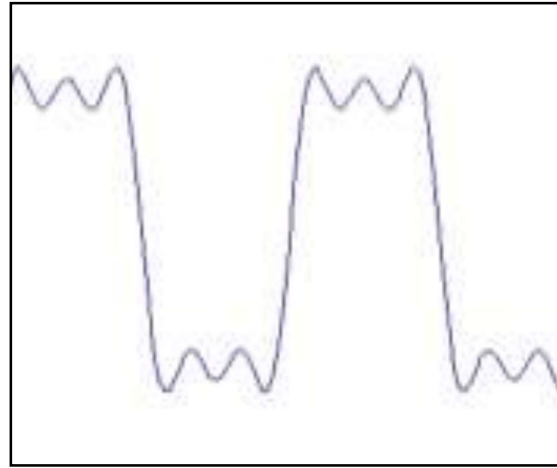


Generating square wave:

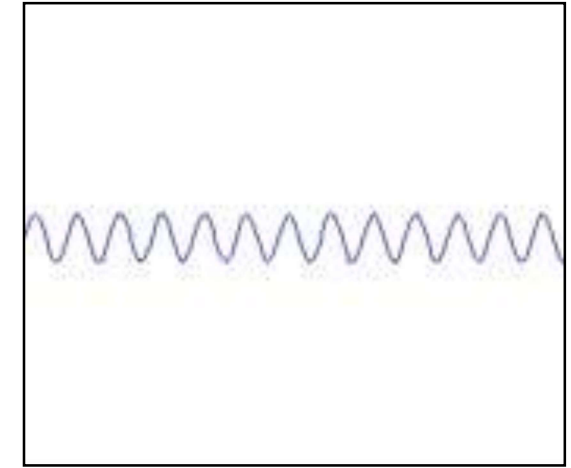


square wave

$\approx$



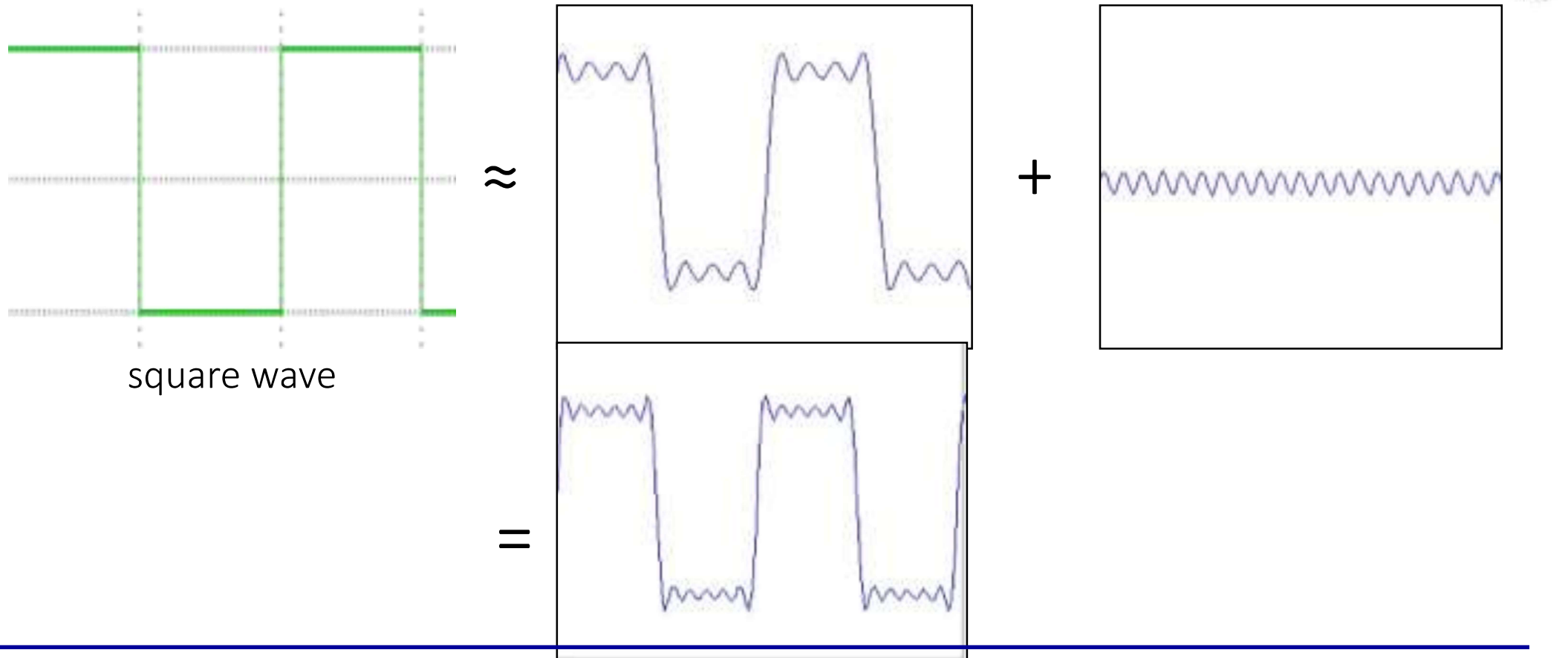
+



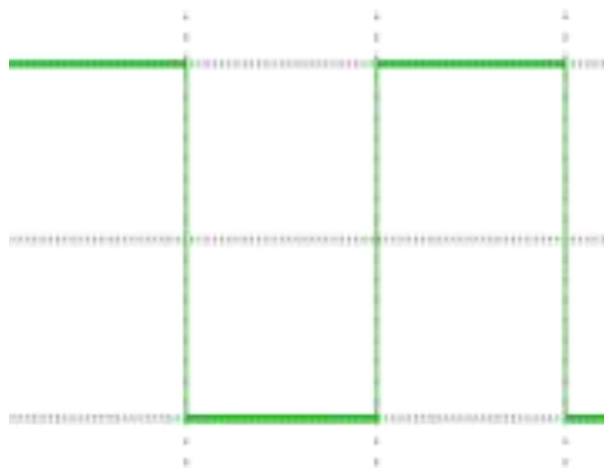
$=$



Basic building block:

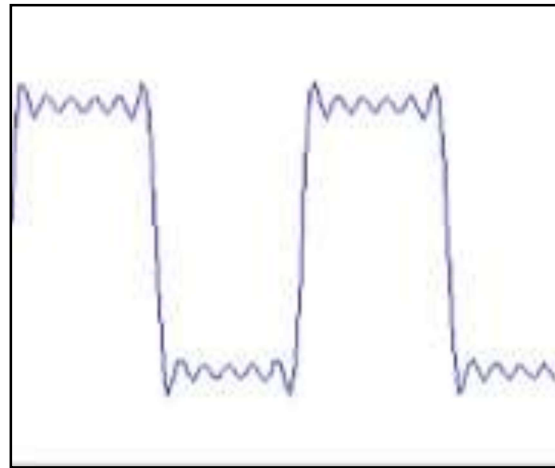


Generating square wave:

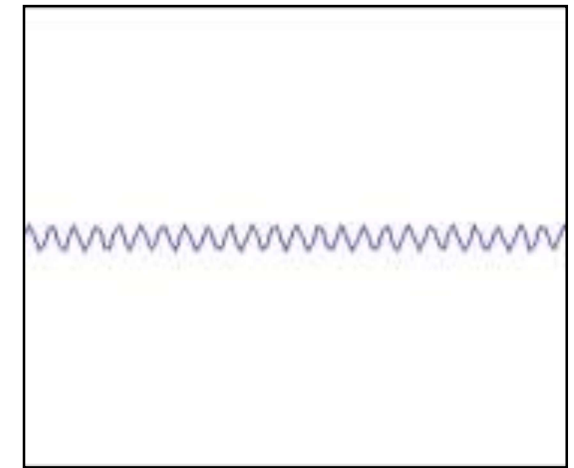


square wave

$\approx$



+

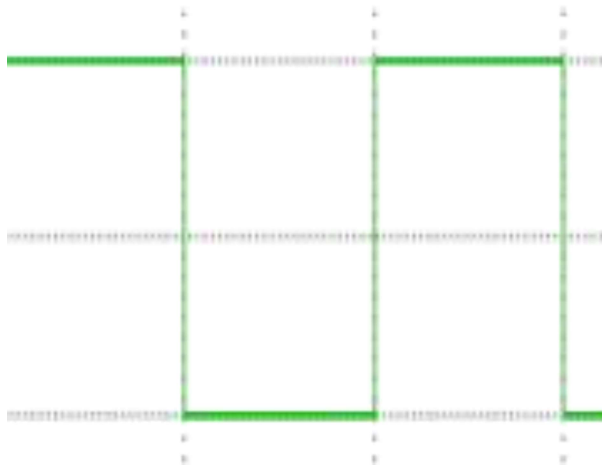


$=$



How would you express
this mathematically?

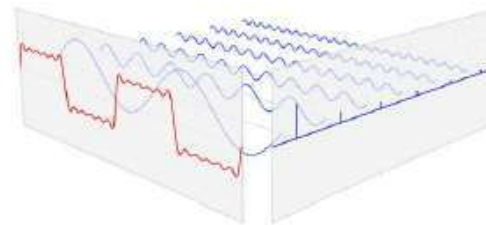
Square wave equation:



square wave

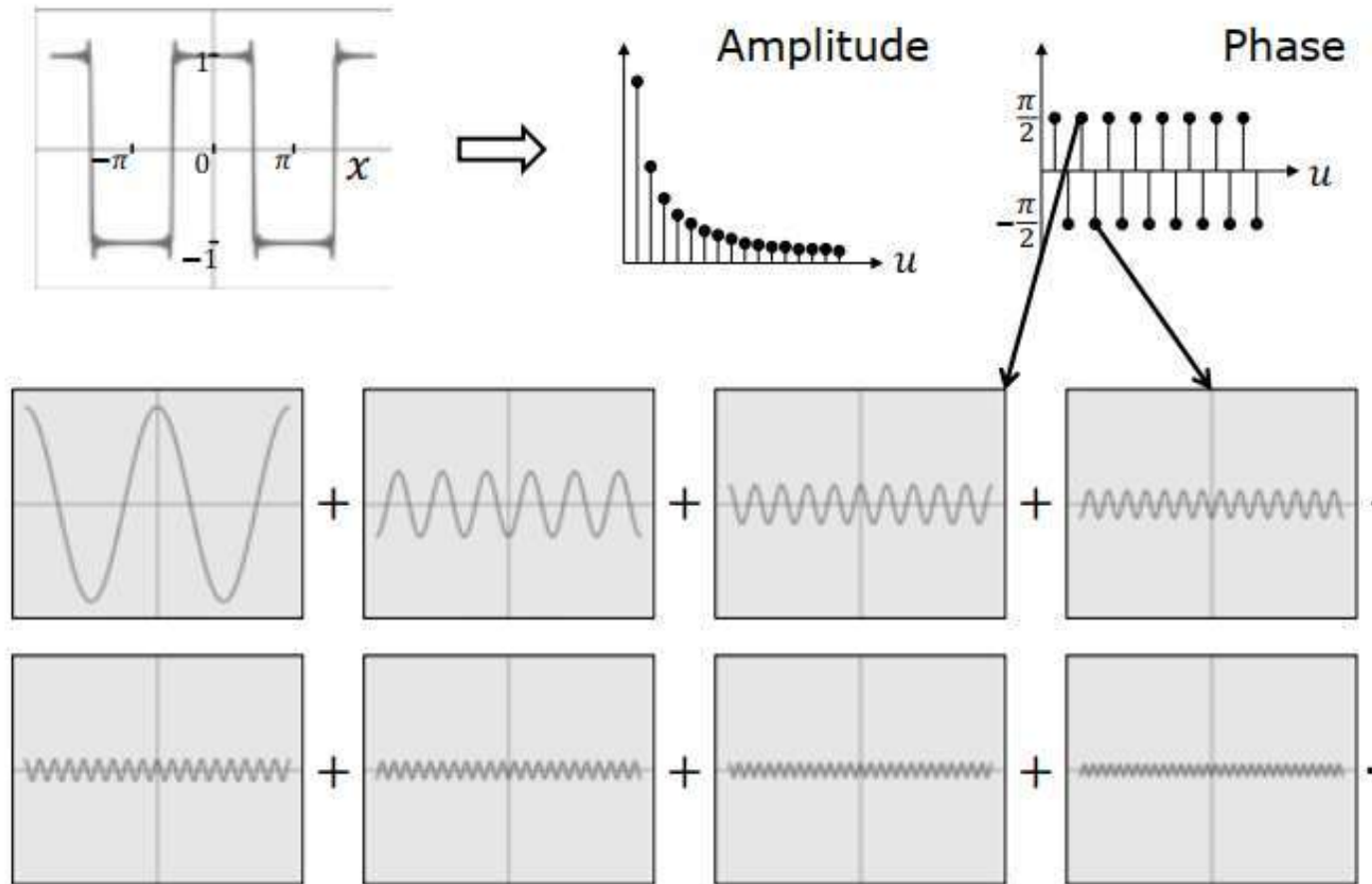
=

$$A \sum_{k=1}^{\infty} \frac{1}{k} \sin(2\pi kx)$$



infinite sum of sine waves

Frequency spectrum of Square wave:

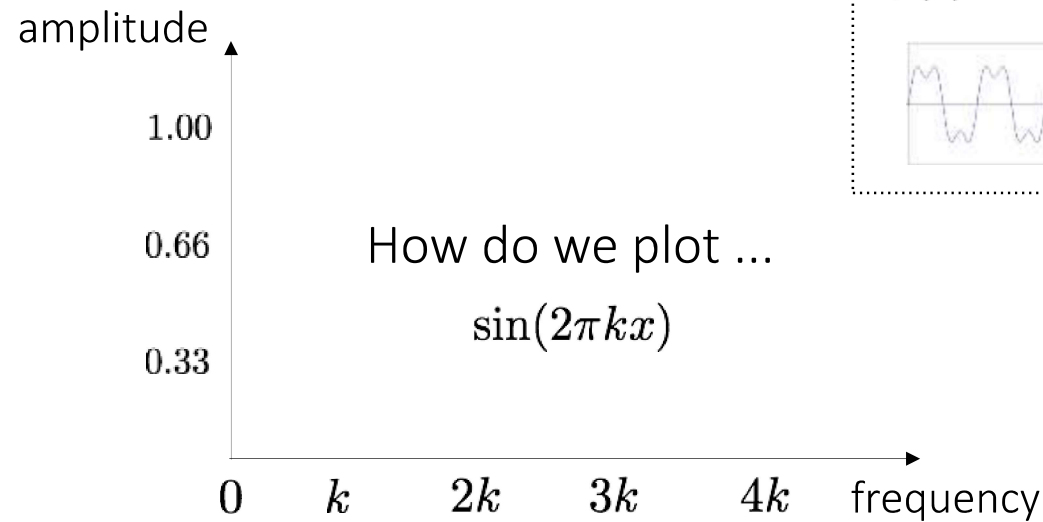




Frequency domain

Visualizing Frequency Spectrum:

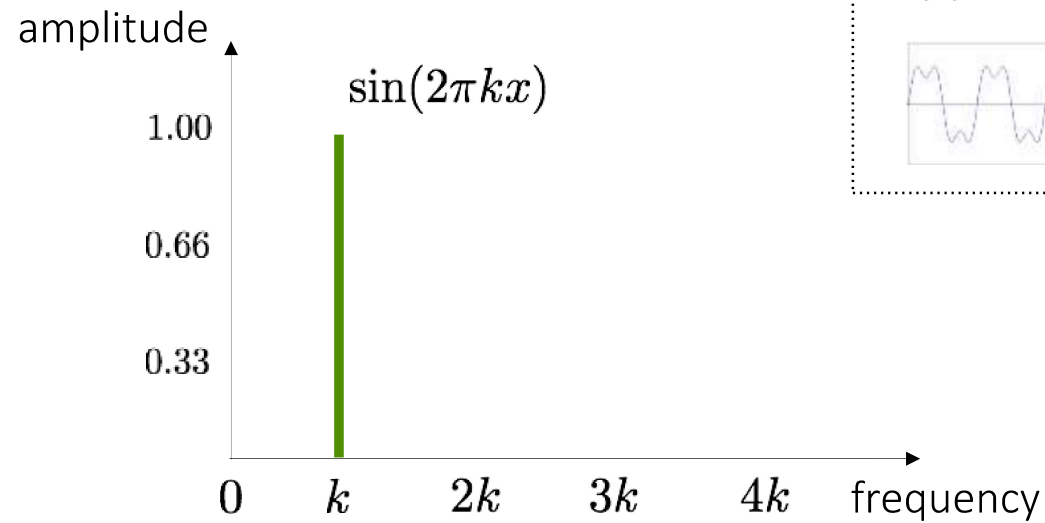
Recall the temporal domain visualizati

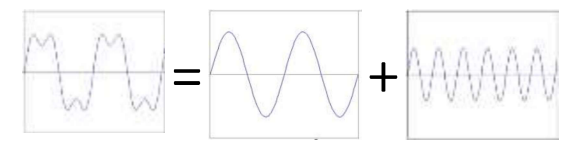


$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

Visualizing Frequency Spectrum:

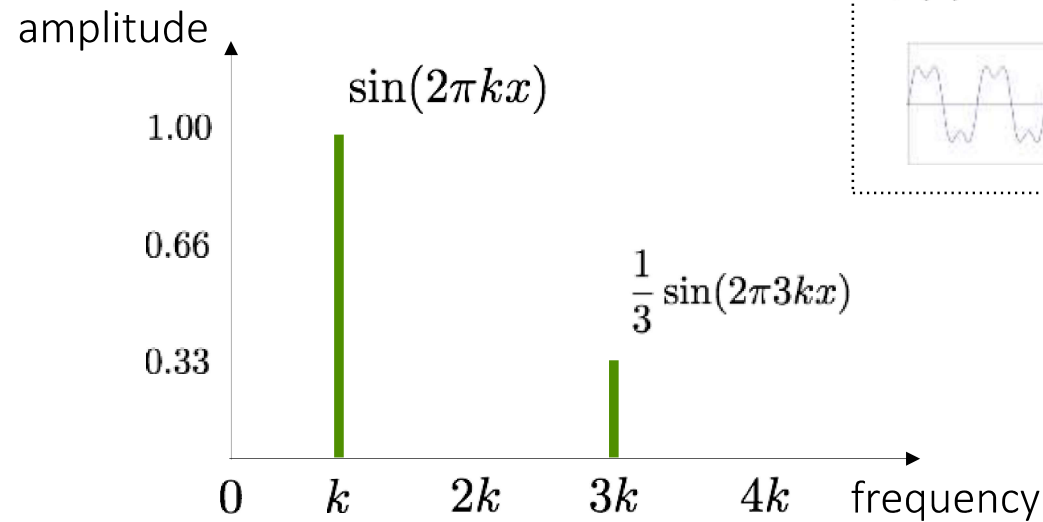
Recall the temporal domain visualizati



$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$


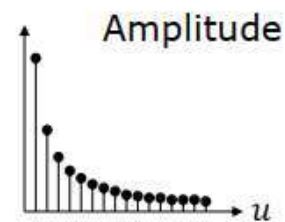
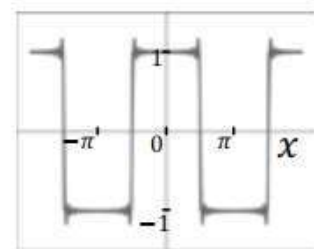
Visualizing Frequency Spectrum:

Recall the temporal domain visualizati

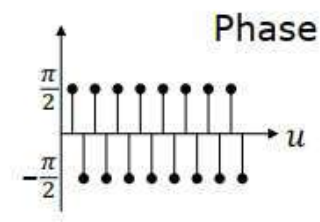


$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

Fourier Transform (FT):



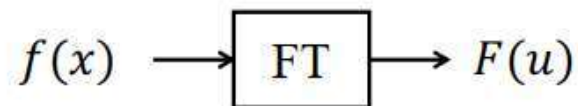
The magnitude represents
the strength of each
frequency component



phase represents its position
or location and orientation
of various frequency
components

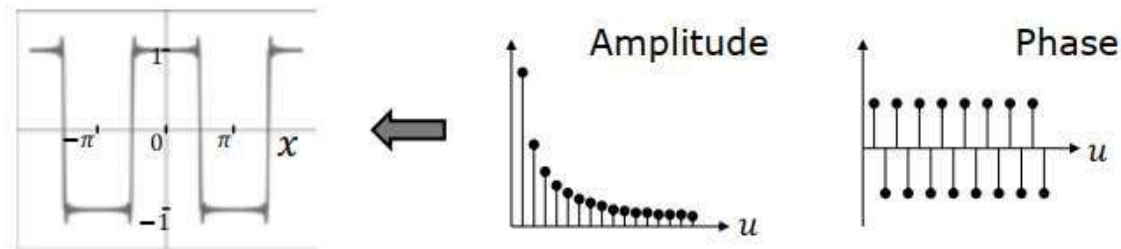


Represents a signal $f(x)$ in terms of Amplitudes and Phases of its Constituent Sinusoids.



$$f(x) \leftarrow \boxed{\text{IFT}} \leftarrow F(u)$$

Inverse fourier transform (IFT): Image Reconstruction



Computes the **signal** ($f(x)$) from the **Amplitudes** and **Phases** of its Constituent Sinusoids.

Image Reconstruction:

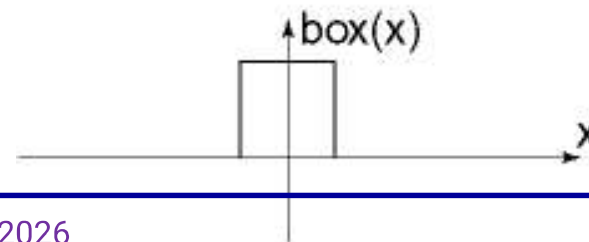
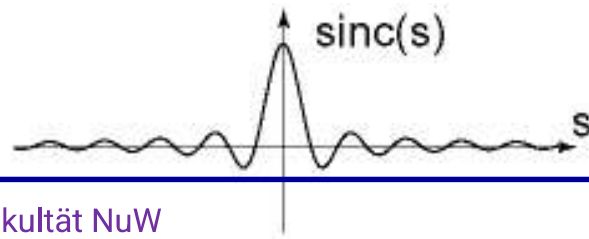
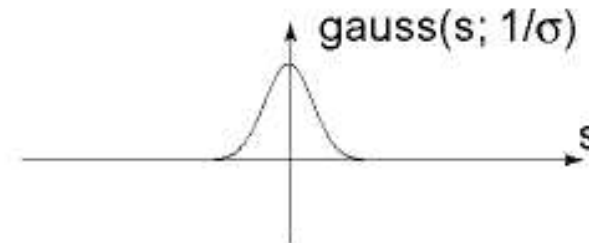
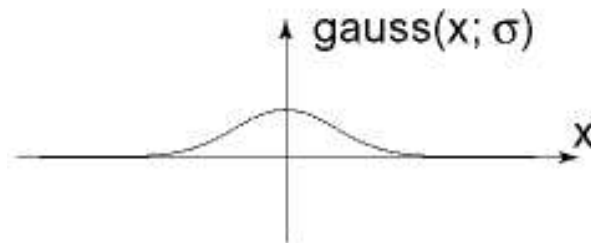
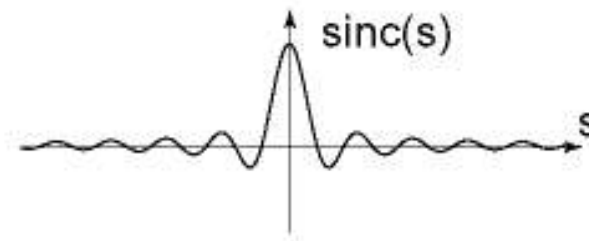
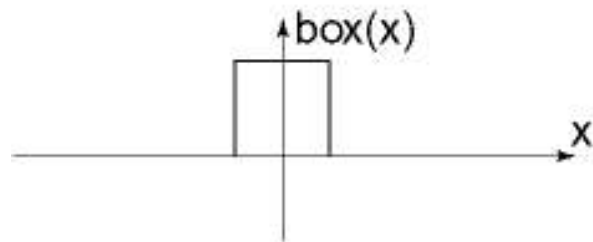
Both **magnitude** and **phase** are necessary for the **complete reconstruction** of the **original image** from its frequency domain representation.



Fourier transform pairs

spatial domain

frequency domain

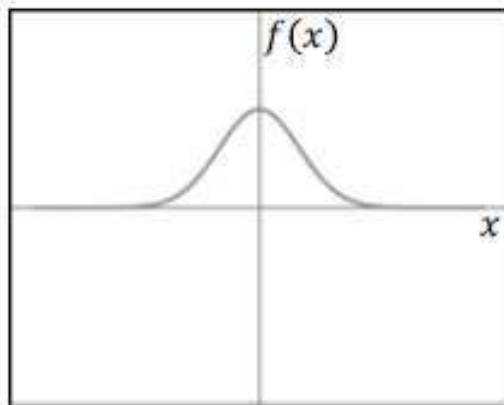


Note :
duality property of
Fourier transform



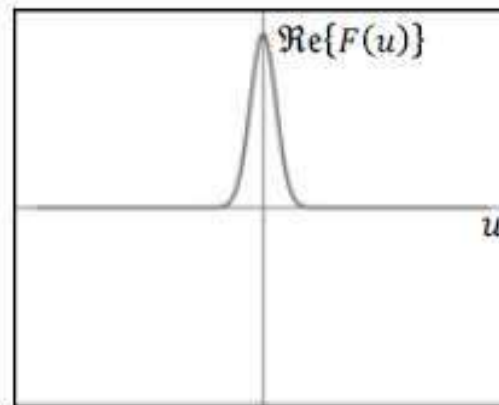
FT(Gaussian)

Signal $f(x)$



$$f(x) = e^{-ax^2}$$

Fourier Transform $F(u)$



$$F(u) = \sqrt{\pi/a} e^{-\pi^2 u^2 / a}$$

FT(Gaussian) is also a Gaussian.

The width is proportional to $1/a$.

For any function, if we widen it in the spatial domain, we actually compress it in the frequency domain and vice versa.

This is referred to as inverse scaling.

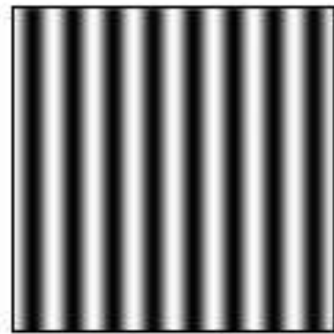


Properties of Fourier Transform

Property	Spatial Domain	Frequency Domain
Linearity	$\alpha f_1(x) + \beta f_2(x)$	$\alpha F_1(u) + \beta F_2(u)$
Scaling	$f(ax)$	$\frac{1}{ a } F\left(\frac{u}{a}\right)$
Shifting	$f(x - a)$	$e^{-i2\pi ua} F(u)$
Differentiation	$\frac{d^n}{dx^n} (f(x))$	$(i2\pi u)^n F(u)$



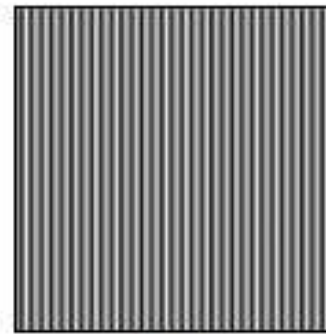
2D Fourier Transform: Example



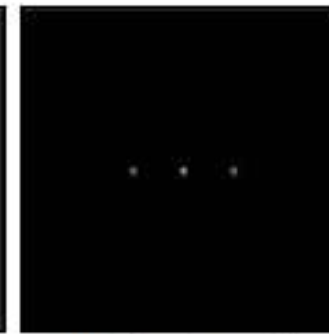
$f(m, n)$



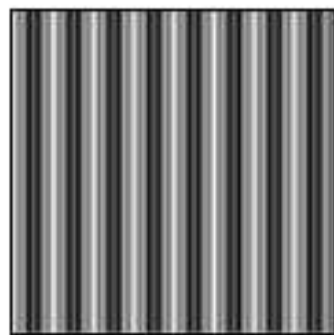
$\log(|F(p, q)|)$



$g(m, n)$



$\log(|G(p, q)|)$



$f(m, n) + g(m, n)$



$\log(|F(p, q) + G(p, q)|)$

compress the wide range
of magnitude values into
a smaller range

Note: $\log()$ is used just for display

How would you generate this Image?:

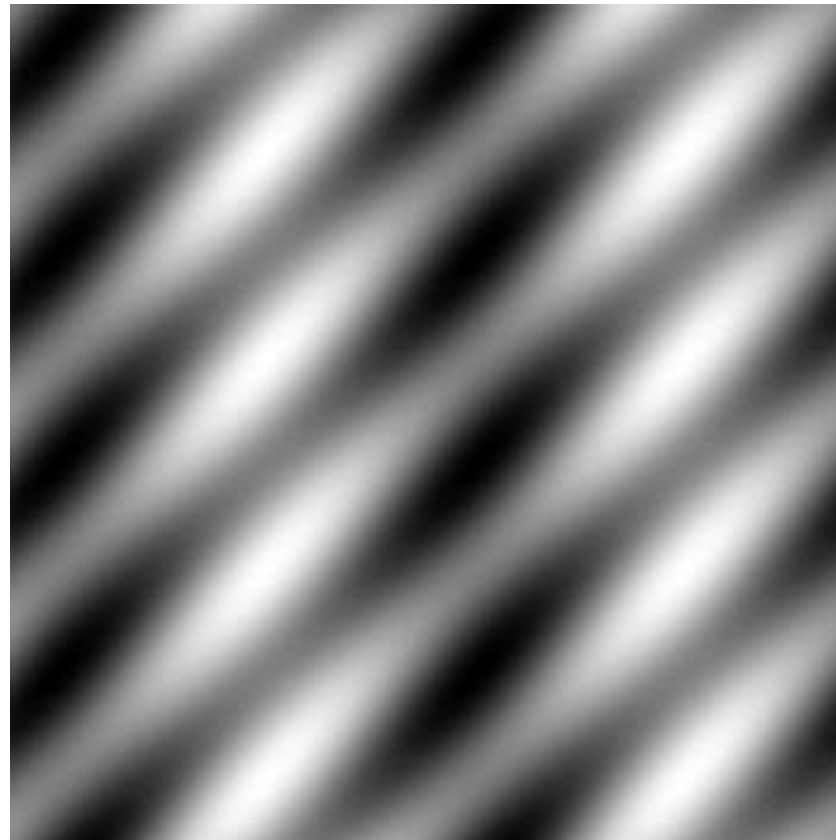
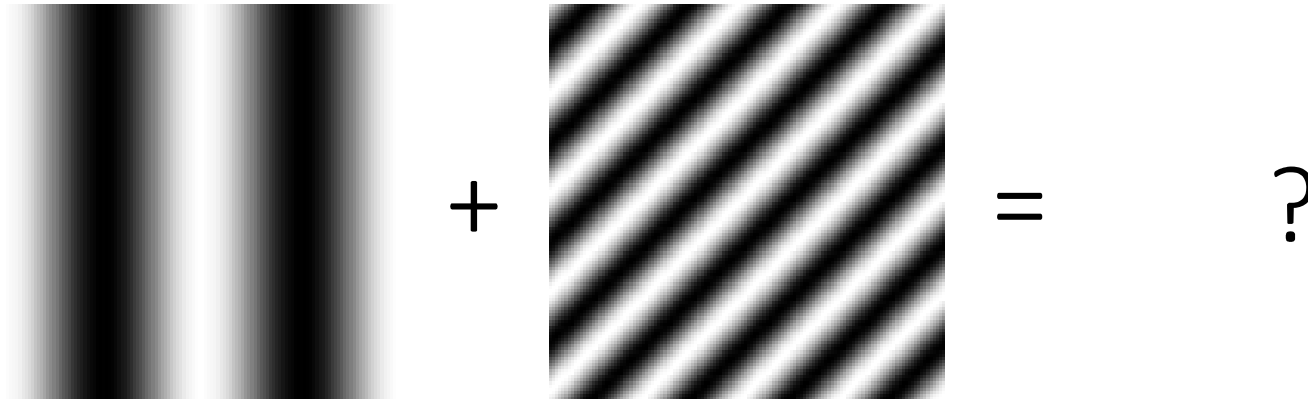
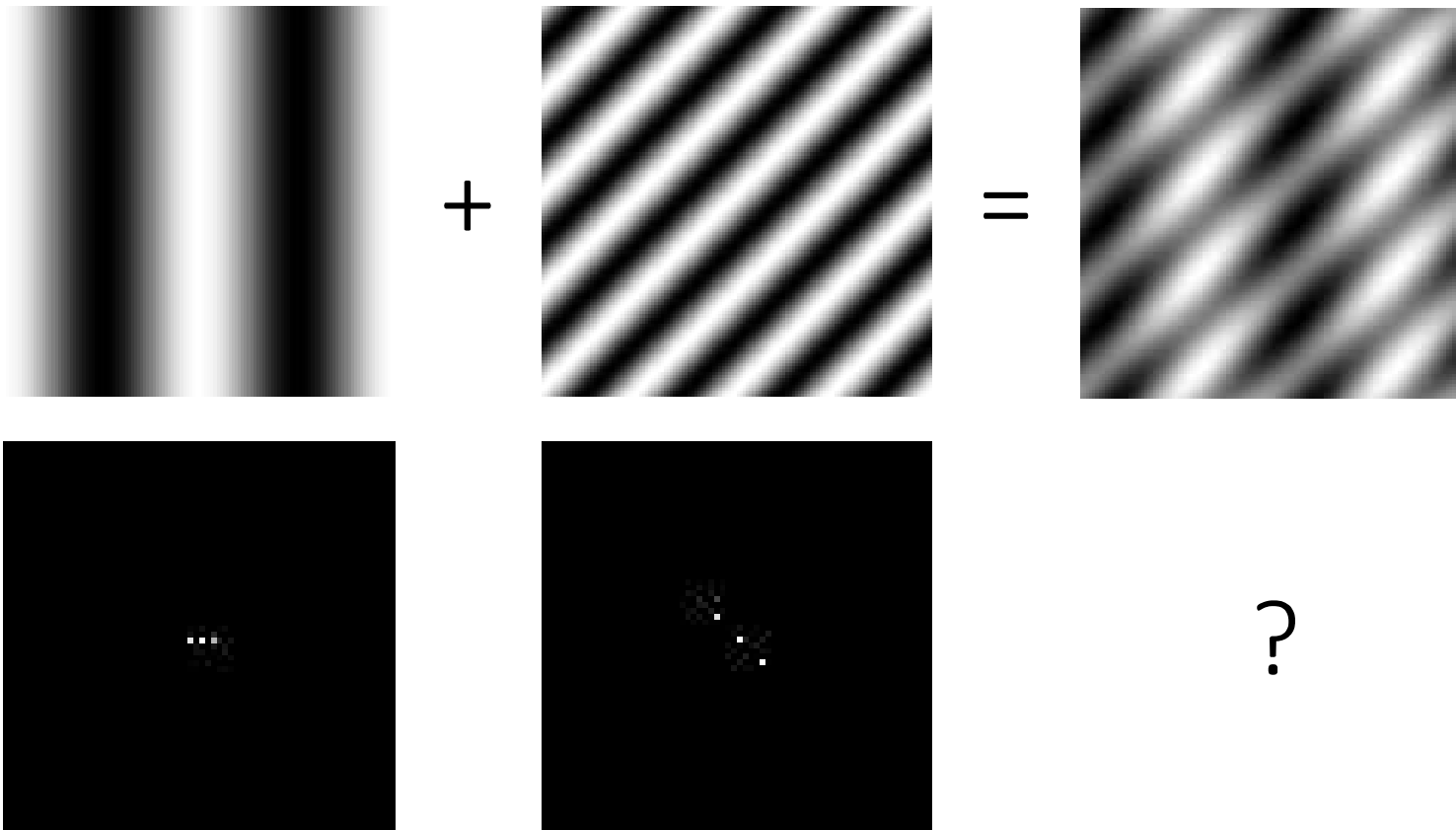


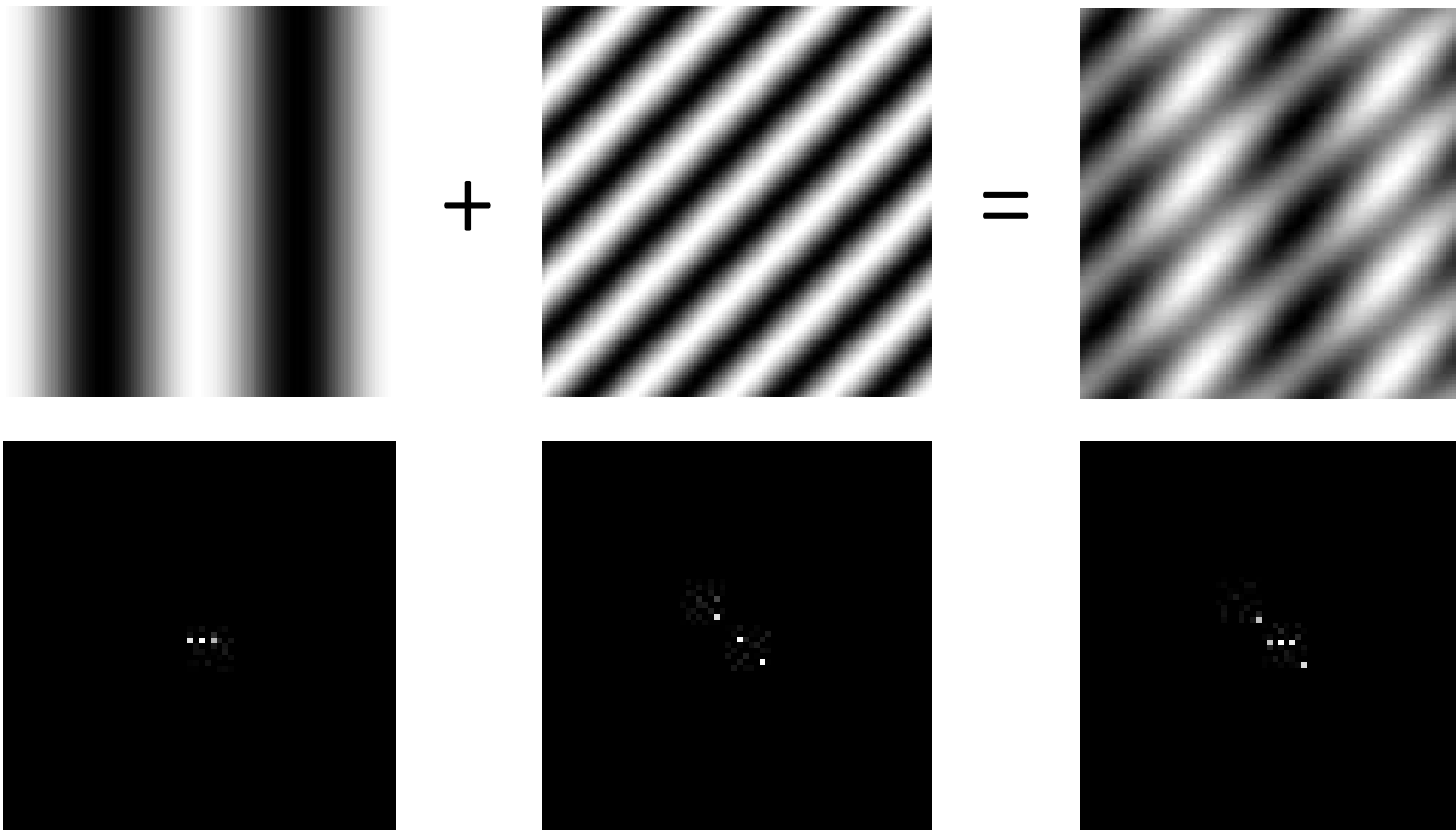
Image Generation and Frequency Domain Visualization:



Phase Importance:

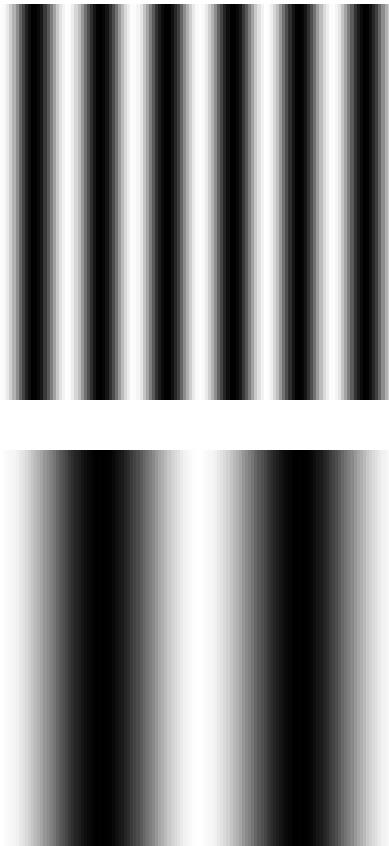


Frequency Domain Visualization:

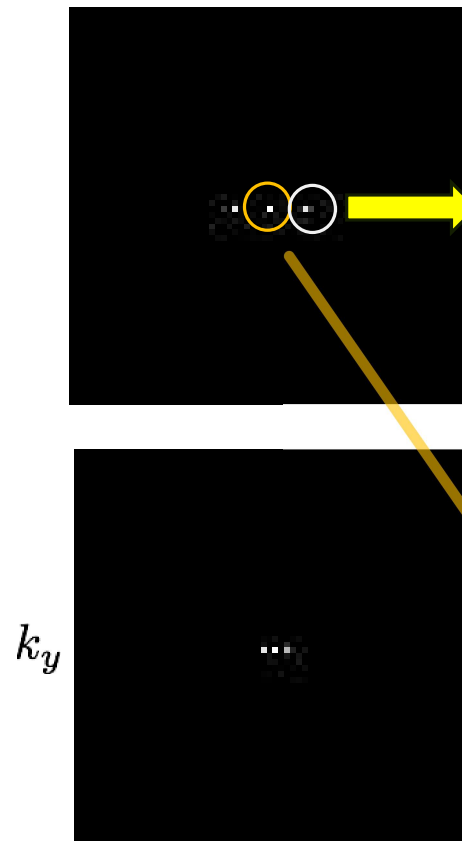


Frequency Domain Visualization:

Spatial domain visualization



Frequency domain visualization



The bright pixels in frequency image (right) *indicate strong intensity at the corresponding positions.*

The high frequency domain, includes those *sharp edges* or *some structure* with the changes of *intensity dramatically along one orientation* (such as the hairs, boundary of hat, etc)

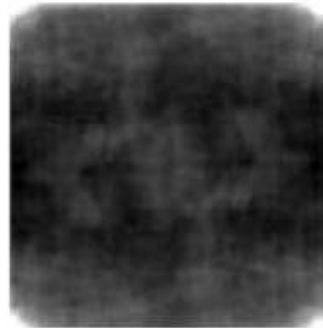
The central part includes the *low frequency components.*

The low frequency domain corresponds to the *smooth areas in the image* (such as skin, hat, background, etc).

Fourier phase importance in reconstruction



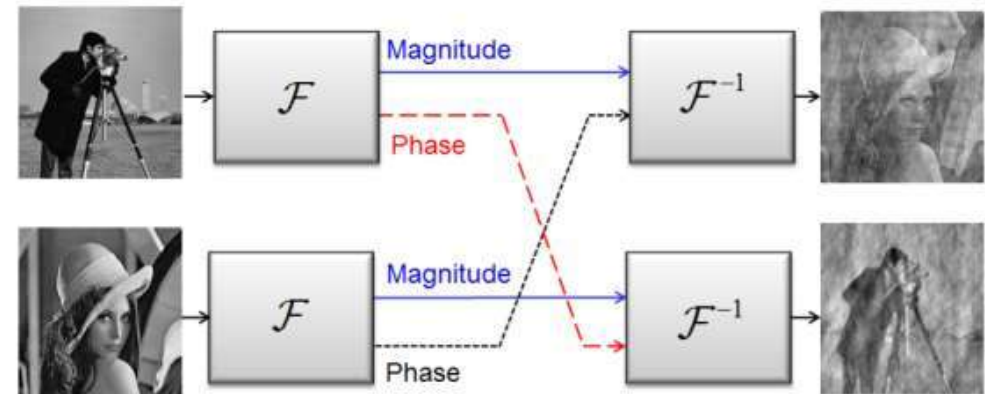
Original Image



Magnitude Preserved,
Phase Set to Zero



Phase Preserved,
Magnitude Set to Average
of Natural Images



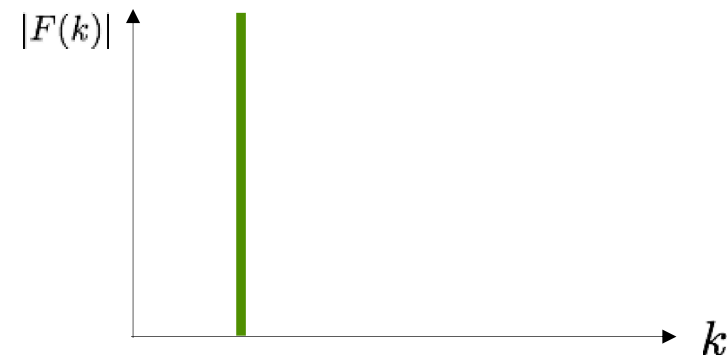
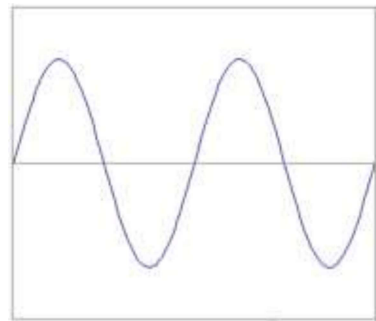
Frequency Domain Visualization:



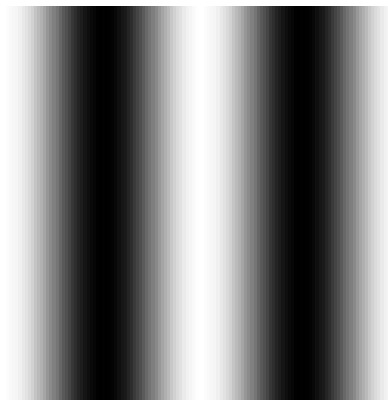
Spatial domain visualization

Frequency domain visualization

1D



2D



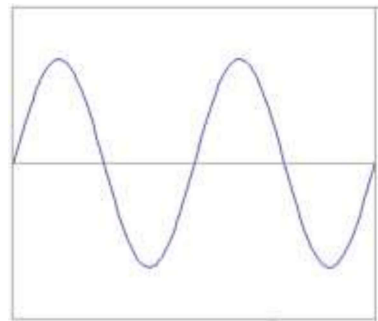
?

Frequency Domain Visualization:

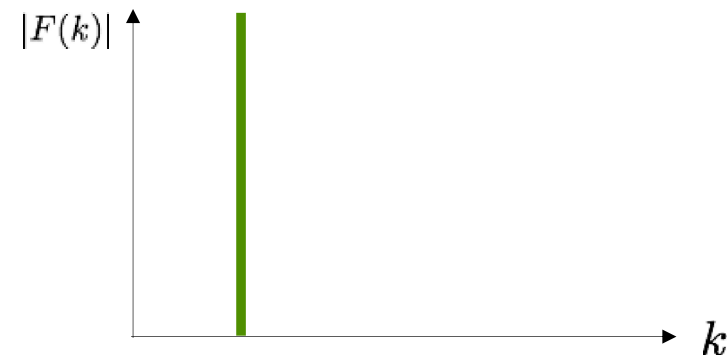


Spatial domain visualization

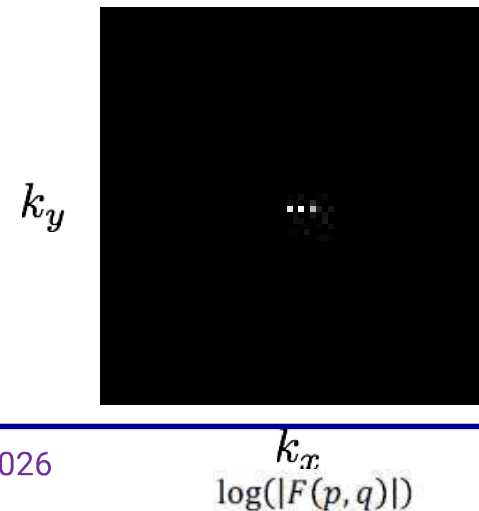
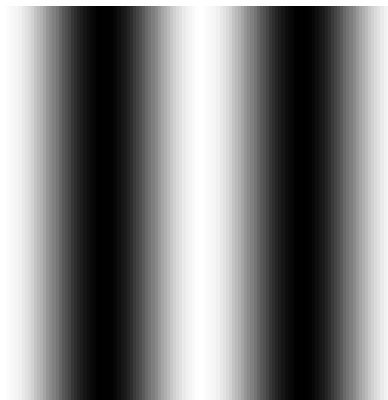
1D



Frequency domain visualization



2D

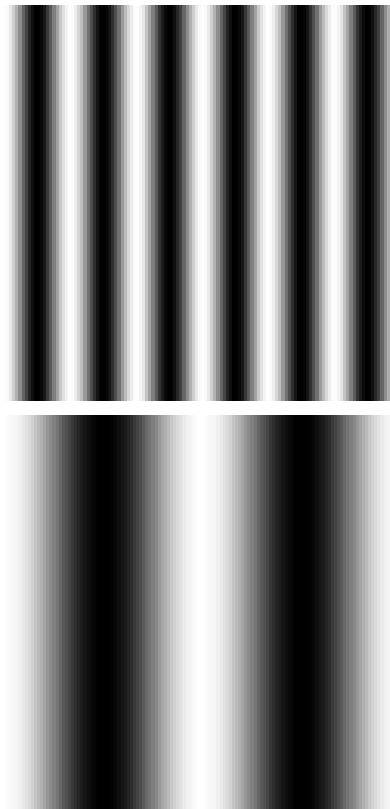


What do the three dots correspond to?

Frequency Domain Visualization:



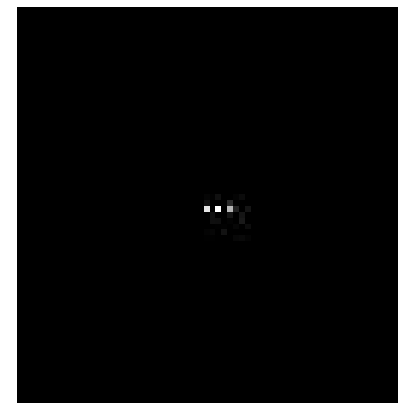
Spatial domain visualization



Frequency domain visualization

?

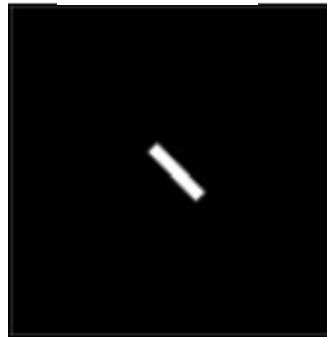
k_y



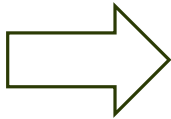
k_x
 $\log(|F(p, q)|)$

2D-Fourier Transform: Example 1

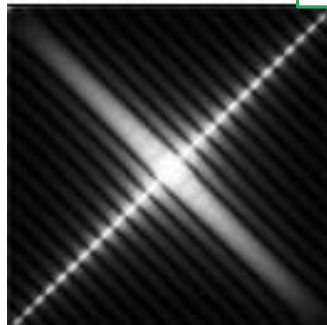
$f(m, n)$



2D-FT



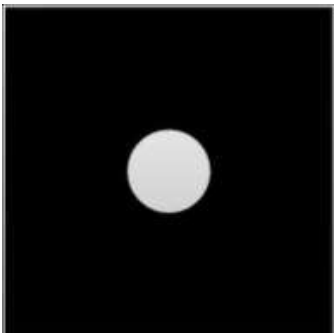
$\log(|F(p, q)|)$



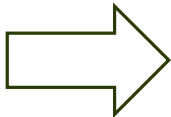
the intensity at each point in the transform image is proportional to the log of the magnitude of the transform value for the corresponding frequency

we get very strong frequencies along two lines.

we have two pairs of strong edges and to re-create each edge we need frequencies in the direction perpendicular to the edge



2D-FT



The flat part of the disk produces some low frequencies.

The edges produce high frequencies.

Note

image is rotationally symmetric, its Fourier transform is rotationally symmetric as well.

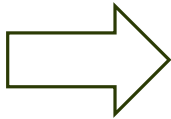
Min  Max

2D-Fourier Transform: Example 2

$f(m, n)$



2D-FT



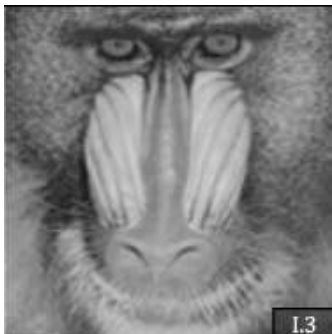
$\log(|F(p, q)|)$



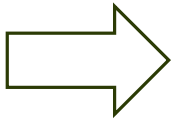
the intensity at each point in the transform image is proportional to the log of the magnitude of the transform value for the corresponding frequency

we see three strong lines in the Fourier transform, indicating high frequencies along these lines.

This is because the original image has edges along three dominant directions: vertical edges, horizontal edges, and slanted edges



2D-FT



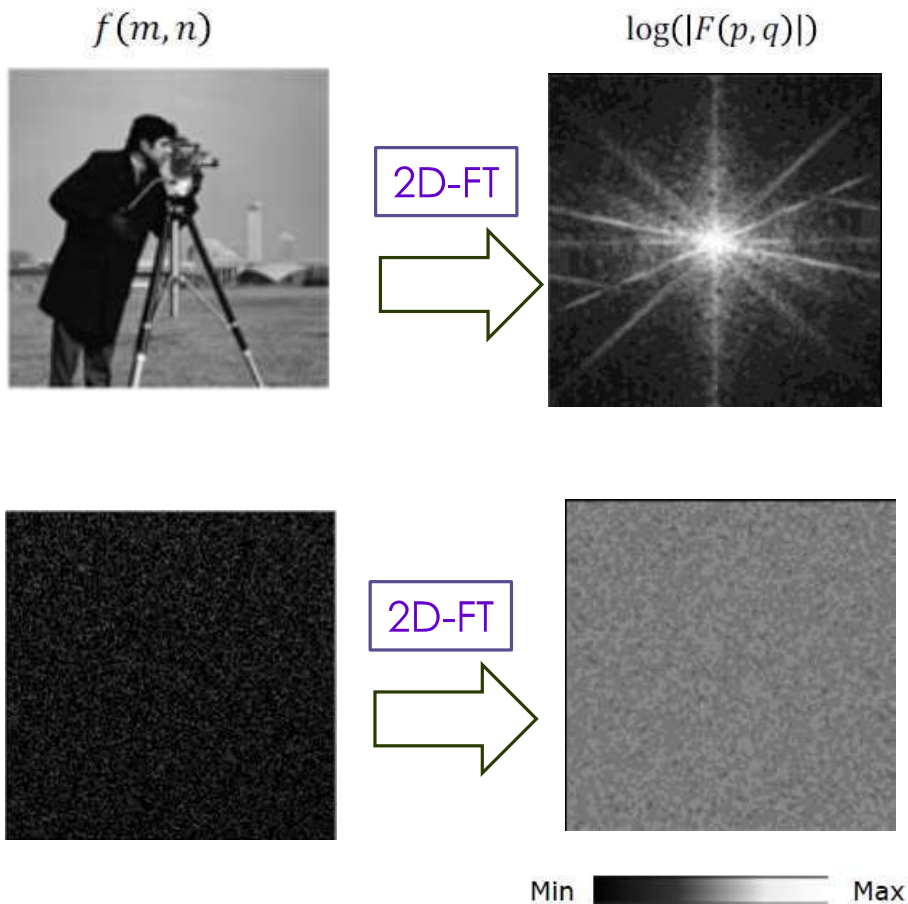
Mandrill, there is not really much that can be said.

The image is complex it is hard to interpret its transform.

As humans, we are not wired to easily interpret signals in frequency domain

Min  Max

2D-Fourier Transform: Example 3



more complex image with lots of strong edges

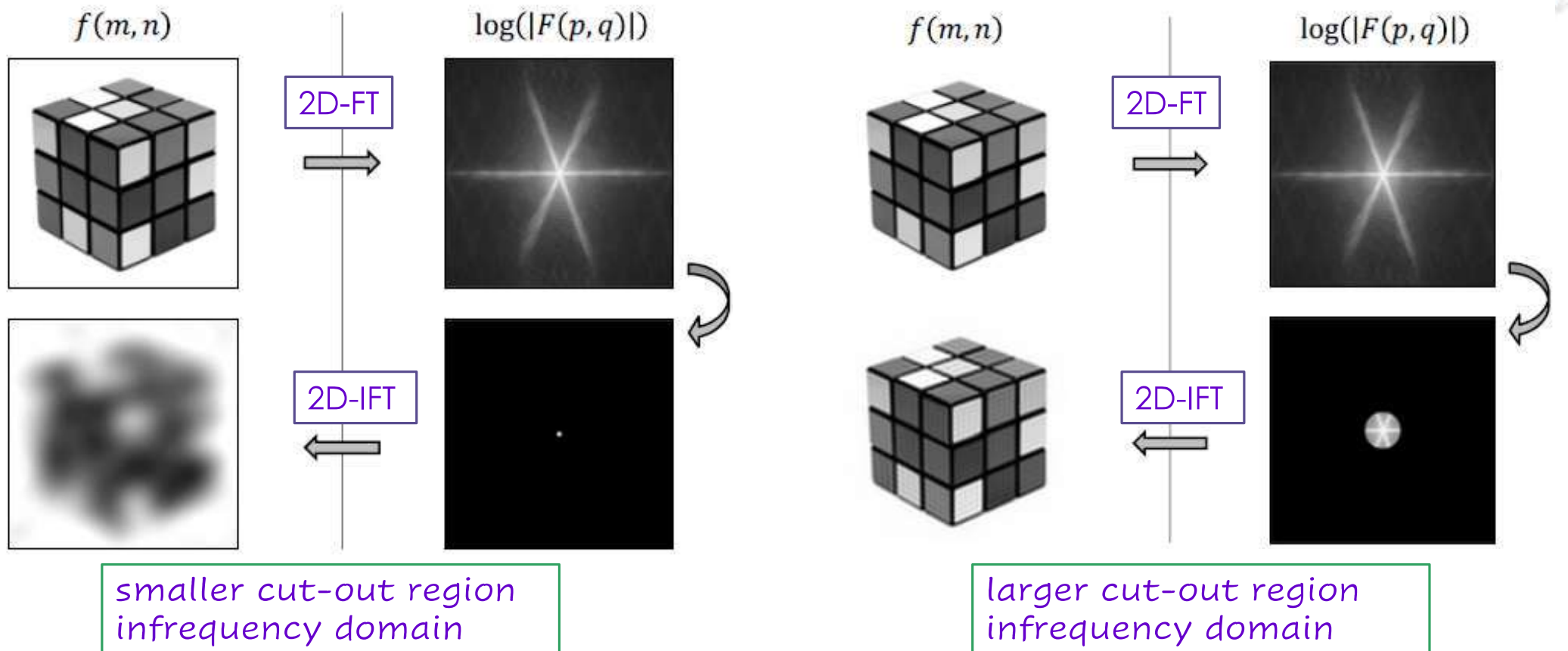
The image which is just noise.

In this case we see that we get very strong values in the Fourier transform for high frequencies.

we want to low-pass filter this image, we can simply cut out the outer parts of the Fourier transform.

Low Pass Filtering

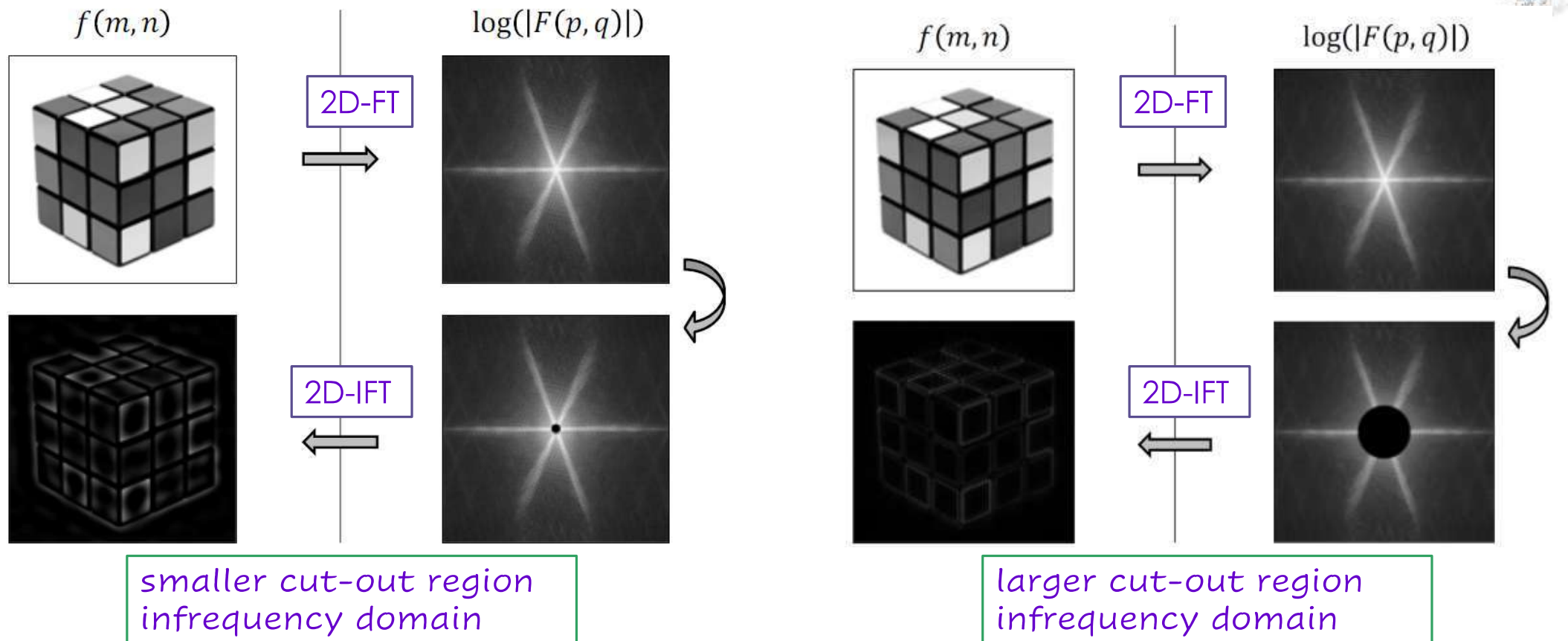
Taking the inverse Fourier transform, we see that we get a blurrier (smoother) version of the image



High Pass Filtering

high-pass filter the image. We eliminate the low frequencies and keep the high frequencies.

Remove all of the constant brightness regions in the image, leaving us with the regions where there is a rapid change in brightness

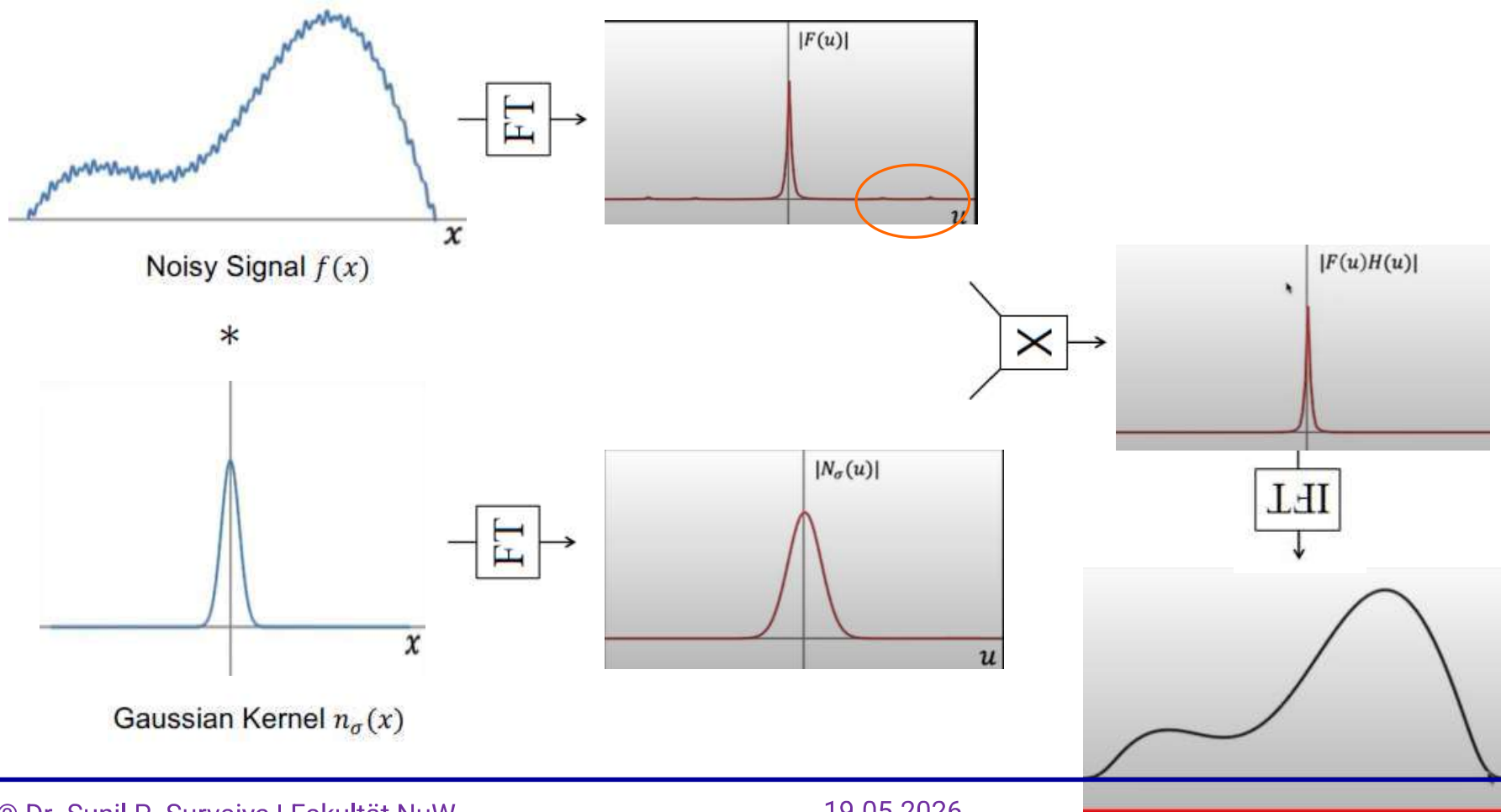


Convolution and Fourier Transform:

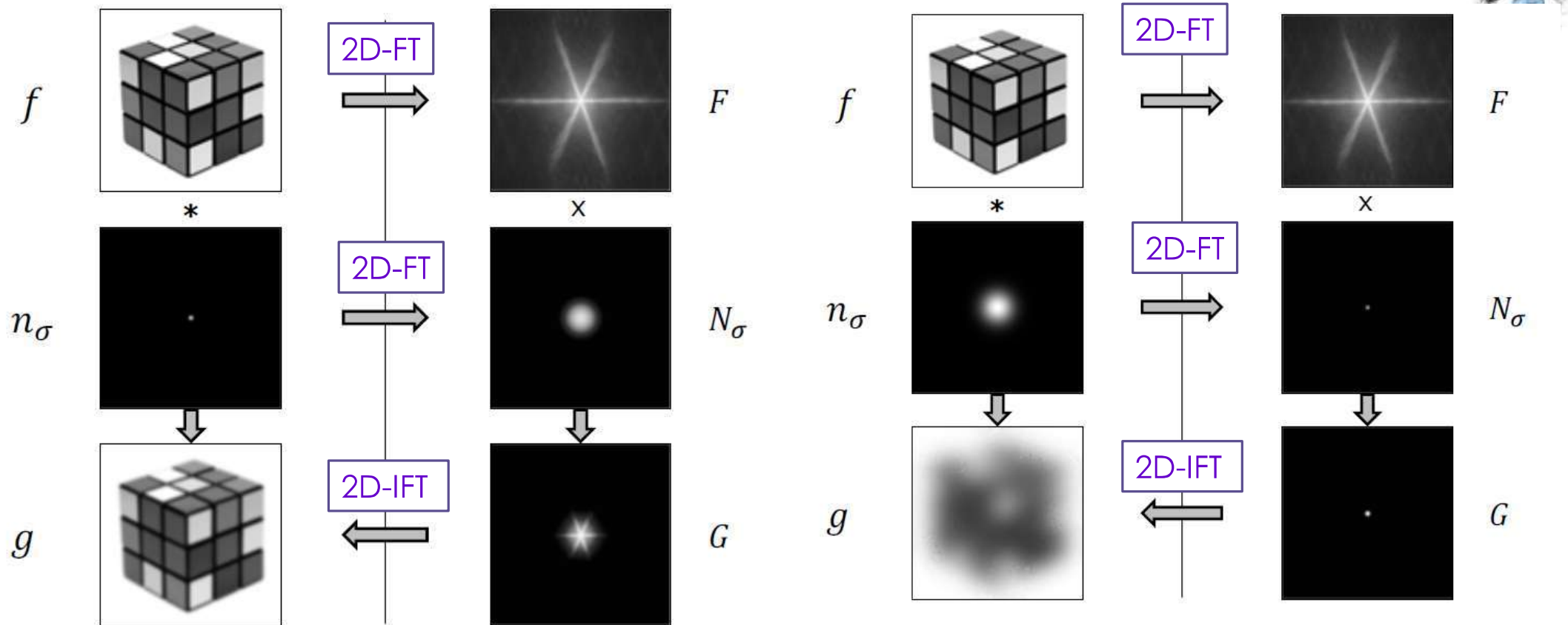


$$\begin{array}{ccccc}
 g(x) & = & f(x) & * & h(x) \\
 \uparrow & & \downarrow & & \downarrow \\
 \boxed{\text{IFT}} & & \boxed{\text{FT}} & & \boxed{\text{FT}} \\
 \downarrow & & \downarrow & & \downarrow \\
 G(u) & = & F(u) & \times & H(u)
 \end{array}$$

Gaussian smooting:

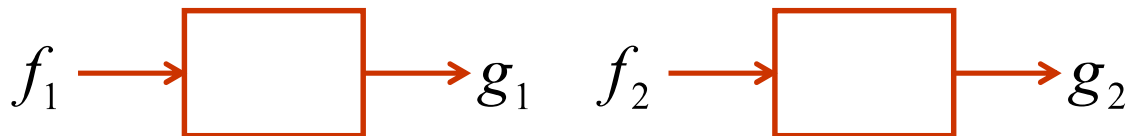


Gaussian Smoothing:



Linear Shift Invariant System (LSIS):

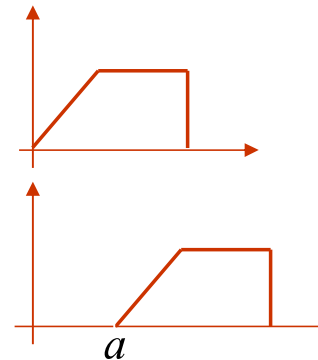
Linearity:



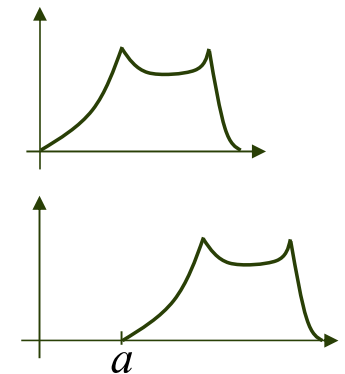
Shift invariance:

Shift Invariance:

$$f(x - a)$$

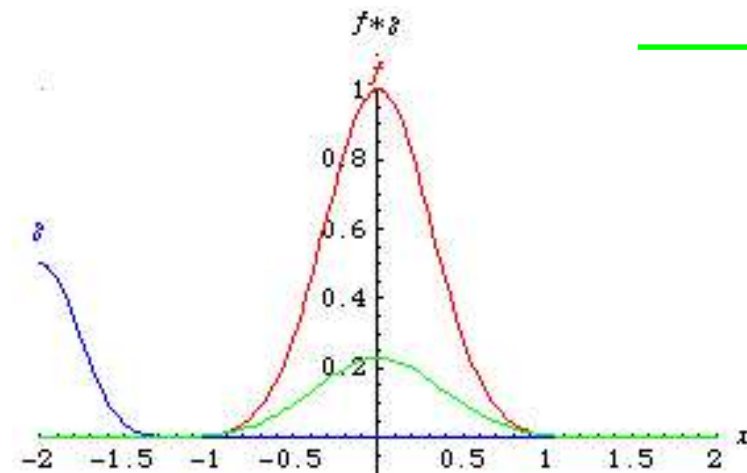
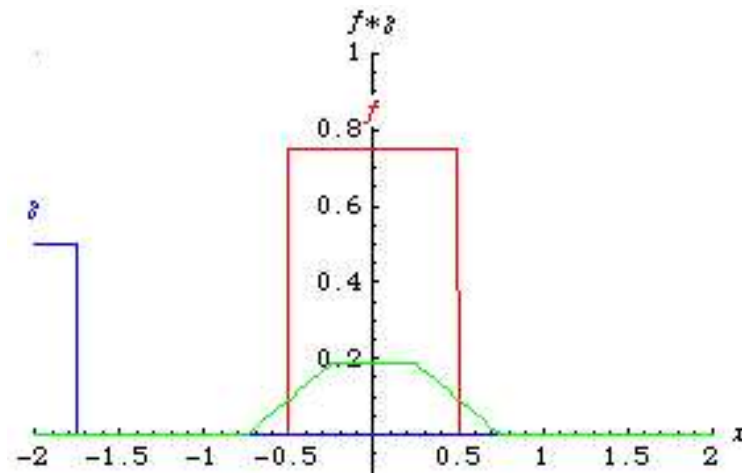


$$g(x - a)$$



293

Convolution: Visual understanding



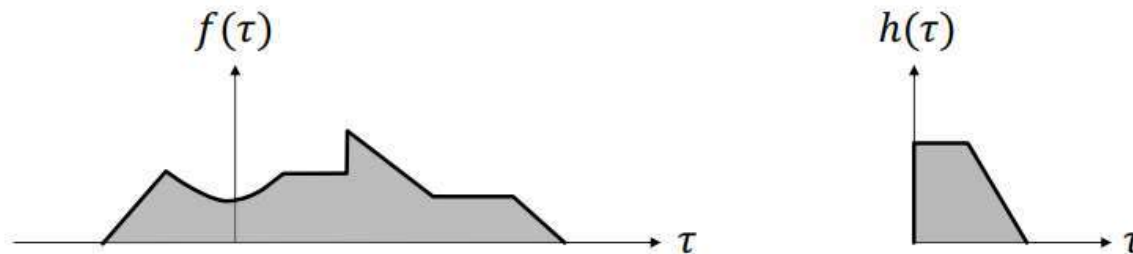
— f
— g
— $f * g$

Convolution:



Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

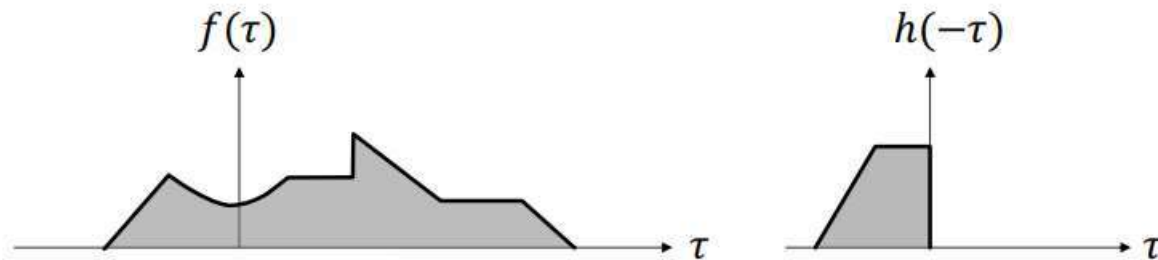


Convolution:



Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

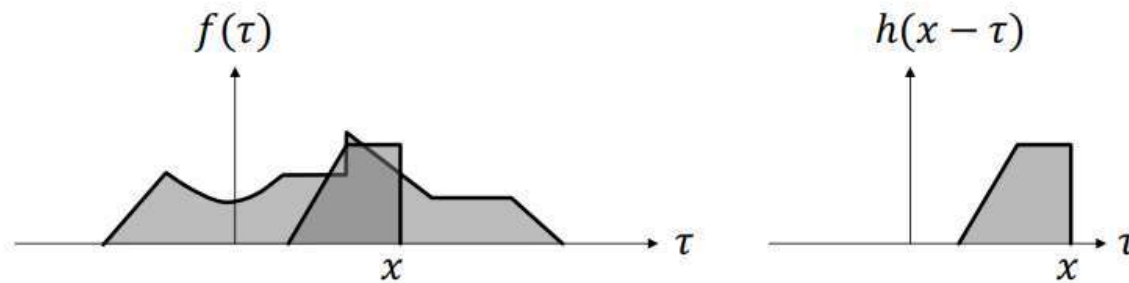


Convolution:



Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

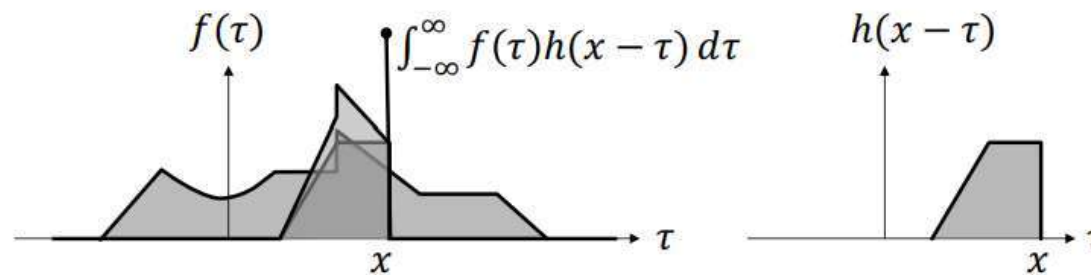


Convolution:



Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

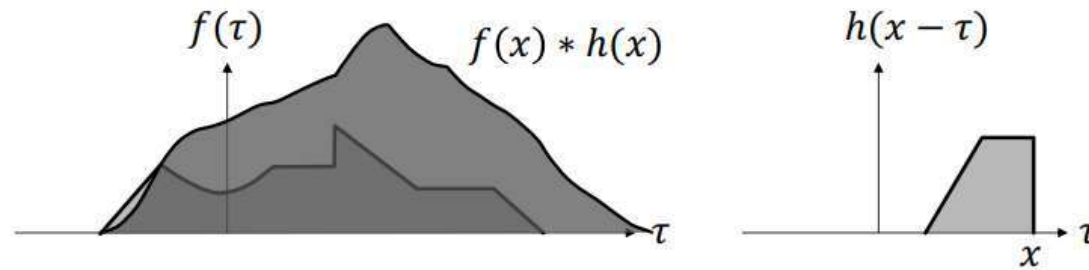


Convolution:



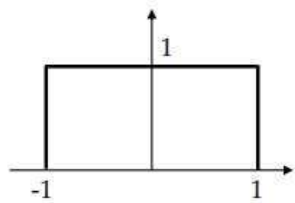
Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

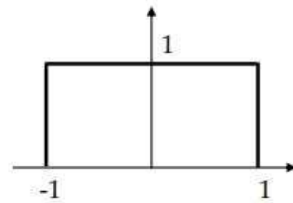


LSIS implies Convolution and Convolution implies LSIS

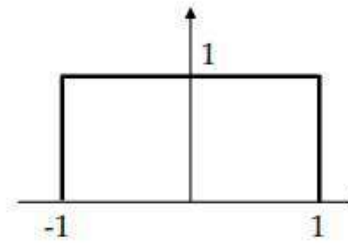
Convolution Examples:



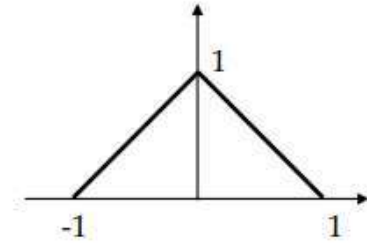
$f(x)$



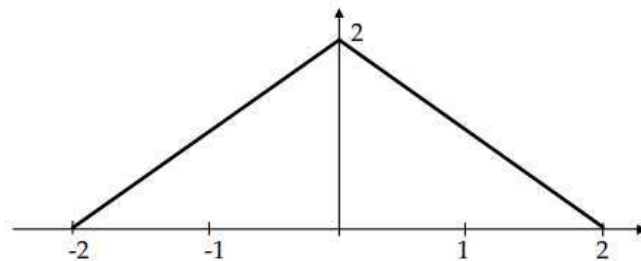
$h(x)$



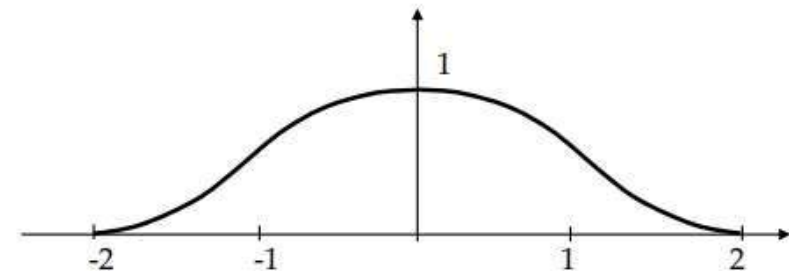
$f(x)$



$h(x)$



$f(x) * h(x)$



$f(x) * h(x)$

Convolution and Fourier Transform:



Convolution: $g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$

Fourier Transform of $g(x)$:

$$G(u) = \int_{-\infty}^{\infty} g(x)e^{-i2\pi ux} dx$$

the convolution of two functions in the time (or spatial) domain is equivalent to the point-wise multiplication of their individual Fourier transforms in the frequency domain

$$G(u) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(\tau)h(x - \tau)e^{-i2\pi ux} d\tau dx$$

$$G(u) = \int_{-\infty}^{\infty} f(\tau)e^{-i2\pi u\tau} d\tau \int_{-\infty}^{\infty} h(x - \tau)e^{-i2\pi u(x-\tau)} dx$$

$F(u)$

$H(u)$

Convolution and Fourier Transform:



Spatial Domain		Frequency Domain
$g(x) = f(x) * h(x)$ Convolution	$\longleftrightarrow$	$G(u) = F(u) H(u)$ Multiplication
$g(x) = f(x) h(x)$ Multiplication	$\longleftrightarrow$	$G(u) = F(u) * H(u)$ Convolution

$$\begin{array}{ccccc}
 g(x) & = & f(x) & * & h(x) \\
 \uparrow & & | & & | \\
 \boxed{\text{IFT}} & & \boxed{\text{FT}} & & \boxed{\text{FT}} \\
 | & & \downarrow & & \downarrow \\
 G(u) & = & F(u) & \times & H(u)
 \end{array}$$

Convolution with Discrete Image:



$$f[i,j] \longrightarrow \boxed{h[i,j]} \longrightarrow g[i,j]$$

$$g[i,j] = \sum_{m=1}^M \sum_{n=1}^N f[m,n] \underbrace{h[i-m, j-n]}_{\text{"Mask," "Kernel," "Filter"}}$$

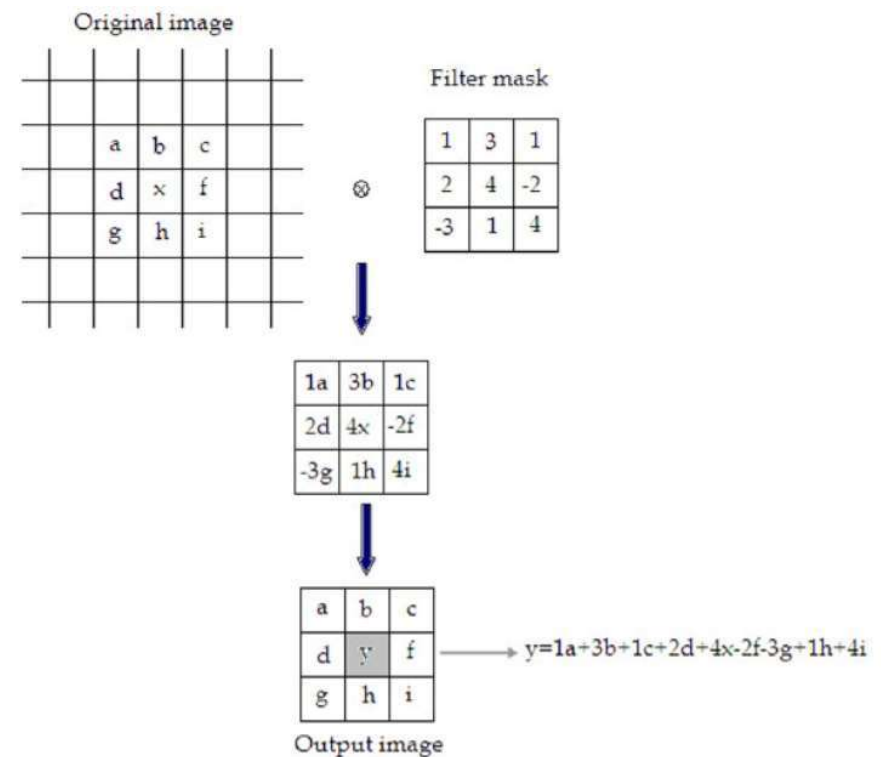
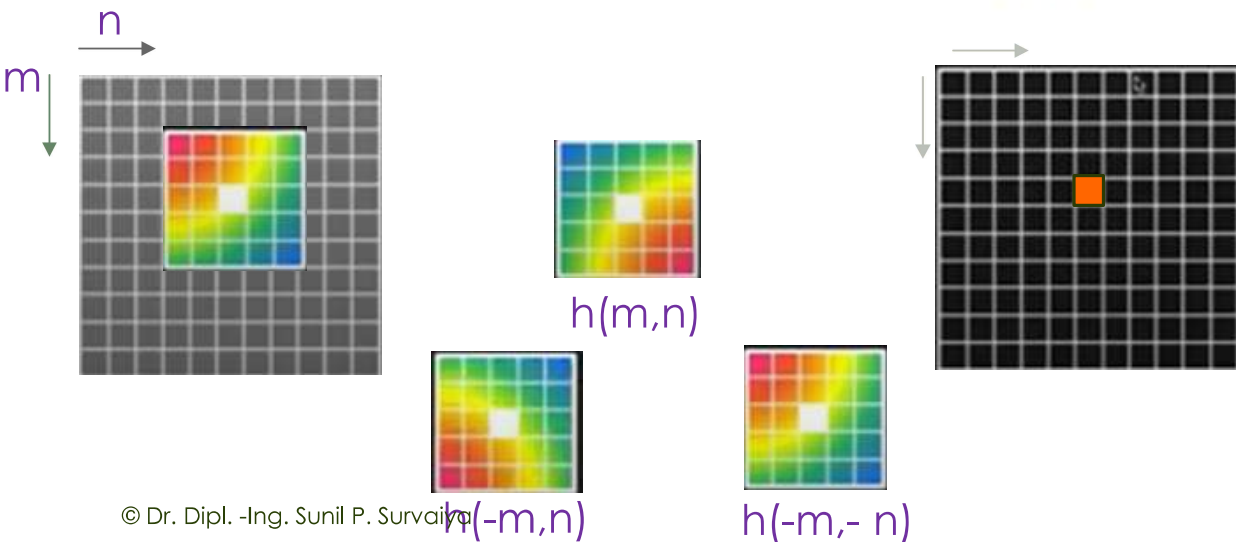


Image filtering

Modify the pixels in an image based on some function of a local neighborhood of each pixel



10	5	3
4	5	1
1	1	7

Local image data

Some function



	7	

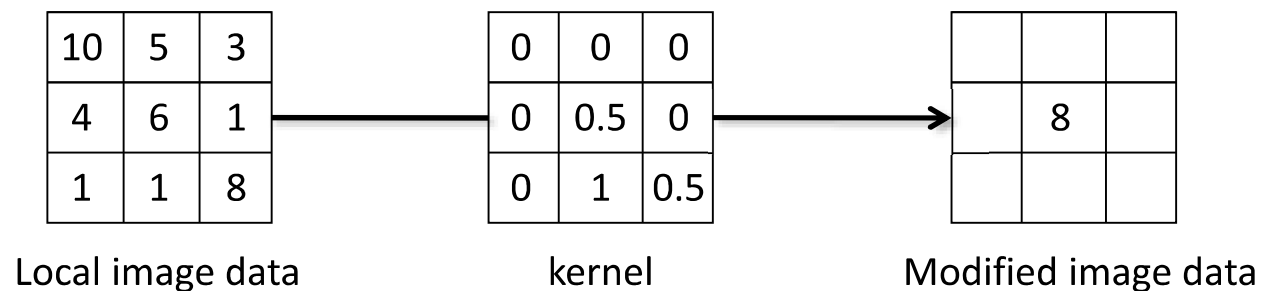
Modified image data

Linear filtering

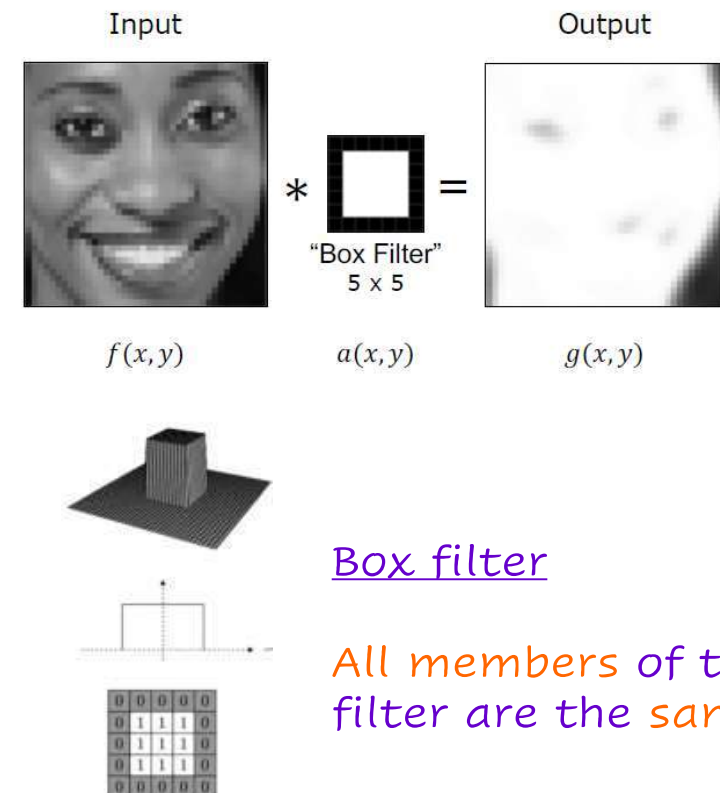
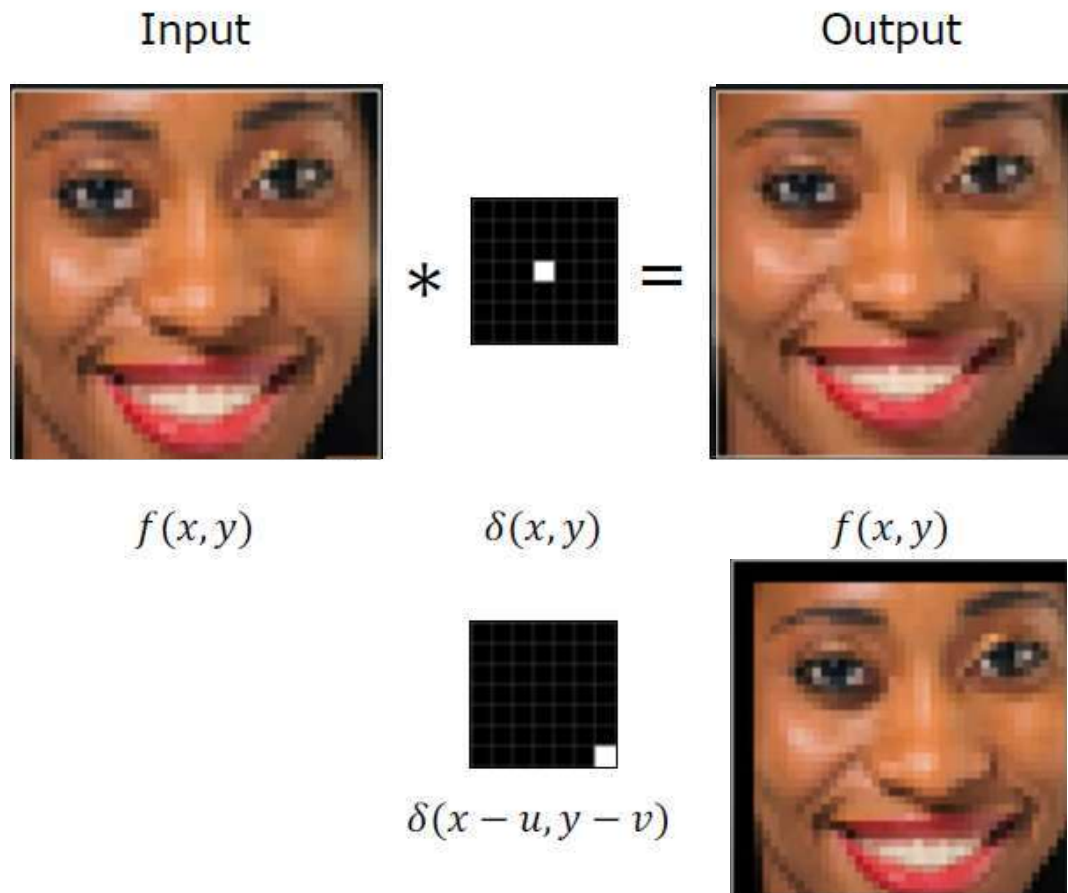
- One simple version:

linear filtering (cross-correlation, convolution)

– Replace each pixel by a linear combination of its neighbors



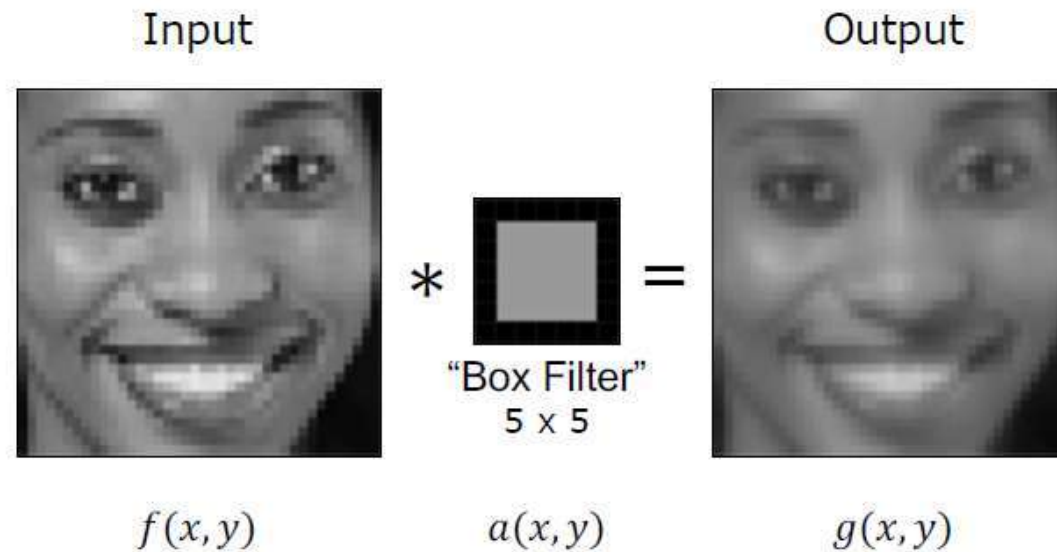
Impulse and Box Filtering:



Box filter

All members of this filter are the same.

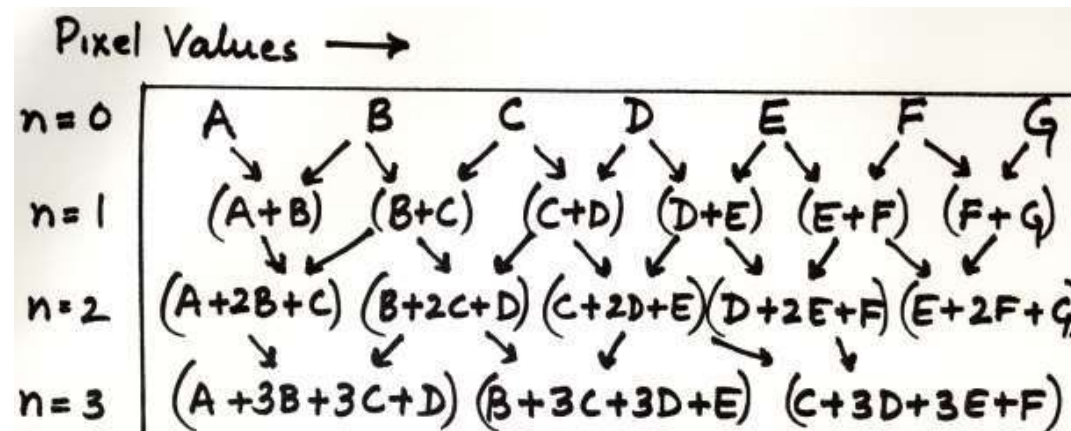
Average filtering:



the values used inside the box are **normalized** by
the **area of the box** itself

Averaging

Let's think about averaging pixel values



For $n=2$, convolve pixel values with

1	2	1
---	---	---

2D images:

(a) use

1	2	1
---	---	---

 then

1
2
1

 or (b) use

1	2	1
---	---	---

 *

1
2
1

 =

1	2	1
2	4	2
1	2	1

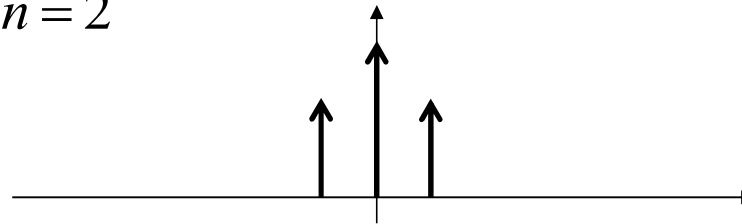
Which is faster? (a) $O(2(n+1))$ (b) $O((n+1)^2)$

Averaging

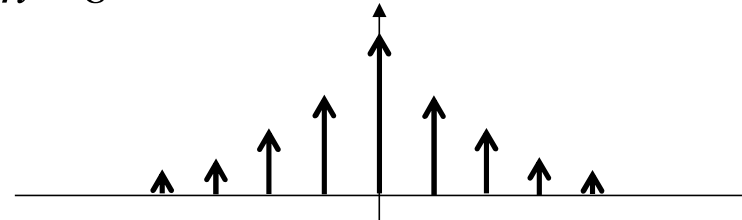
The convolution kernel



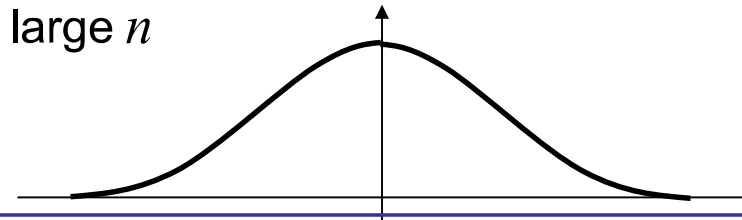
$n = 2$



$n = 8$

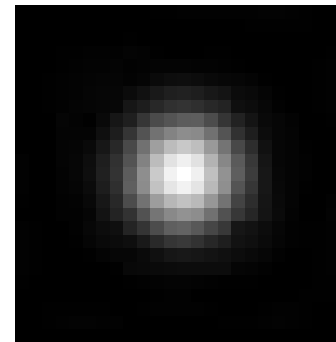
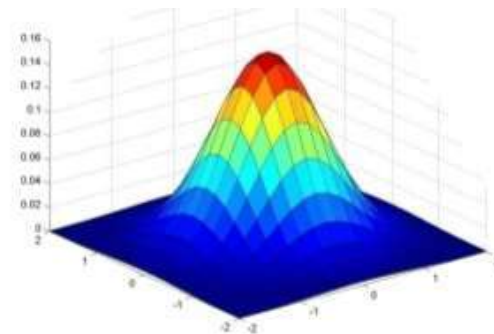


large n



Repeated averaging $\approx$ Gaussian smoothing

Gaussian Kernel

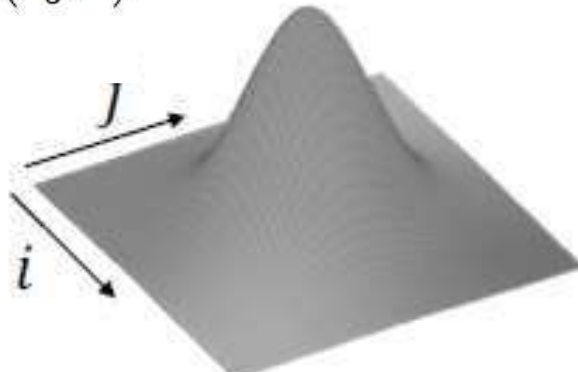


$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Fuzzy /Gaussian Filter:

$$n_{\sigma}[i, j] = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2}\left(\frac{i^2+j^2}{\sigma^2}\right)}$$

σ^2 : Variance



1/16

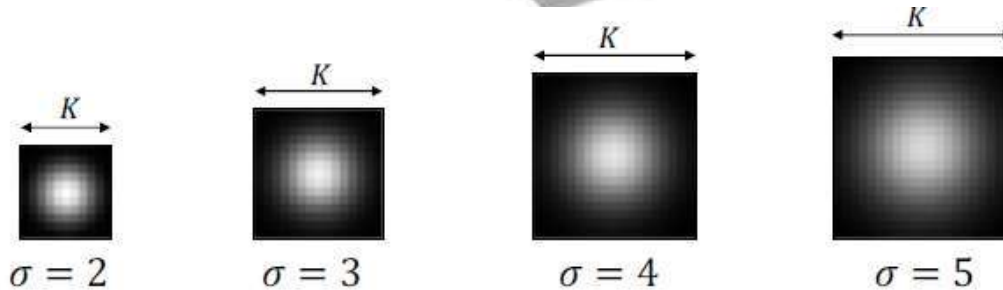
1	2	1
2	4	2
1	2	1

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

1/1003

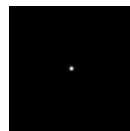
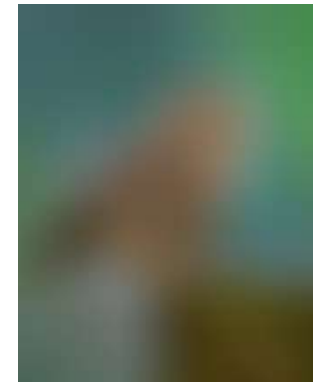
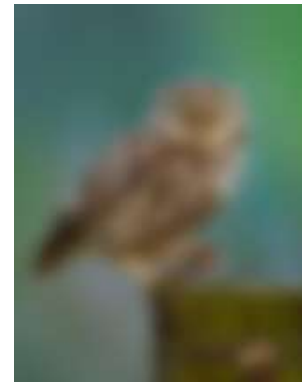
0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0



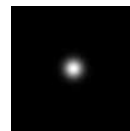
Rule of thumb: Set kernel size $K \approx 2\pi\sigma$

For visualization purposes, we are showing them as having equal brightness at the center, but in reality the filter with $\sigma=5$ would be much dimmer because there are a lot more pixels in it.

Gaussian filters



$\sigma = 1$ pixel



$\sigma = 5$ pixels



$\sigma = 10$ pixels



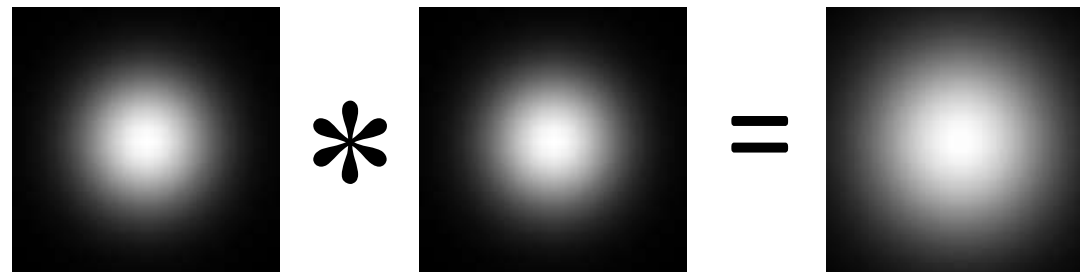
$\sigma = 30$ pixels

Gaussian filter



Removes “high-frequency” components from the image (low-pass filter)

Convolution with self is another Gaussian



Convolving two times with Gaussian kernel of width σ = convolving once with kernel of width $\sigma\sqrt{2}$

Linear filters: Examples



Original



0	0	0
0	1	0
0	0	0



Identical image

Linear filters: Examples



Original



0	0	0
1	0	0
0	0	0



Shifted left
By 1 pixel

Sharpening revisited

- What does blurring take away?



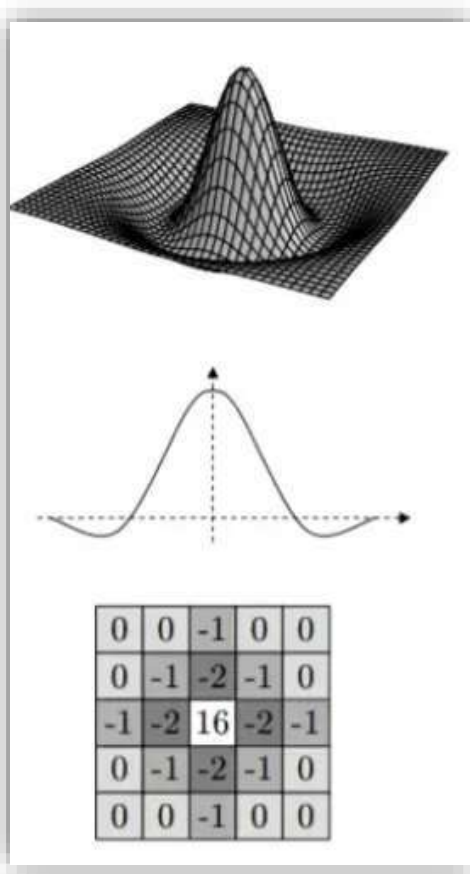
Let's add it back:



Sharpening Examples:



Laplacian filter: Linear filter



Laplace or Mexican hat filter.

Some members of this filter are negative

it can calculate by summation of positive member and negative member.

$$\nabla \cdot \nabla f = \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

NOTE:

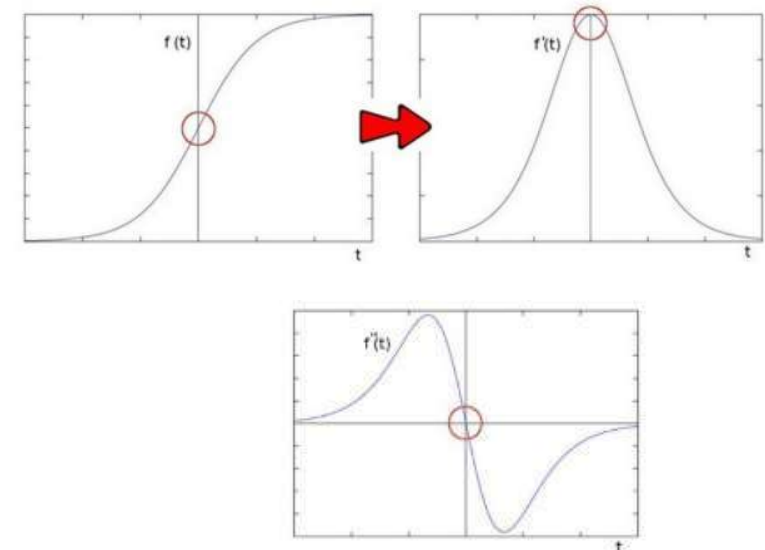
zeros will not only appear in edges.

They can actually appear in other meaningless locations. his can be solved by applying filtering when needed

0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Laplacian Mask.



Non-Linear Filters:

Noise removing with smoothing filter (a linear filter) provide the result in burred of the image structure, line and edge.

Non-Linear Filters were used to solve this problem, and it works in non-linear manner.

Minimum and Maximum Filters:

The minimum and maximum value in the moving region R of the original image.

$$I'(u, v) \leftarrow \min \{I(u+i, v+j) \mid (i, j) \in R\}$$

$$I'(u, v) \leftarrow \max \{I(u+i, v+j) \mid (i, j) \in R\}$$

Median Filter:

The median of all value in moving region R is the result of the median filter. This filter typically use for remove salt and pepper noise in the image



$$I'(u, v) \leftarrow \text{median} \{I(u+i, v+j) \mid (i, j) \in R\}$$

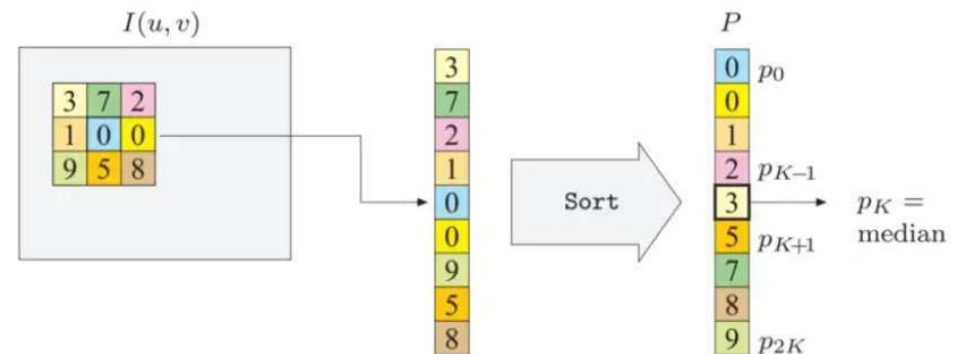
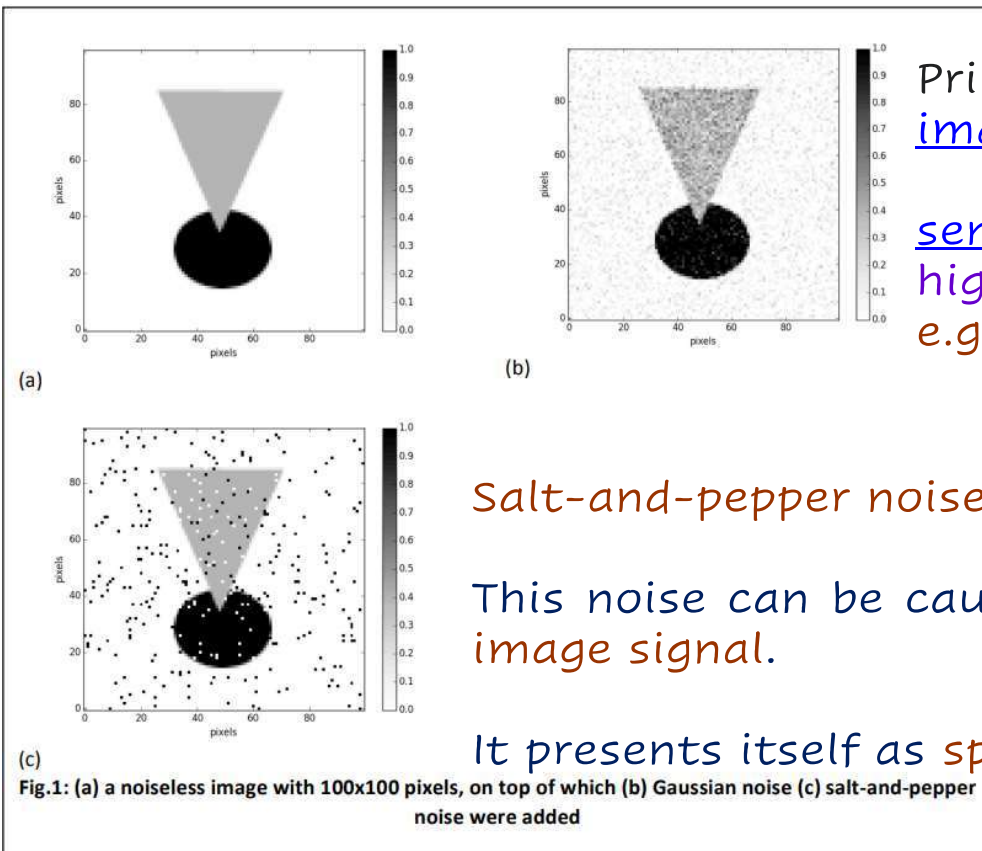


Image noise: Gaussian / Salt and Pepper



Principal sources of Gaussian noise in digital images arise during acquisition

sensor noise caused by poor illumination and/or high temperature, and/or transmission e.g. electronic circuit noise.

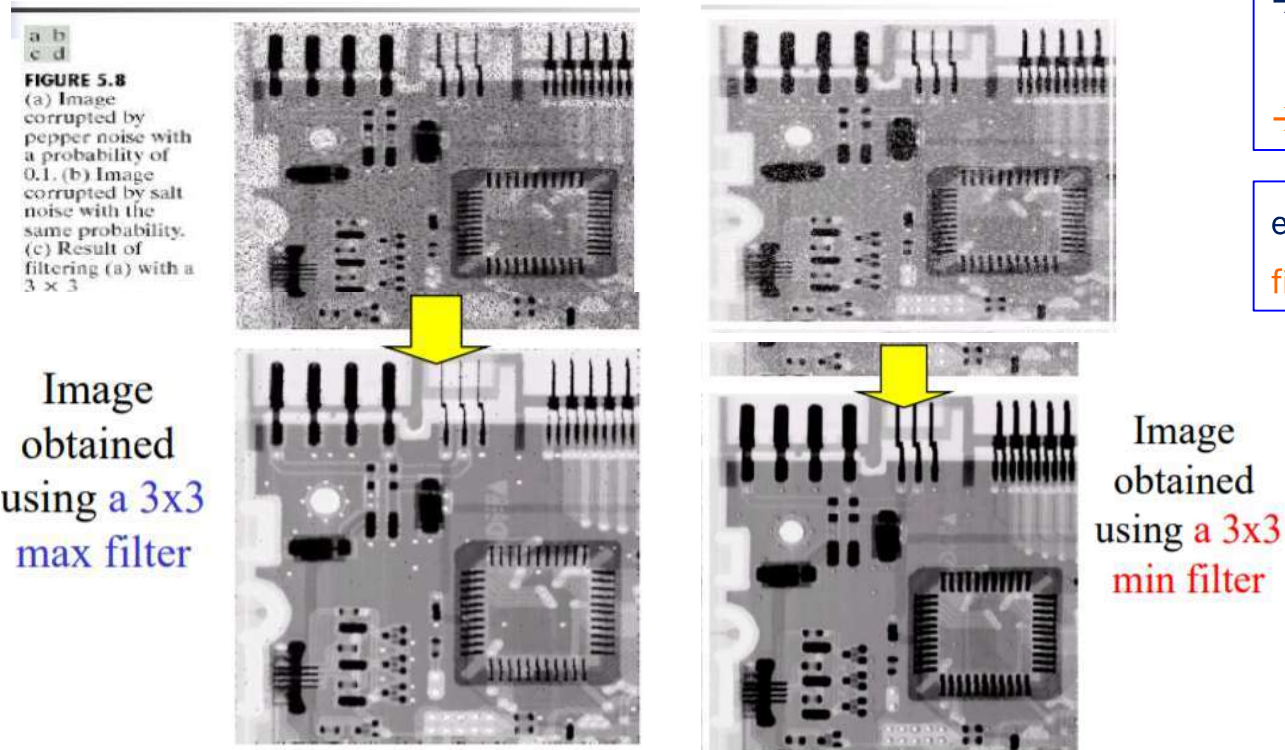
Salt-and-pepper noise, also known as impulse noise,

*

This noise can be caused by sharp and sudden disturbances in the image signal.

It presents itself as sparsely occurring white and black pixels.

Min and Max Filters:



Salt-and-pepper

- sometimes seen on images (aka **impulse noise**)
- e.g. caused by **sharp & sudden disturbances** in the image signal
- **sparsely occurring white & black pixels**

effective noise reduction e.g. by applying a **median filter**, **max filter**, **min filter**

min filter replaces the central pixel with the darkest one in the running window

max filter replaces the central pixel with the lightest one in the running window

Geometric mean filter:

$$\bar{x}_{\text{geom}} = \sqrt[n]{\prod_{i=1}^n x_i} = \sqrt[n]{x_1 \cdot x_2 \cdots x_n}$$

$$\hat{f}(x, y) = \left[\prod_{(s,t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}}$$

- Geometric mean filter
 - → achieves **smoothing** comparable to the **arithmetic mean filter**

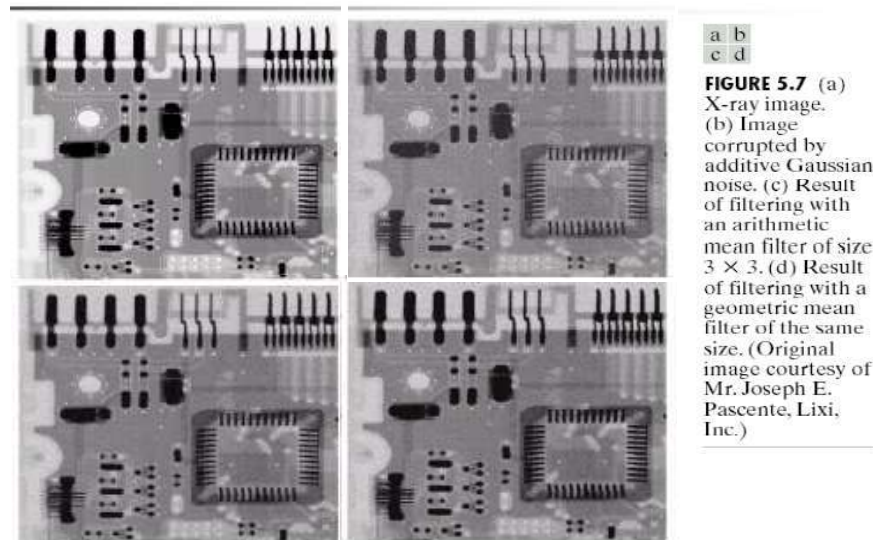


Image Sharpening: OpenCV



➤ OpenCV → Image Sharpening

- different types of kernels for sharpening and blurring images
- sharpening kernel
 - → highlights differences in adjacent pixel values
 - → increases contrast
 - different sharpening kernels, e.g.

- Sharpening kernel type 1:
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Sharpening kernel type 2:
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Exercise : OpenCV code



```

1  # CV Test blurring and sharpening
2  #
3  import numpy as np
4  import cv2
5
6  image = cv2.imread('Test_image.jpg')
7  cv2.imshow('My image', image)
8  cv2.waitKey()
9  cv2.destroyAllWindows()
10
11  gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
12  cv2.imshow('Self Driving Car in Gray', gray_image)
13  cv2.waitKey()
14  cv2.destroyAllWindows()
15
16  Sharp_Kernel_1 = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
17  Sharpened_image_1 = cv2.filter2D(gray_image,-1,Sharp_Kernel_1)
18  cv2.imshow('Sharpened Image', Sharpened_image_1)
19  cv2.waitKey()
20  cv2.destroyAllWindows()
21
22  Sharp_Kernel_2 = np.array([[1,-1,-1],[-1,9,-1],[-1,-1,-1]])
23  Sharpened_image_2 = cv2.filter2D(gray_image,-1,Sharp_Kernel_2)
24  cv2.imshow('Sharpened Image 2', Sharpened_image_2)
25  cv2.waitKey()
26  cv2.destroyAllWindows()
27
28  Blurr_Kernel1 = np.ones((3,3))
29  Blurr_Kernel1
30  Blurred_image1 = cv2.filter2D(gray_image, -1, Blurr_Kernel1)
31  cv2.imshow('Blurred image', Blurred_image1)
32  cv2.waitKey()
33  cv2.destroyAllWindows()
34
35  Blurr_Kernel2 = np.ones((3,3))/9
36  Blurr_Kernel2
37  Blurred_image2 = cv2.filter2D(gray_image, -1, Blurr_Kernel2)
38  cv2.imshow('Blurred Image', Blurred_image2)
39  cv2.waitKey()
40  cv2.destroyAllWindows()
41

```

```

41  |
42  Blurr_kernel3 = np.ones((8,8))*1/64
43  Blurr_kernel3
44  Blurred_image3 = cv2.filter2D(gray_image, -1, Blurr_kernel3)
45  cv2.imshow('Blurred Image', Blurred_image3)
46  cv2.waitKey()
47  cv2.destroyAllWindows()
48
49  image = cv2.imread('Test_image.jpg')
50  height, width = image.shape[:2]
51  w0 = int(width*0.3)
52  h0 = int(height*0.3)
53  w1 = int(width*1)
54  h1 = int(height*1)
55  image_cropped = image[h0:h1, w0:w1]
56  cv2.imshow('cropped image', image_cropped)
57  cv2.waitKey()
58  cv2.destroyAllWindows()
59
60
61  kernel = np.ones((3,3), np.uint8)
62  image_erosion = cv2.erode(image, kernel, iterations=3)
63  image_dilation = cv2.dilate(image, kernel, iterations=3)
64
65  cv2.imshow('input', image)
66  cv2.imshow('erosion', image_erosion)
67  cv2.imshow('dilation', image_dilation)
68  cv2.waitKey()
69  cv2.destroyAllWindows()
70

```

Results: OpenCV



Original Image



Gray Image



Enhanced Image 1



Enhanced image 2



Geometrical Transformation



- Why Image transformation required?
- Scaling, shearing, rotation
- Projective geometry and its transformations
- Classification of 2D transformation

Why Image Transformation required?

Image transforms are used to manipulate or process images



Data augmentation:

- Increase data diversity and improve model generalization.
- a rotation, scaling, and flipping of images.

Image pre-processing:

It can be applied to images before they are fed into a model in order to improve performance. This can include operations such as **resizing, cropping, and normalization**.

Compression:

It can be used to **compress images** by reducing the amount of data required to represent them, **while maintaining acceptable image quality**.

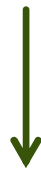
Feature extraction:

It can be used to extract features from images that are useful for tasks such as **object recognition or image segmentation**.

Filtering and Wrapping



F



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

Filtering

changes *range* of
image function

G



F

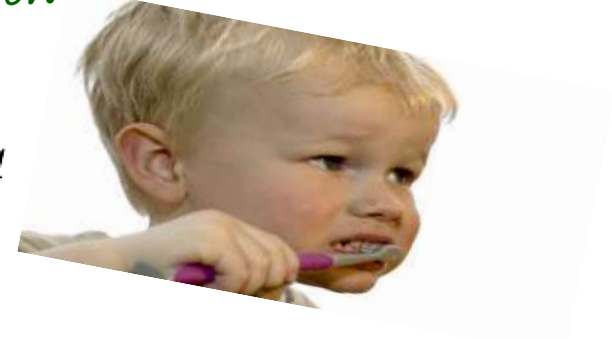


$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

Warping

changes *domain* of
image function

G



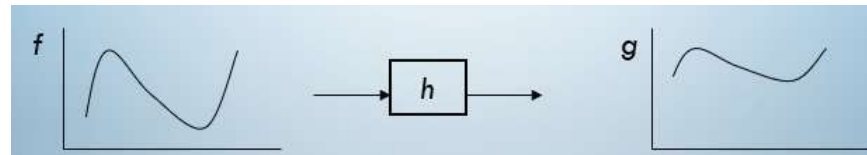
What is image filtering and warping?



Pixel color is altered without pixel position

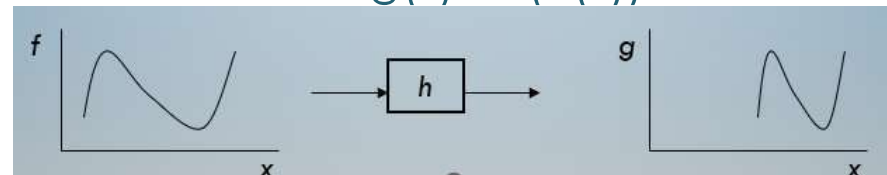
Image filtering: change range of image

$$g(x) = h(f(x))$$



Pixel position is mapped to the other position without changing the pixel value
image warping: change domain of image

$$g(x) = f(h(x))$$



Pixel or Point Processing:



Transformation T of intensity f at each pixel to intensity g :

$$g(x,y) = T[f(x,y)]$$



Original image (f)



Darken ($f-128$)



Lighten ($f+128$)



Invert ($255-f$)

Pixel or Point Processing:



Original image (f)



low Contrast ($f/2$)



High Contrast ($2*f$)



Gray ($0.3f_R + 0.6f_G + 0.1f_B$)

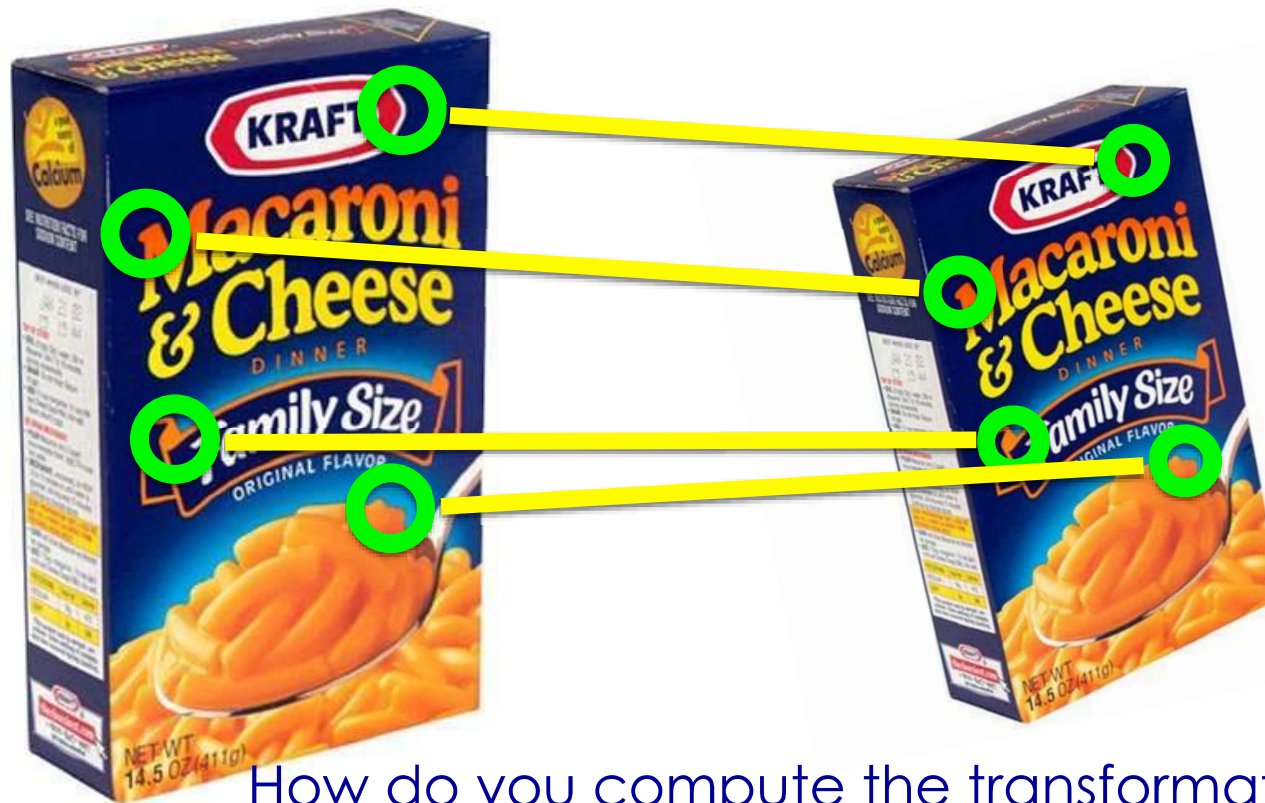
Wrapping



Wrapping Ex. Feature Matching



Wrapping Ex. Feature Matching



How do you compute the transformation?

Wrapping Ex. Feature Matching

Given a set of matched feature points:

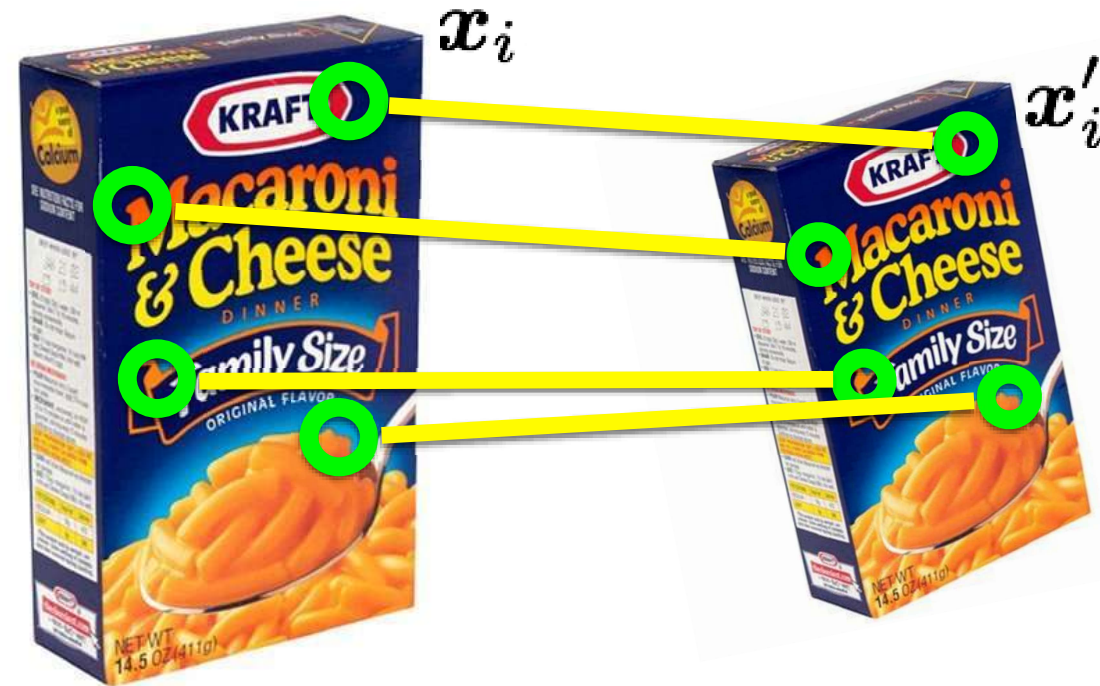
$\{x_i, x'_i\}$
 point in one image point in the other image
 and a transformation:

$$x' = f(x; p)$$

transformation function parameters

find the best estimate of the parameters p

What kind of transformation functions f are there?



2D TRANSFORMATIONS

Parametric wraps:



translation



rotation



aspect



affine



perspective



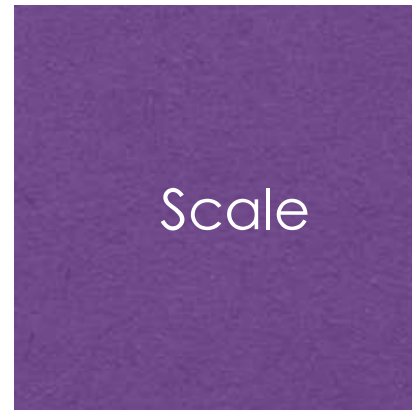
2D Planar Transformation



2D Planar Transformation: **Scaling**



y

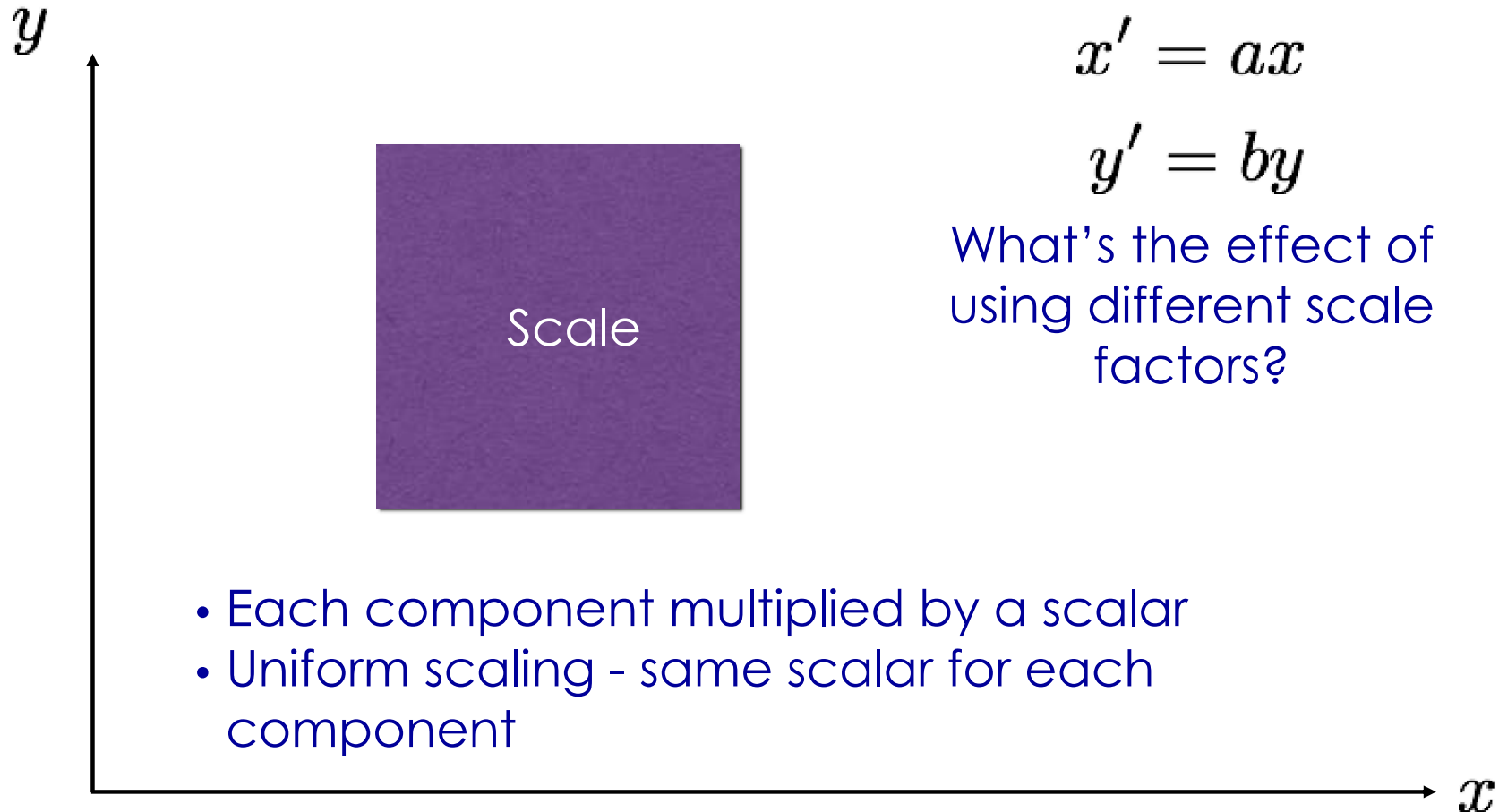


How would you implement scaling?

- Each component multiplied by a scalar
- Uniform scaling - same scalar for each component

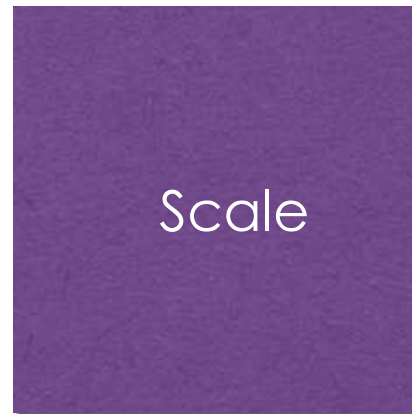
x

2D Planar Transformation: **Scaling**



2D Planar Transformation: Scaling

y



$$x' = ax$$

$$y' = by$$

matrix representation of scaling:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

scaling matrix S

- Each component multiplied by a scalar
- Uniform scaling - same scalar for each component

x

Uniform scaling example:



Uniform scaling by s :



$$\mathbf{S} = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}$$

2D Planar Transformation: Shearing

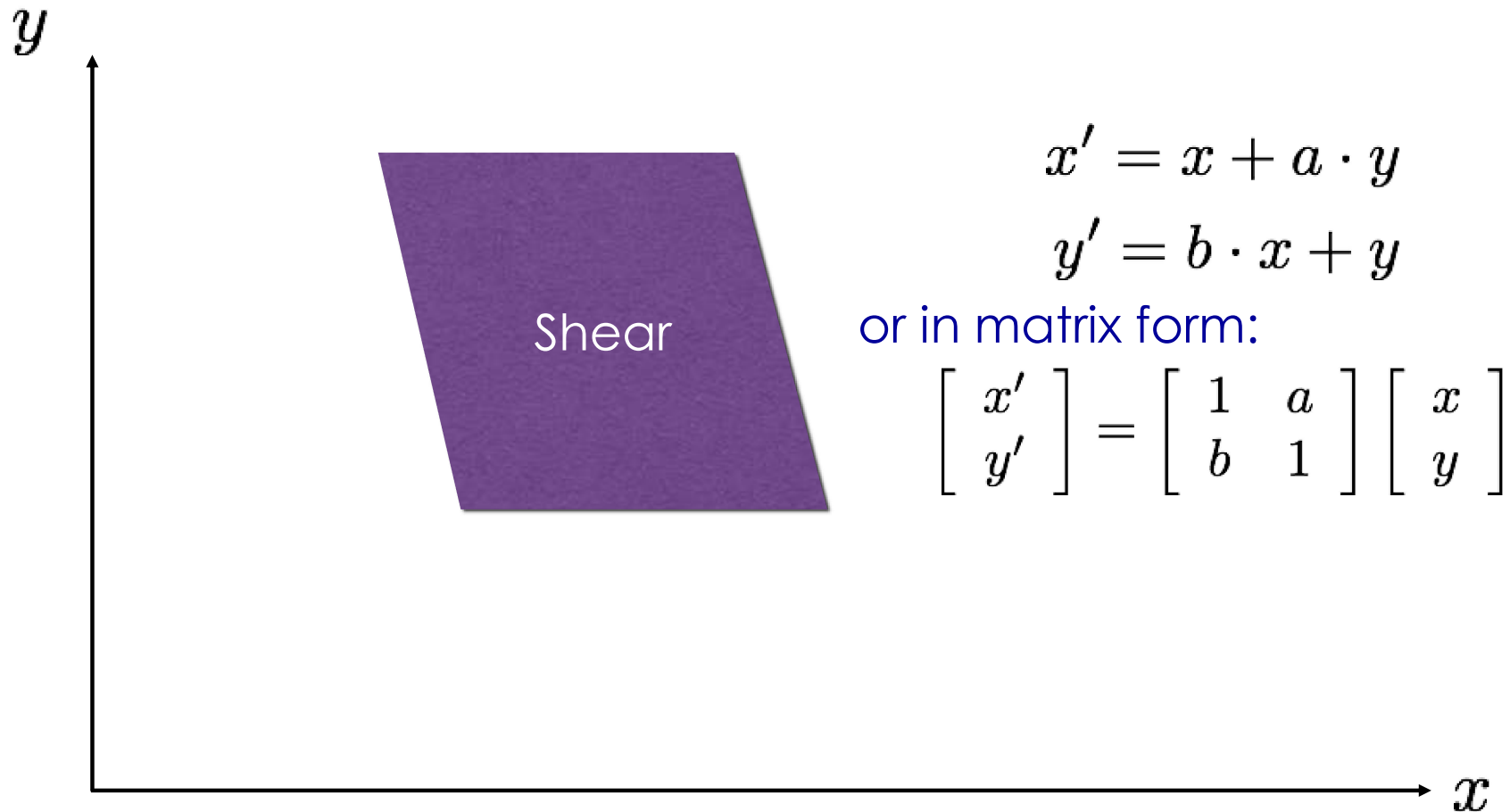
y



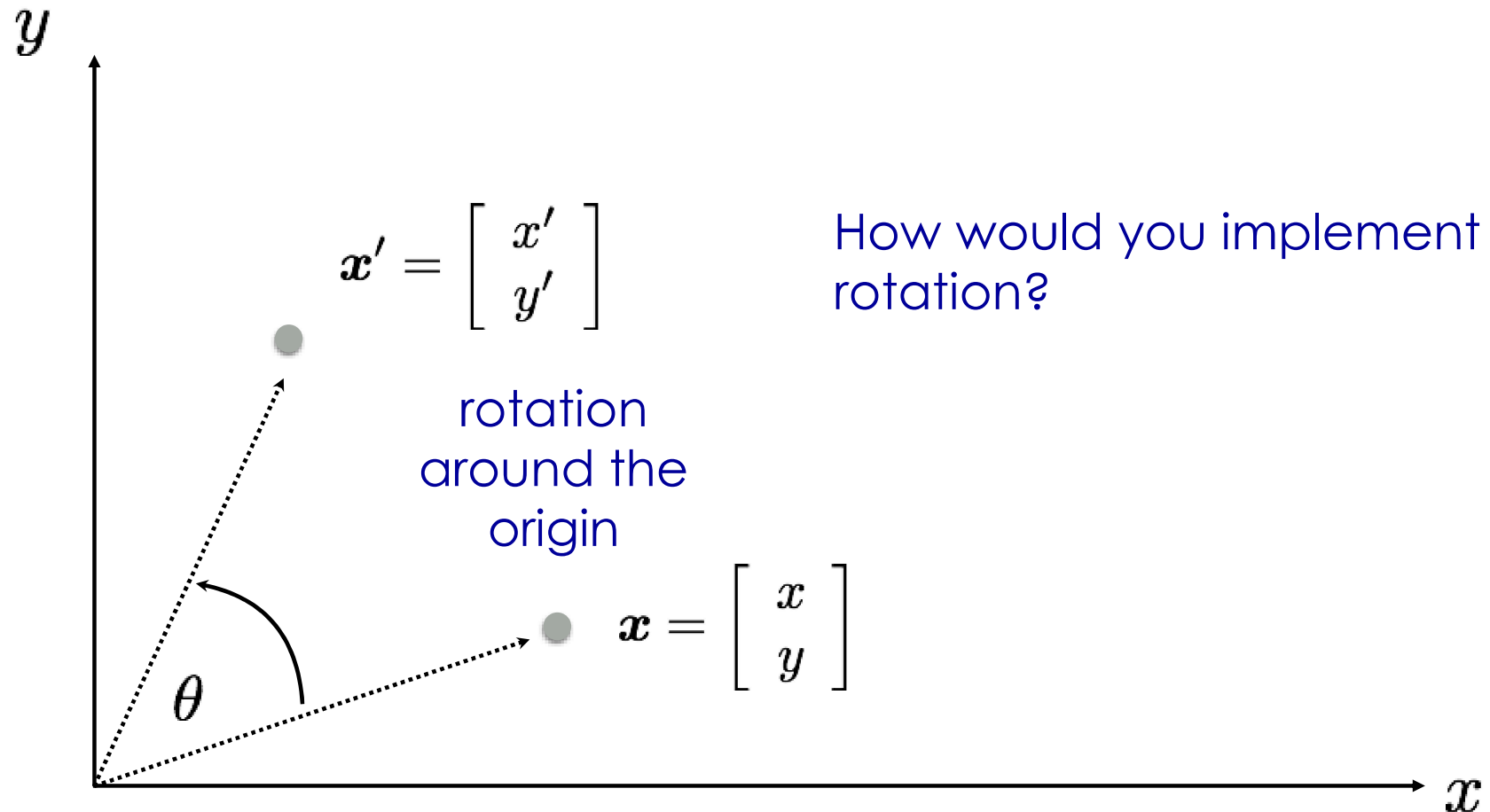
How would you implement shearing?

x

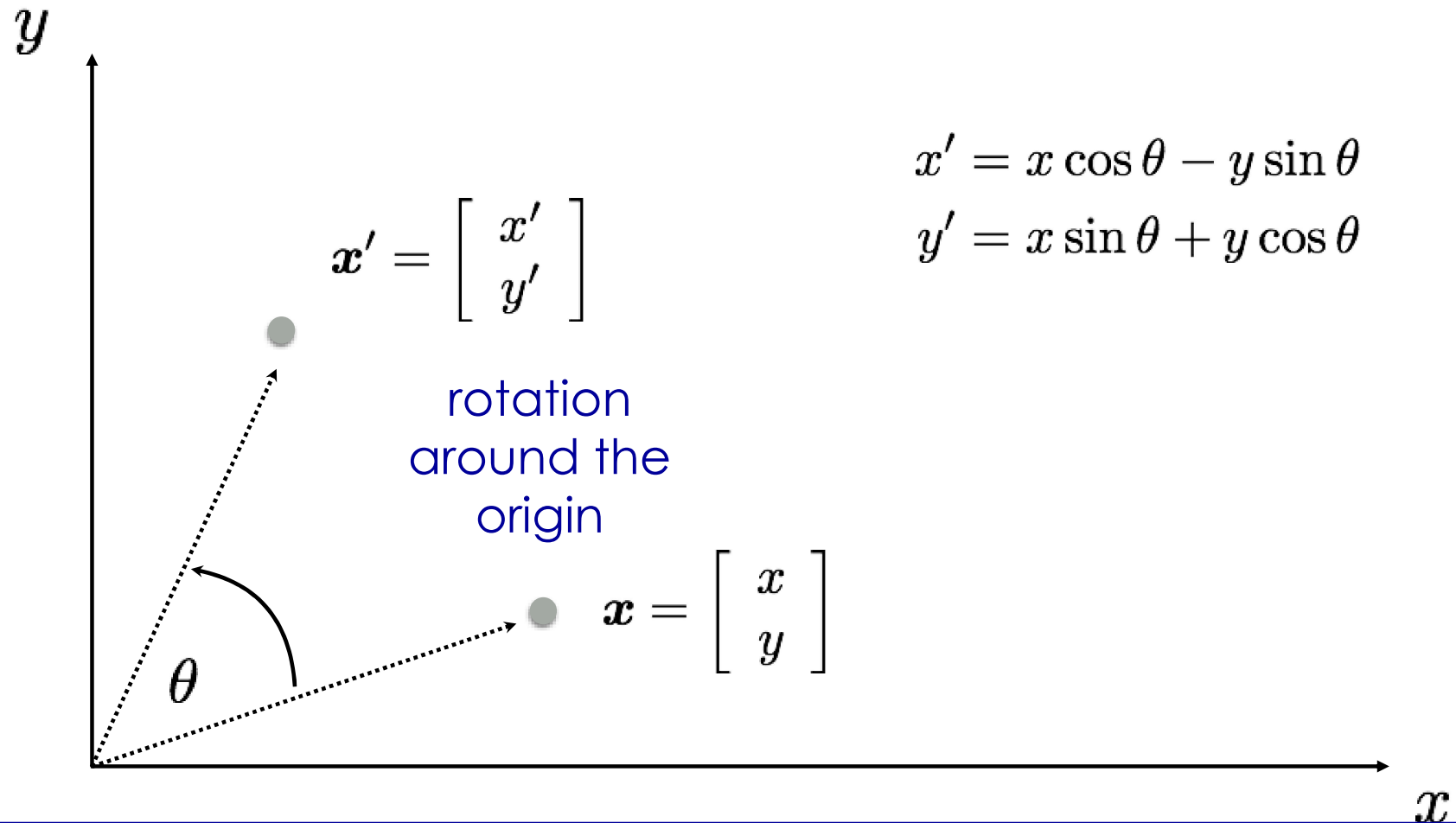
2D Planar Transformation: Shearing



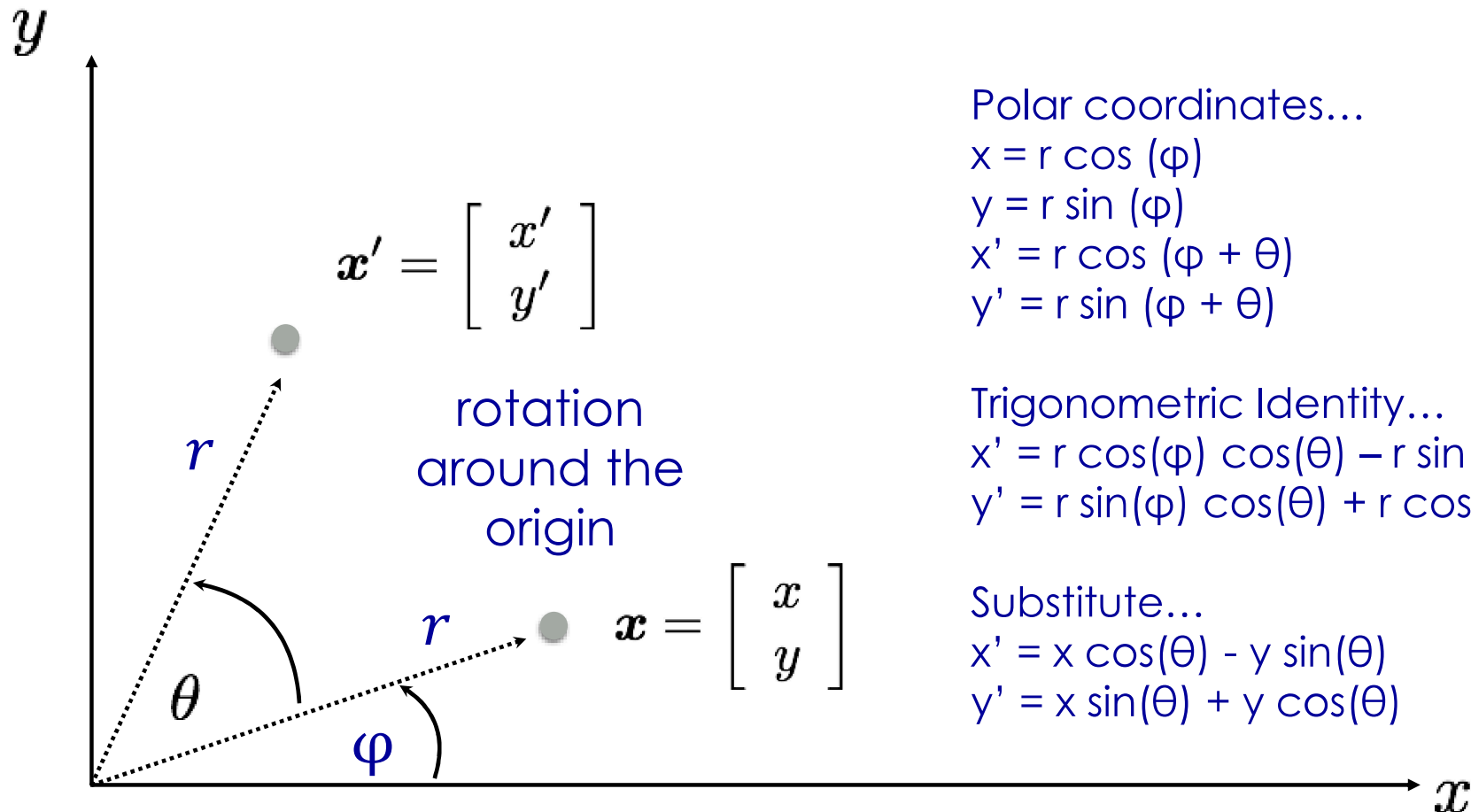
2D Planar Transformation: Rotation



2D Planar Transformation: Rotation



2D Planar Transformation: Rotation



Polar coordinates...

$$x = r \cos(\varphi)$$

$$y = r \sin(\varphi)$$

$$x' = r \cos(\varphi + \theta)$$

$$y' = r \sin(\varphi + \theta)$$

Trigonometric Identity...

$$x' = r \cos(\varphi) \cos(\theta) - r \sin(\varphi) \sin(\theta)$$

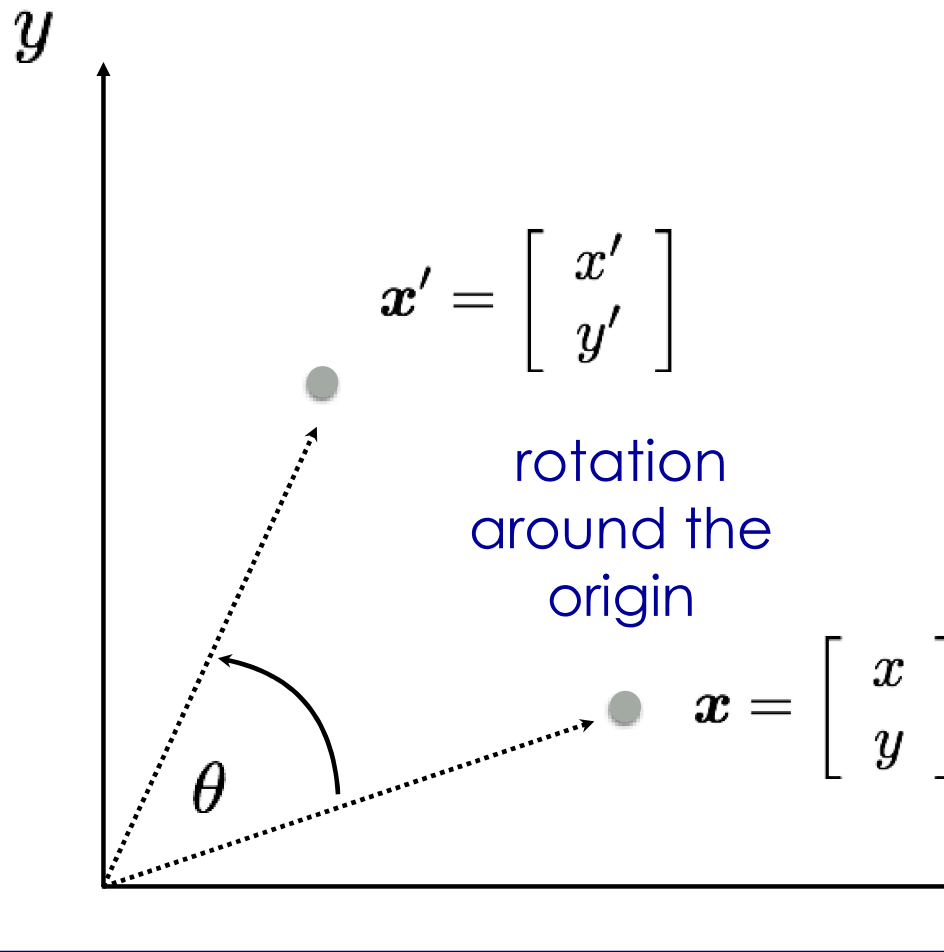
$$y' = r \sin(\varphi) \cos(\theta) + r \cos(\varphi) \sin(\theta)$$

Substitute...

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

2D Planar Transformation: Rotation



$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

or in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

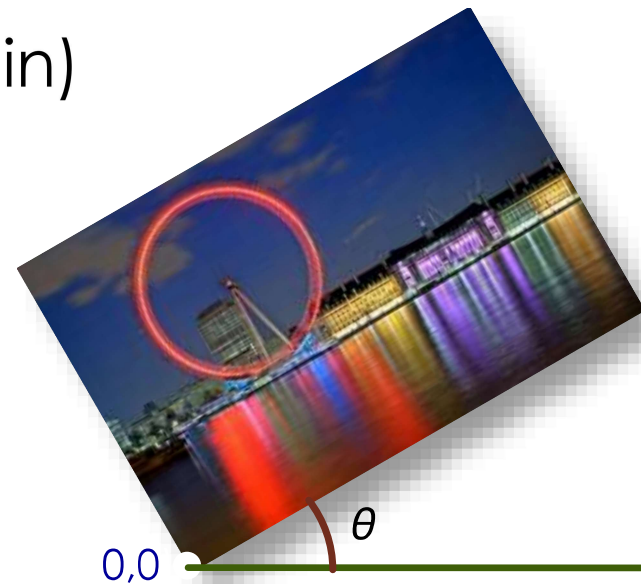
Rotation example:



Rotation by angle θ (about the origin)



$$\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$



0,0

What is the inverse?

For rotations:

$$\mathbf{R}^{-1} = \mathbf{R}^T$$

2D Planar and Linear Transformation



$$\mathbf{x}' = f(\mathbf{x}; p)$$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

parameters p

point $\mathbf{x}$

2D Planar and Linear Transformation



Scale

$$\mathbf{M} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Flip across y

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Rotate

$$\mathbf{M} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Flip across origin

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

Shear

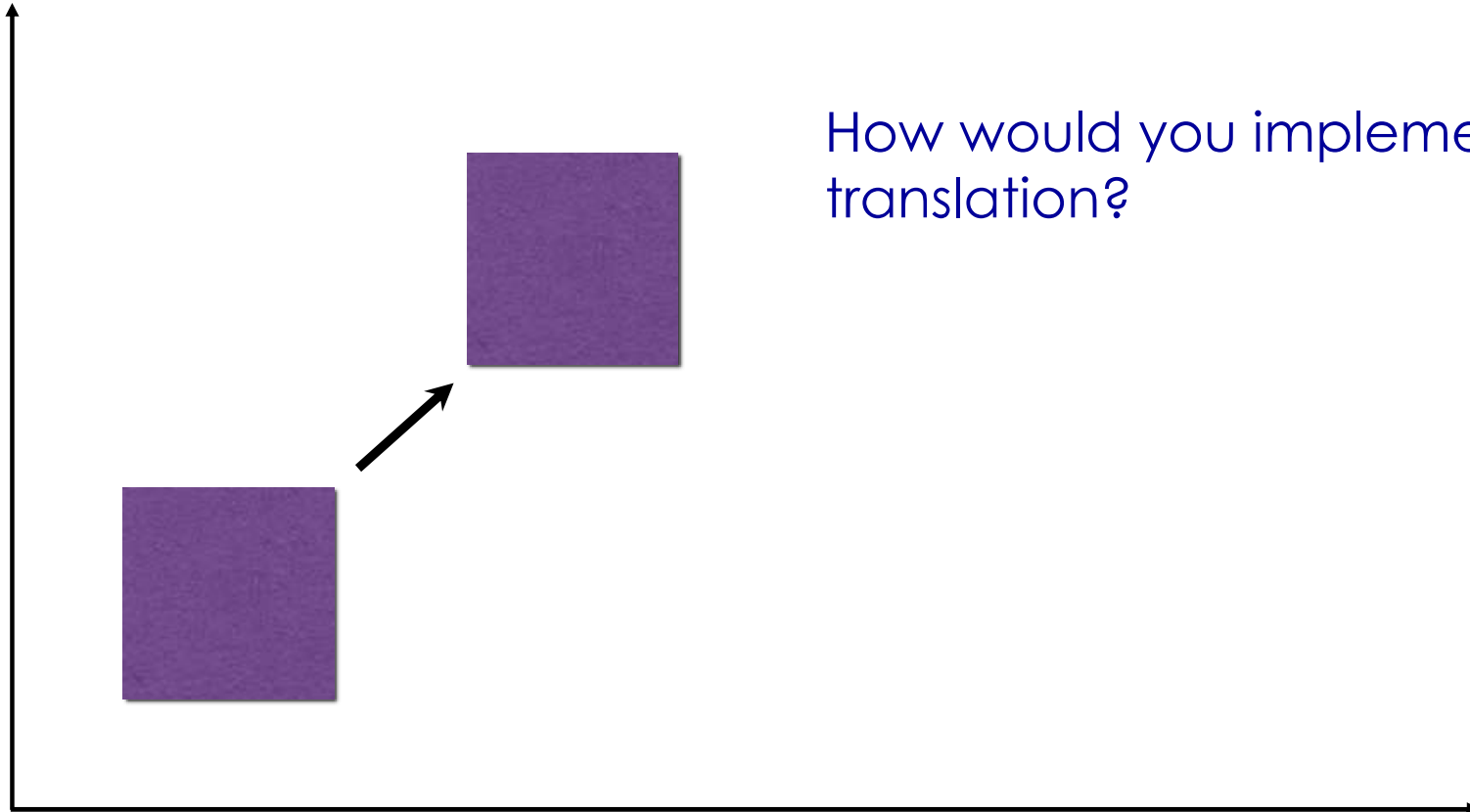
$$\mathbf{M} = \begin{bmatrix} 1 & s_x \\ s_y & 1 \end{bmatrix}$$

Identity

$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2D Translation

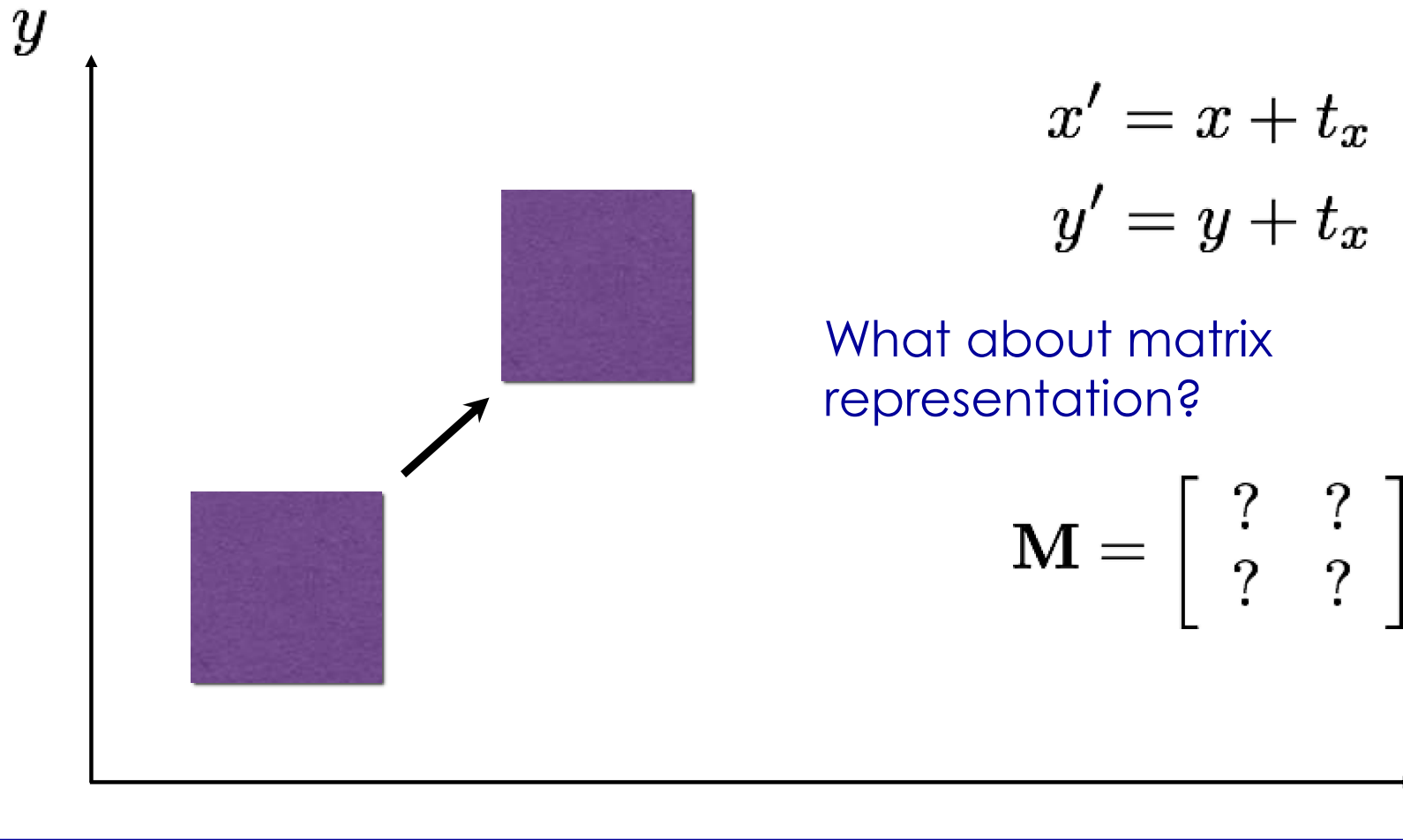
y



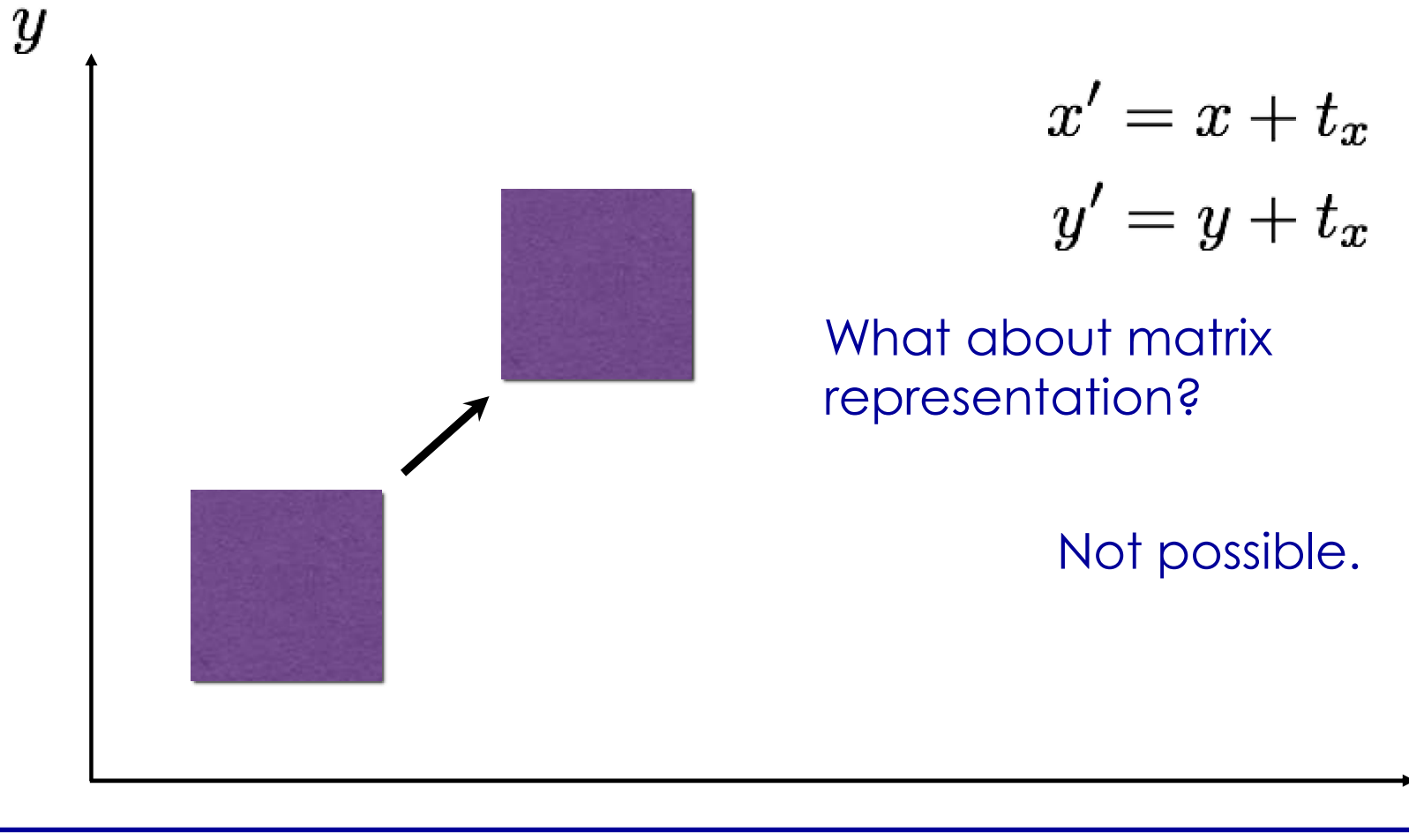
How would you implement translation?



2D Translation



2D Translation





Projection Geometry

Projective Geometry



image point in
pixel
coordinates

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

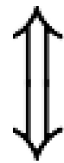
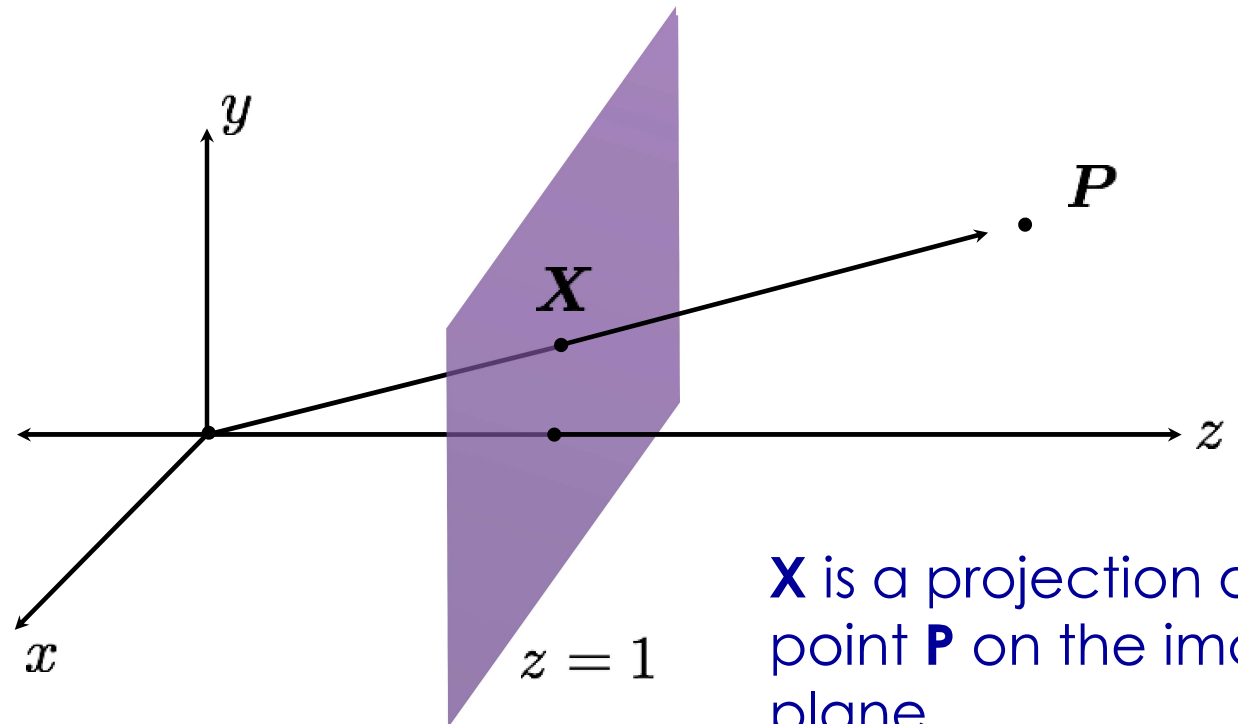


image point in
homogeneous
coordinates

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



X is a projection of a
point **P** on the image
plane

What does scaling **X** correspond to?

Homogeneous coordinates



heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

add a 1 here

- Represent 2D point with a 3D vector

Homogeneous coordinates



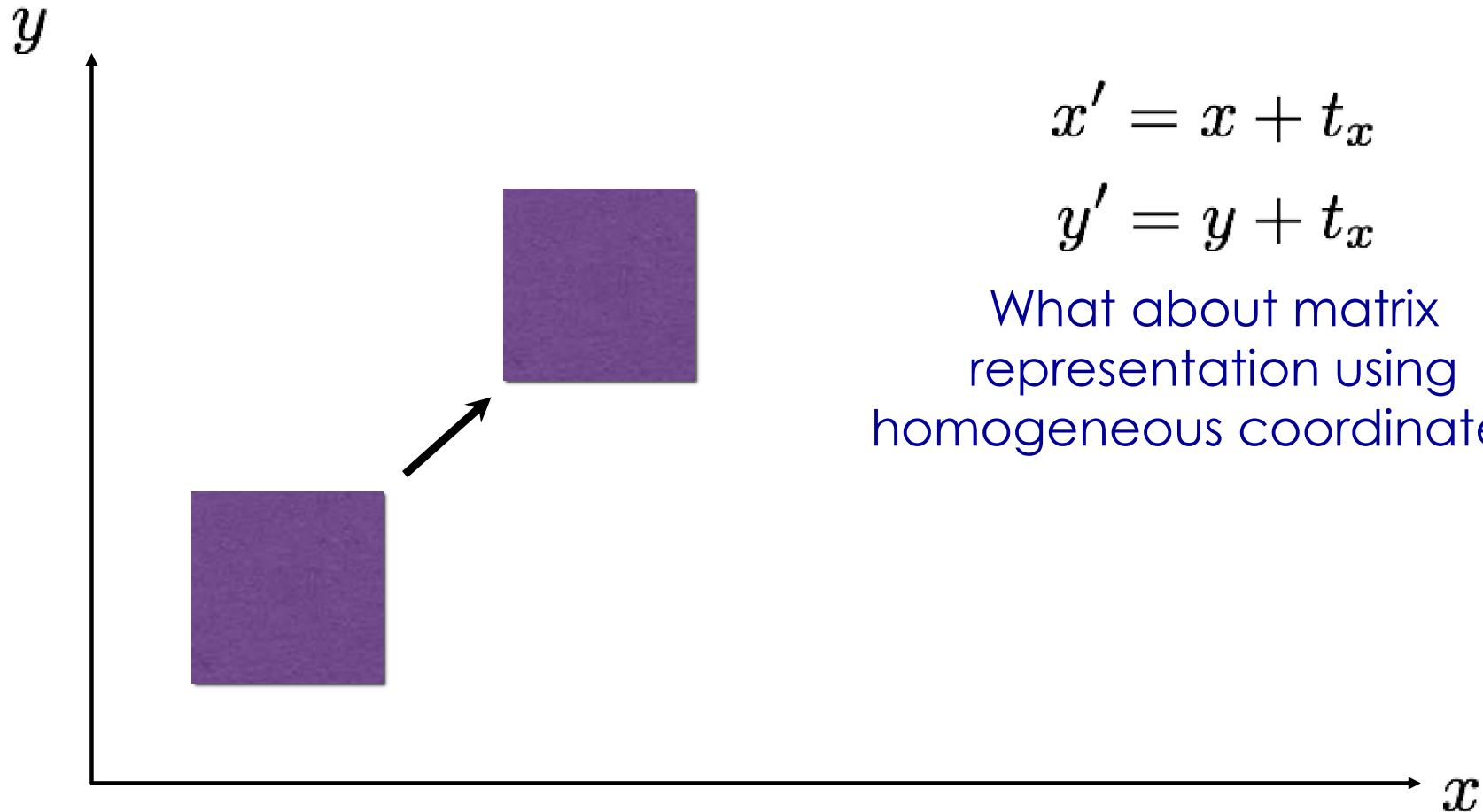
heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} ax \\ ay \\ a \end{bmatrix}$$

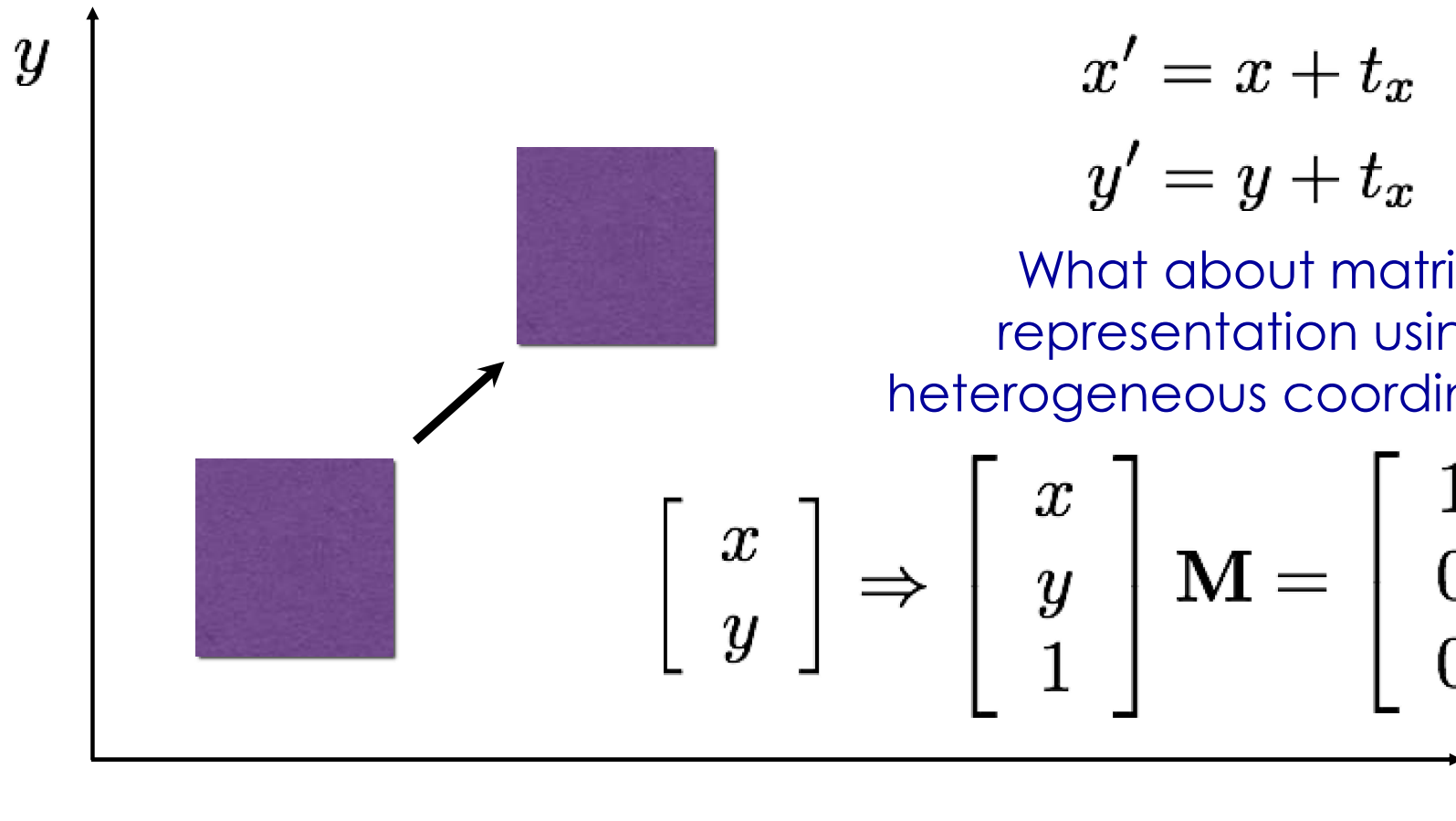
- Represent 2D point with a 3D vector
- 3D vectors are only *defined up to scale*

Homogeneous coordinates



What about matrix
representation using
homogeneous coordinates?

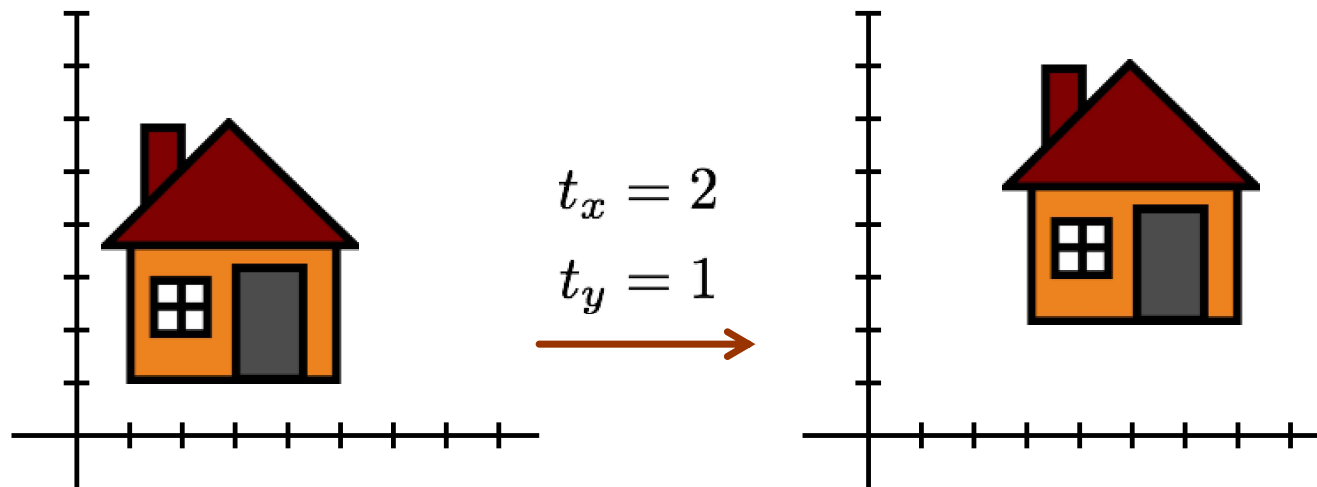
Homogeneous coordinates : 2D Translation



2D Translation using Homogeneous Coordinate



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$





Transformation in Projection Space

2D Transformation to Homogenous Coordinates



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

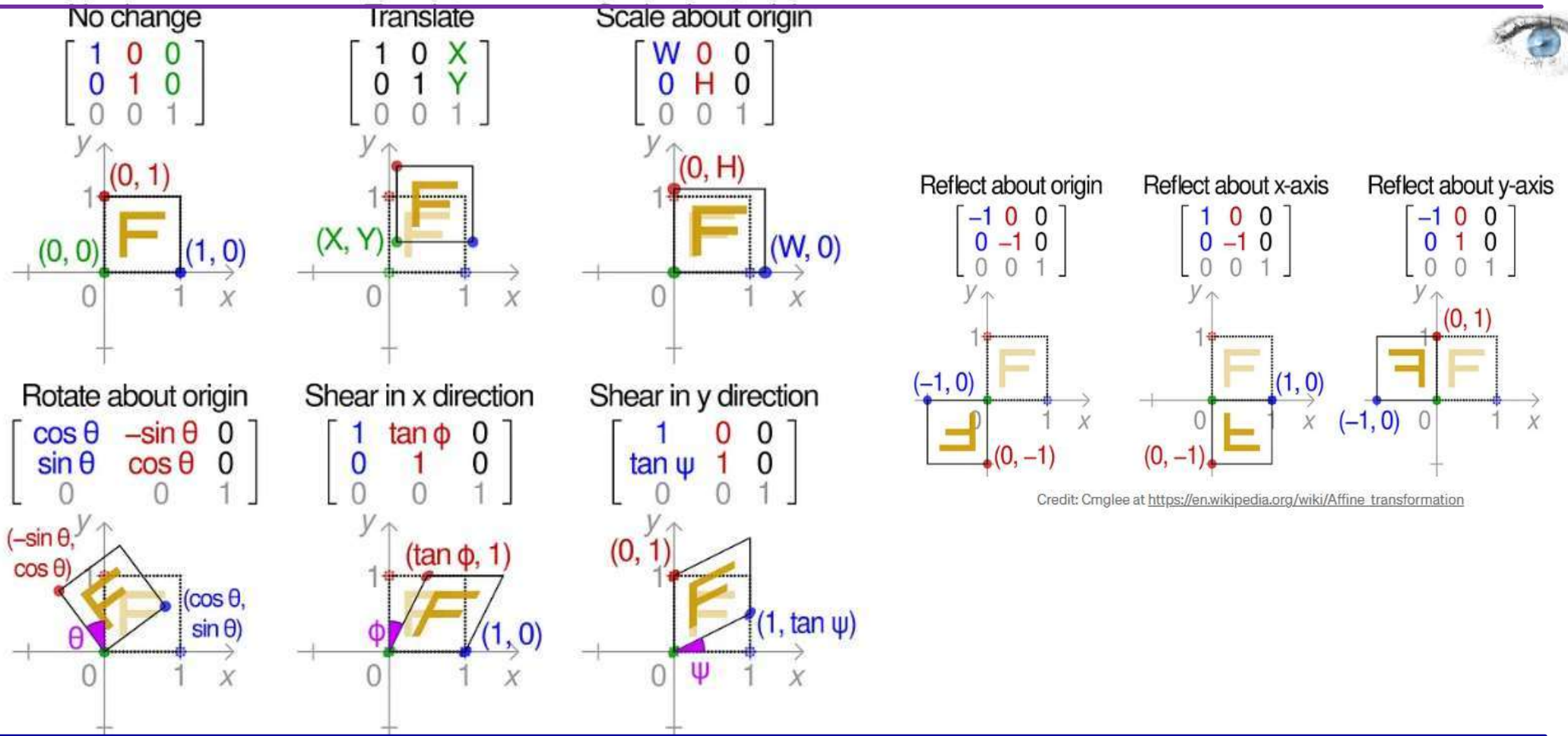
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

Summary: Transformations



Matrix Multiplication:



$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{pmatrix} \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{pmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

p' = ? ? ? **p**

Matrix Multiplication:

Transformations can be combined by matrix multiplication:



$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}'$ = translation(t_x, t_y)

rotation(Θ)

scale(s, s)

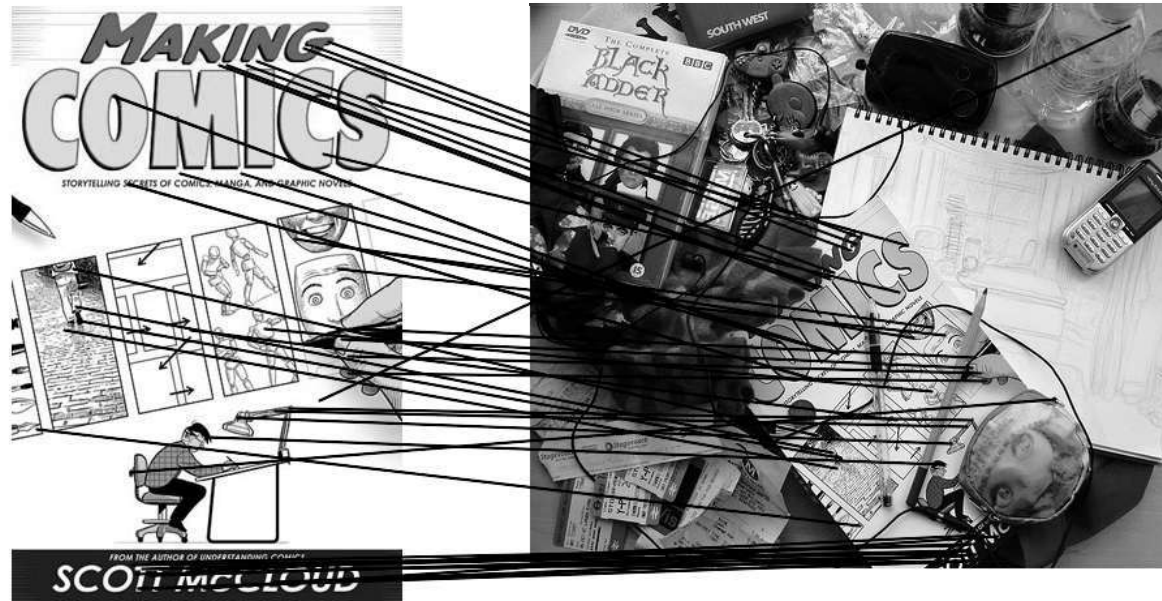
Does the multiplication order matter?

Computing Transformation:

Given a set of matches between images A and B



How can we compute the transform T from A to B?



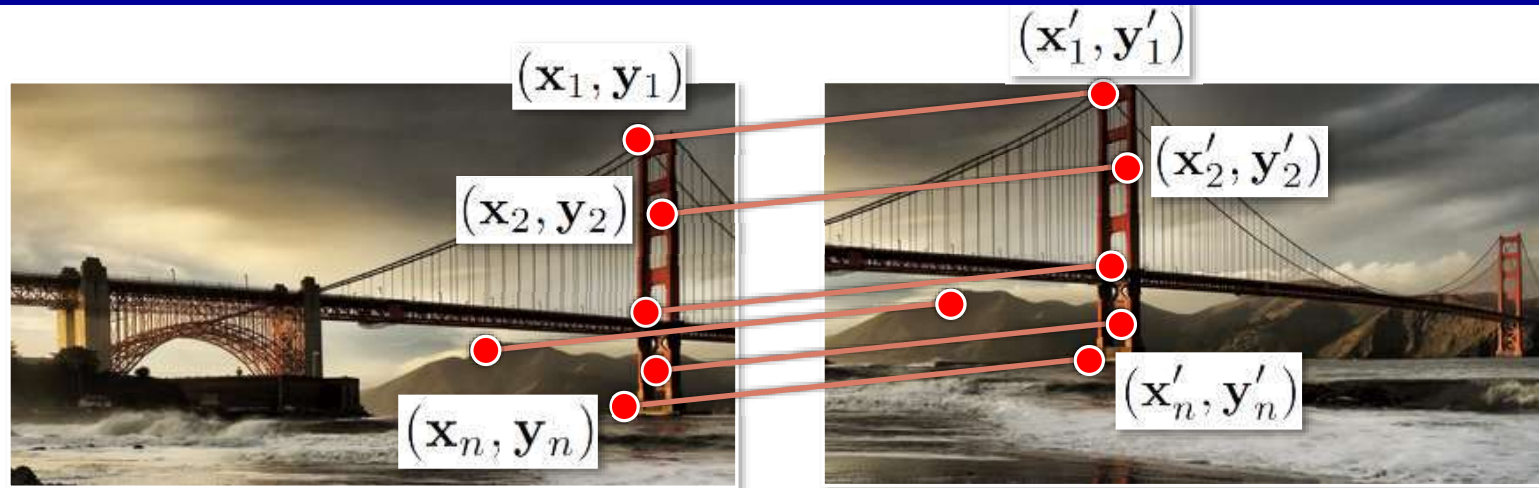
Find transform T that best “agrees” with the matches

Simple Case: Translation



How do we solve for
 (x_t, y_t) ?

Simple Case: Translation



Displacement of match= $(\mathbf{x}'_i - \mathbf{x}_i, \mathbf{y}'_i - \mathbf{y}_i)$

$$(\mathbf{x}_t, \mathbf{y}_t) = \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}'_i - \mathbf{x}_i, \frac{1}{n} \sum_{i=1}^n \mathbf{y}'_i - \mathbf{y}_i \right)$$



Function()

Function is a block of code, and it will contain a series of statements to execute whenever it is called in the program.

It is useful to reduce code duplication and improve code reusability.

you need to use the **def** keyword followed by the **function name** and parentheses **()**.

```
def function_name(parameters):
    statement(s)
    ...
    ...
    return [expr]
```

Function structure

```
def userdetails():
    ''' Function to get user details'''
    name = "Sunil Survaiya"
    location = "Campus Cham"
    print("Name: {}".format(name))
    print("Location: {}".format(location))

userdetails()
print("Introduction to AI")
```

Function example WITHOUT input arguments

```
def userdetails(name, location):
    ''' Function to get user details'''
    print("Name: {}".format(name))
    print("Location: {}".format(location))

userdetails("Sunil Survaiya", "Campus Cham")
print("Introduction to AI")
```

Function example with input arguments



Module with Example

Module is a file that will contain a set of objects such as functions, variables, classes, etc., to use in your applications.

Any file that contains a python code with a **.py extension**, we will call it a **module**

The modules are useful to reduce the code redundancy by keeping the common functionalities such as functions, variables, etc., in one place

```
1 def add(x, y):
2     return x + y
3 def subtract(x, y):
4     return x - y
5 users = ["Sunil", "Mehraj", "Hari"]
```

samplemodule.py

```
import samplemodule

a = samplemodule.add(10, 40)
b = samplemodule.subtract(100, 50)
print("Sum:", a)
print("Subtract:", b)
for user in samplemodule.users:
    print(user)
```

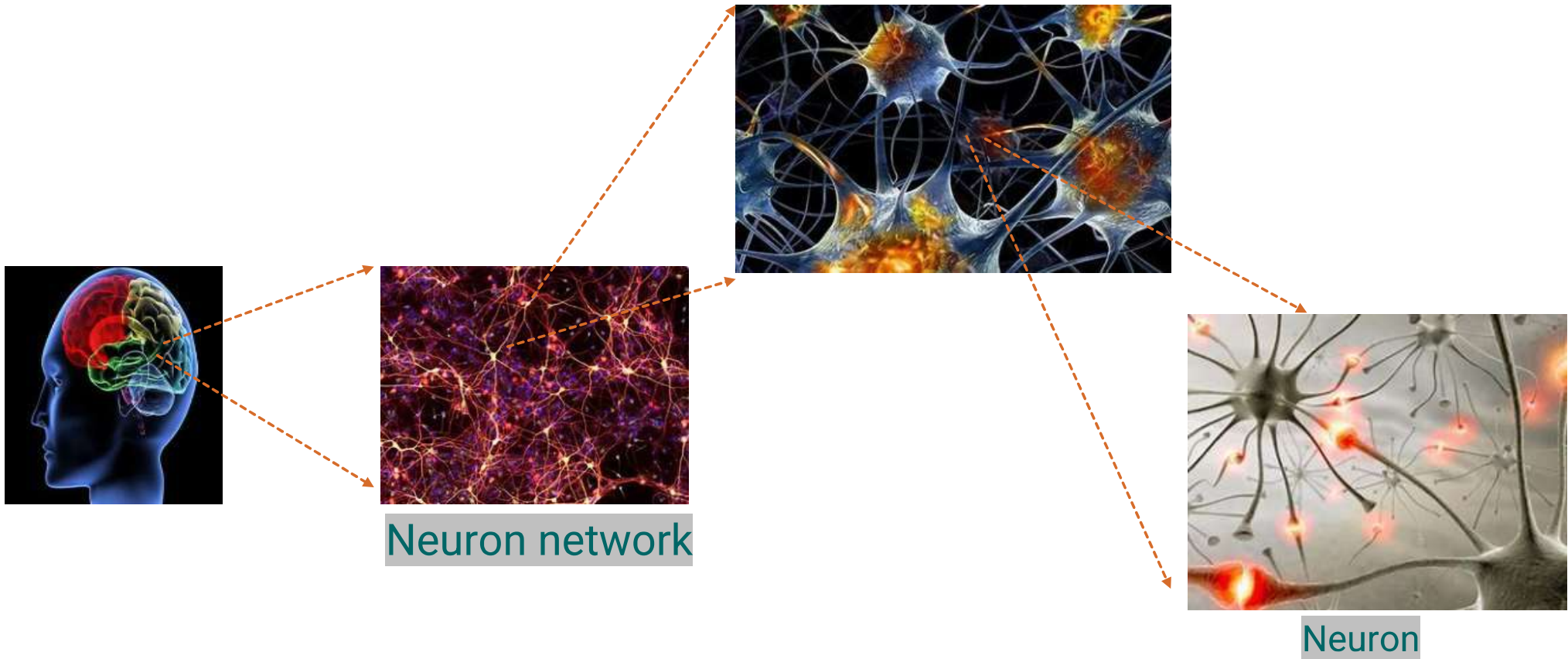
In test.py, import samplemodule.py

```
import samplemodule as sm

a = sm.add(10, 40)
b = sm.subtract(100, 50)
print("Sum:", a)
print("Subtract:", b)
for user in sm.users:
    print(user)
```

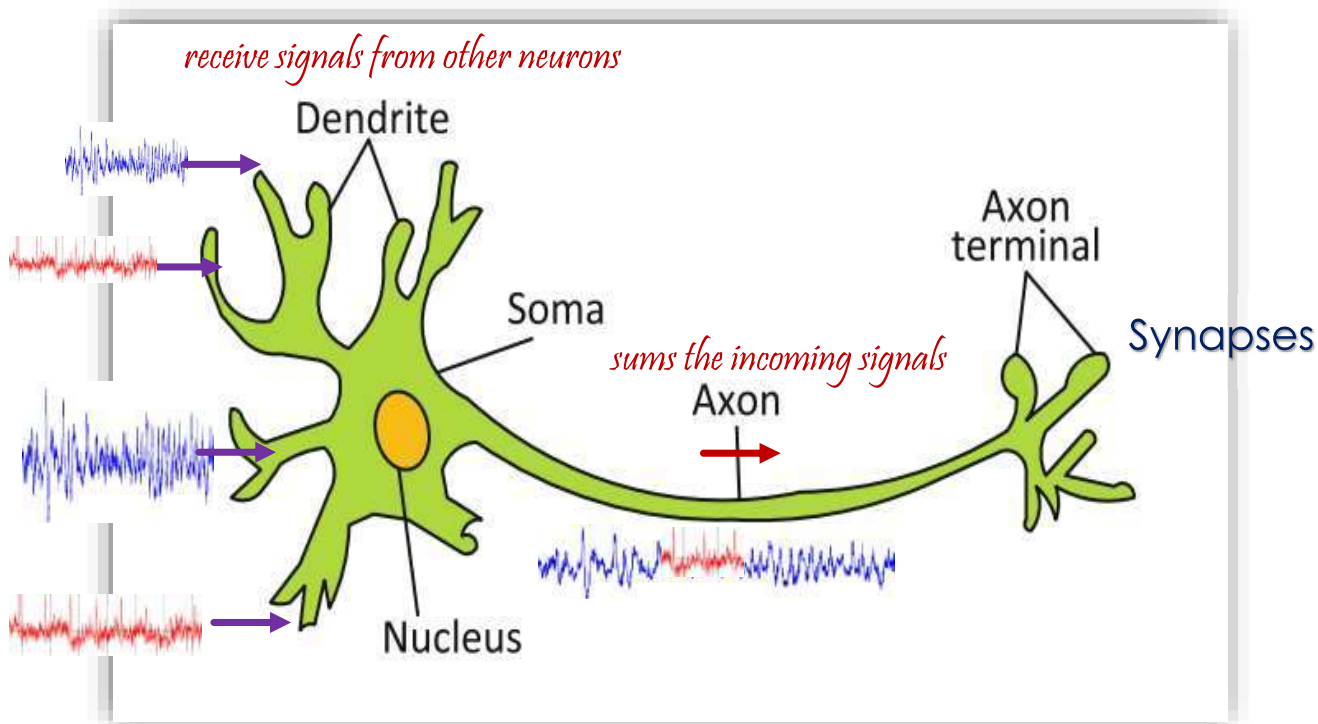
In test.py, abbreviation samplemodule

Neuron network | Neuron



Biological Neuron

A nerve cell *neuron* is a special biological cell that processes information



Dendrites: neuron receptors (inputs)

They are tree-like branches, responsible for receiving the information from other neurons.

Soma: neuron body.

It is the neuron cell body and is responsible for information processing.

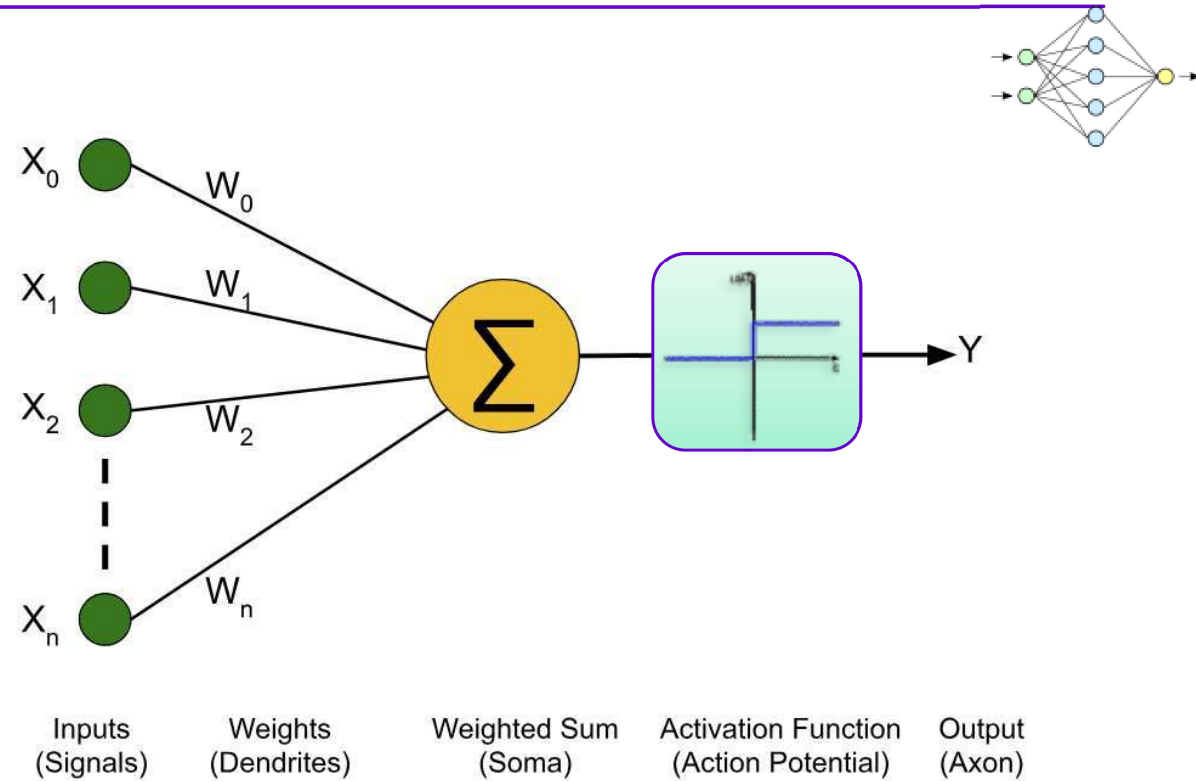
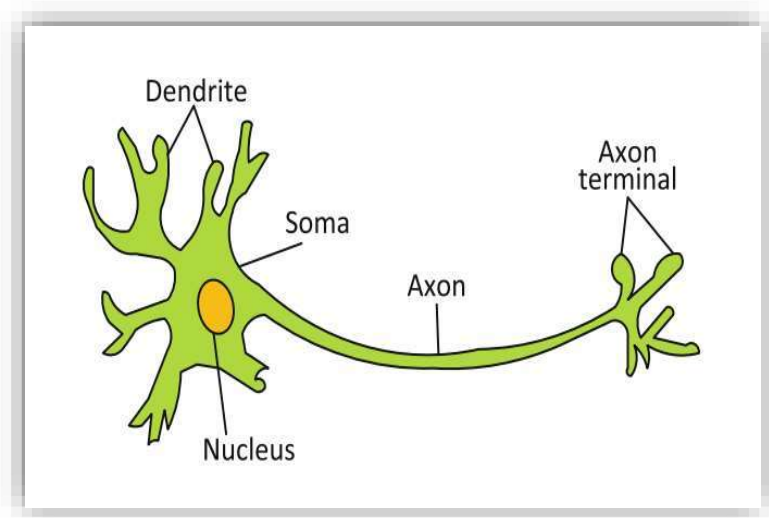
Axon: neuron output

It is just like a cable through which neurons send the information.

Synapses: information transfer between neurons

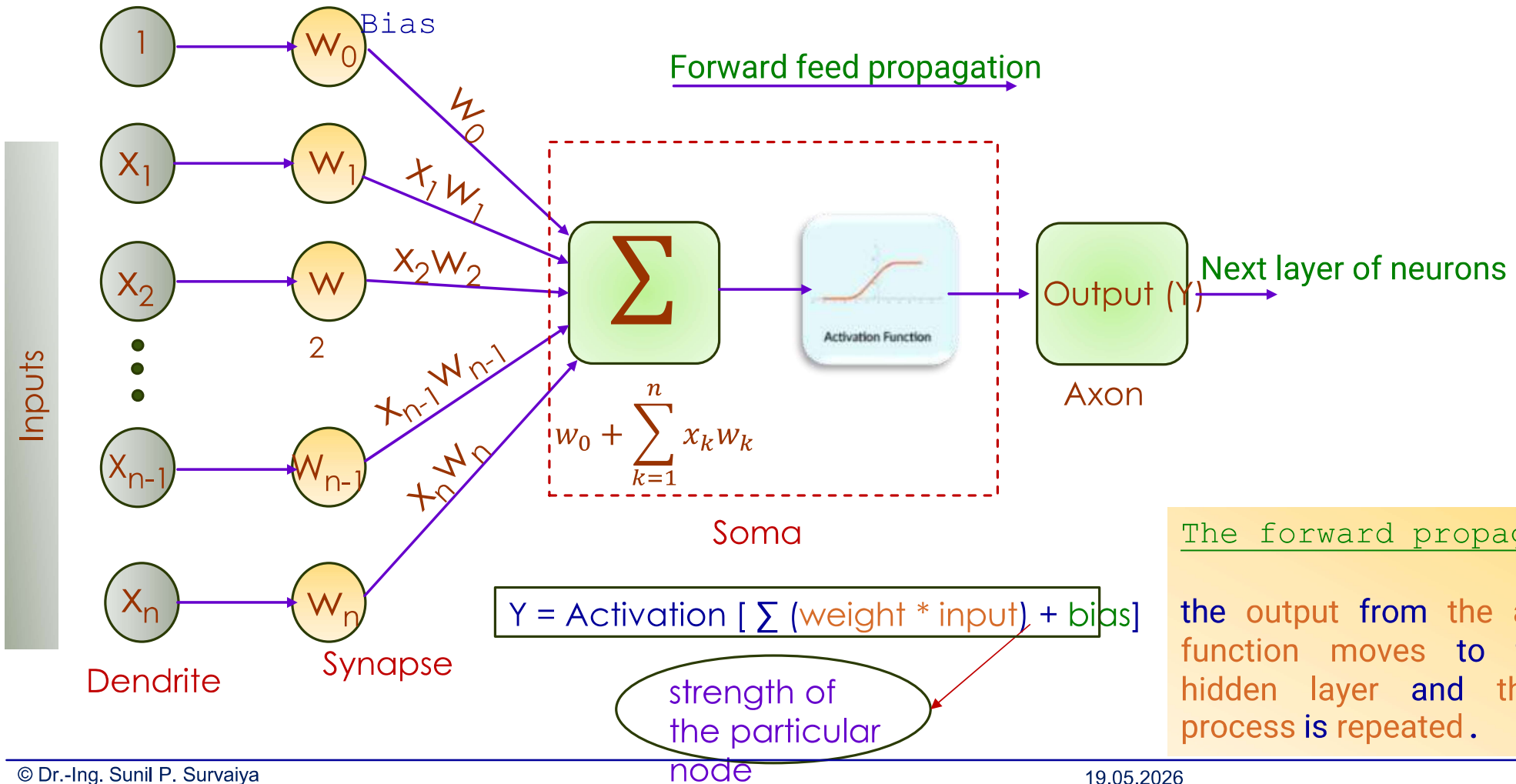
It is the connection between the axon and other neuron dendrites (electrical-chemical-electrical)

Neuron Mathematica Model





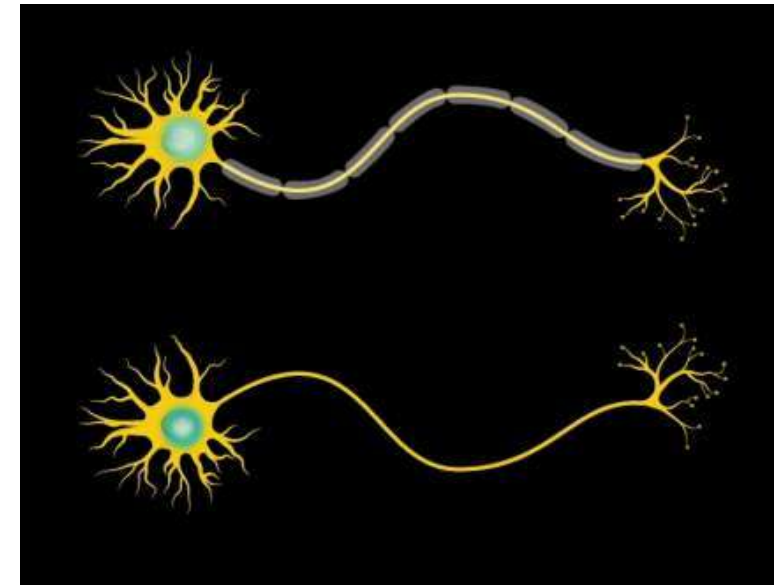
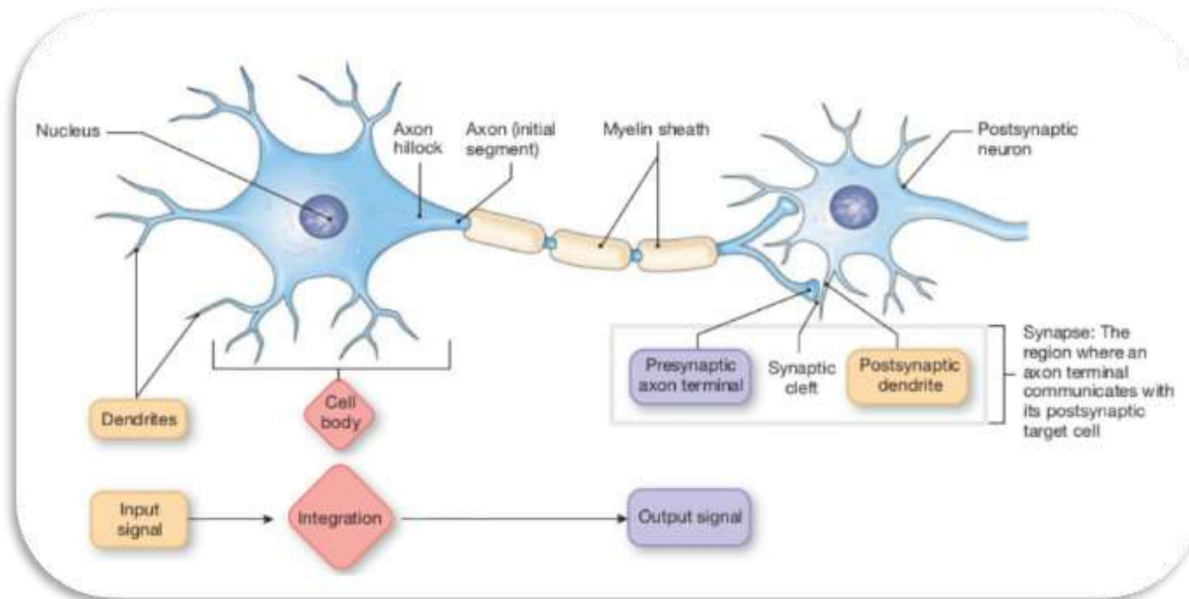
The Forward feed propagation



The forward propagation:

the output from the activation function moves to the next hidden layer and the same process is repeated.

Two biological Neuron:



© MIT McGovern Institute

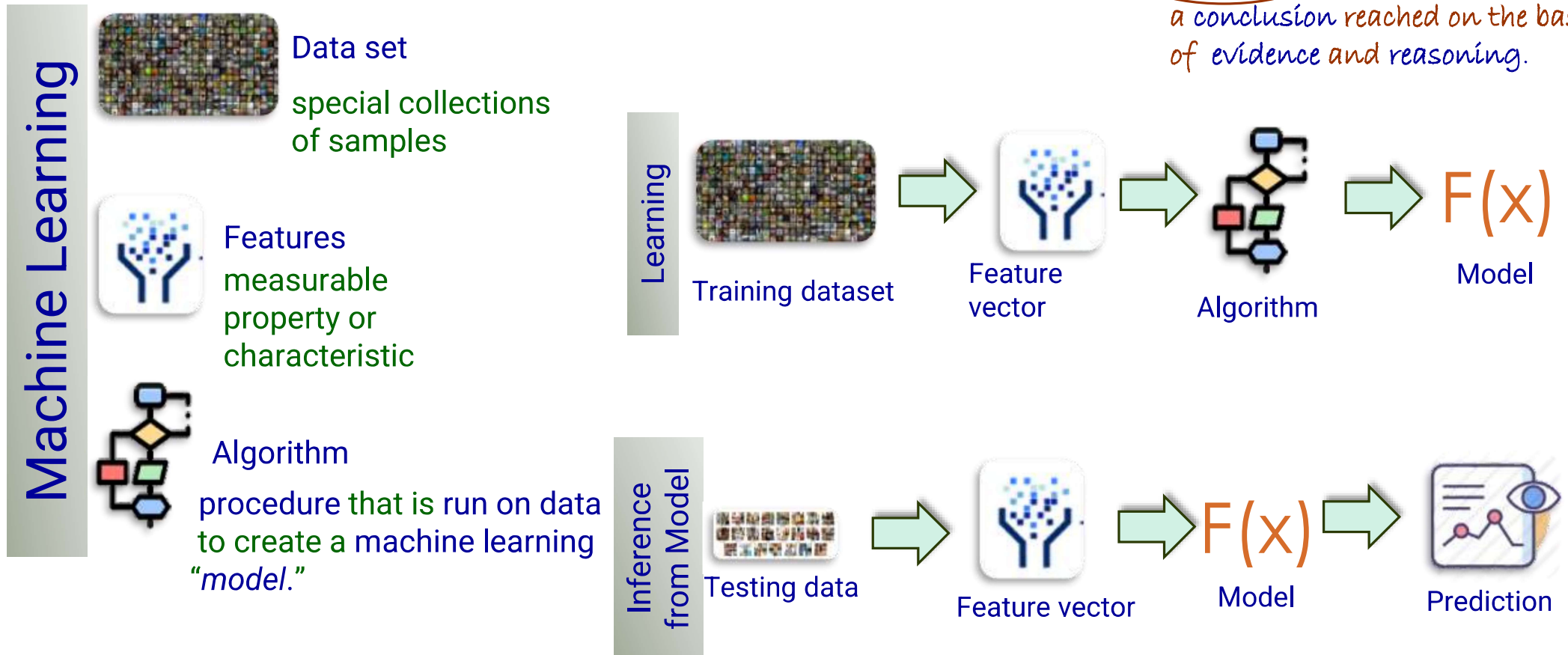
a **transponder** is a device that, upon receiving a signal, emits a different signal in response. The term is a blend of *transmitter* and *responder*.



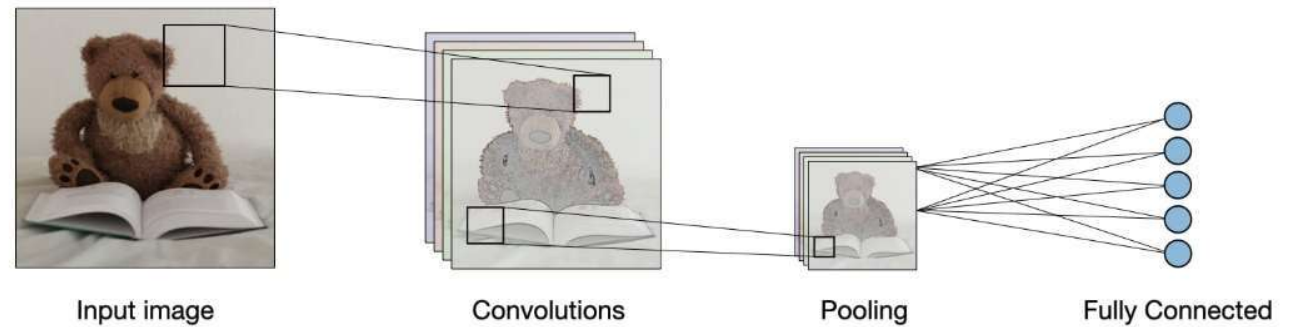
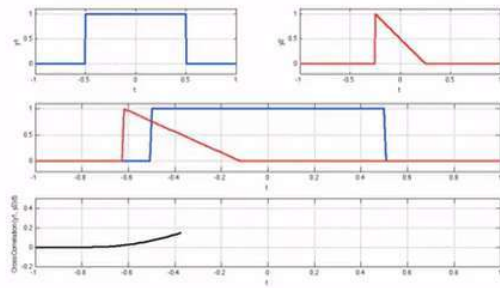
Machine Learning (ML)

machine learning = learning and inference

a conclusion reached on the basis of evidence and reasoning.



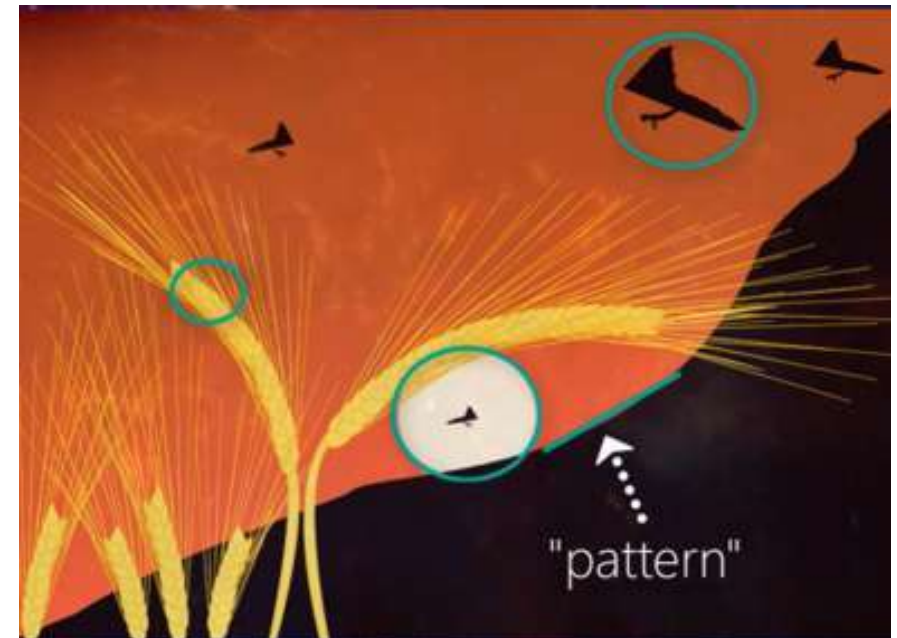
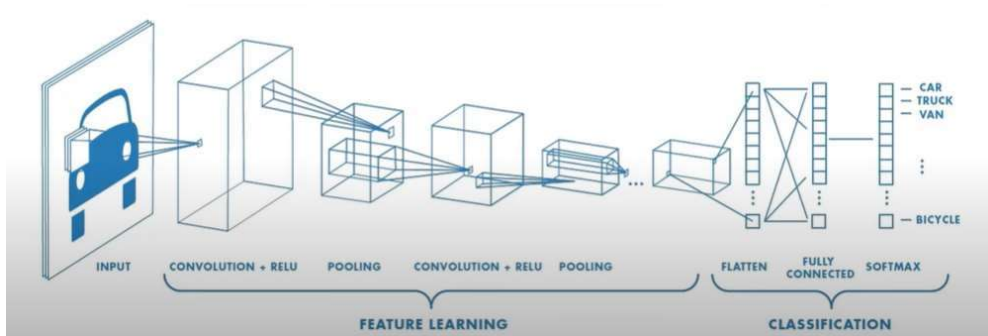
Convolution Neural Network (CNN)



Convolution for two pulses

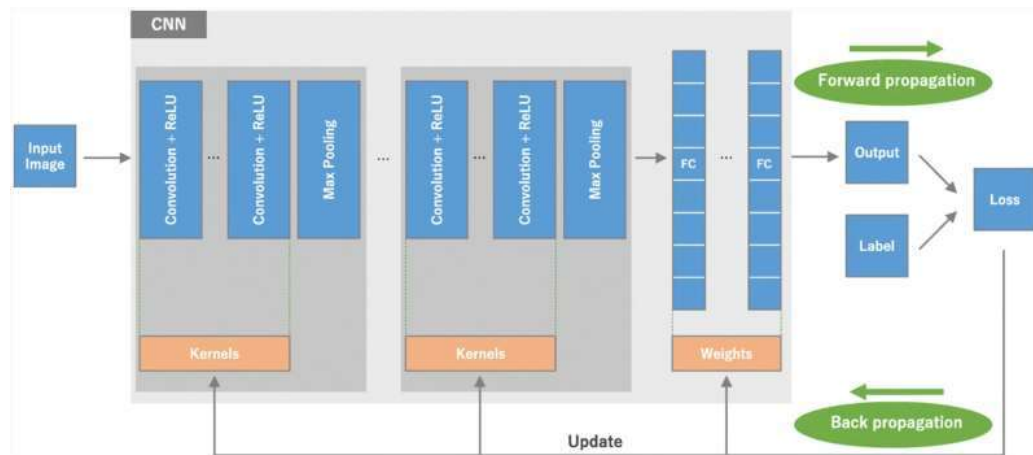
Convolution neural network (ConvNet)

Pattern in the images / Data sets



CNN Principle

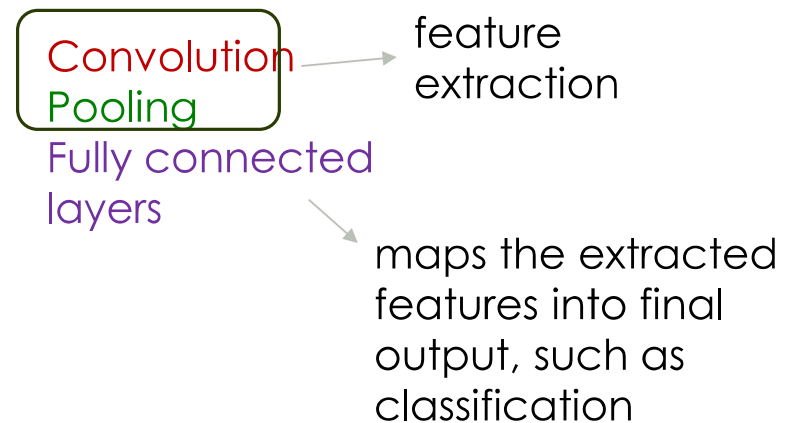
A “kernel” refers to the sets of learnable parameters applied in convolution operations



CNN Architecture

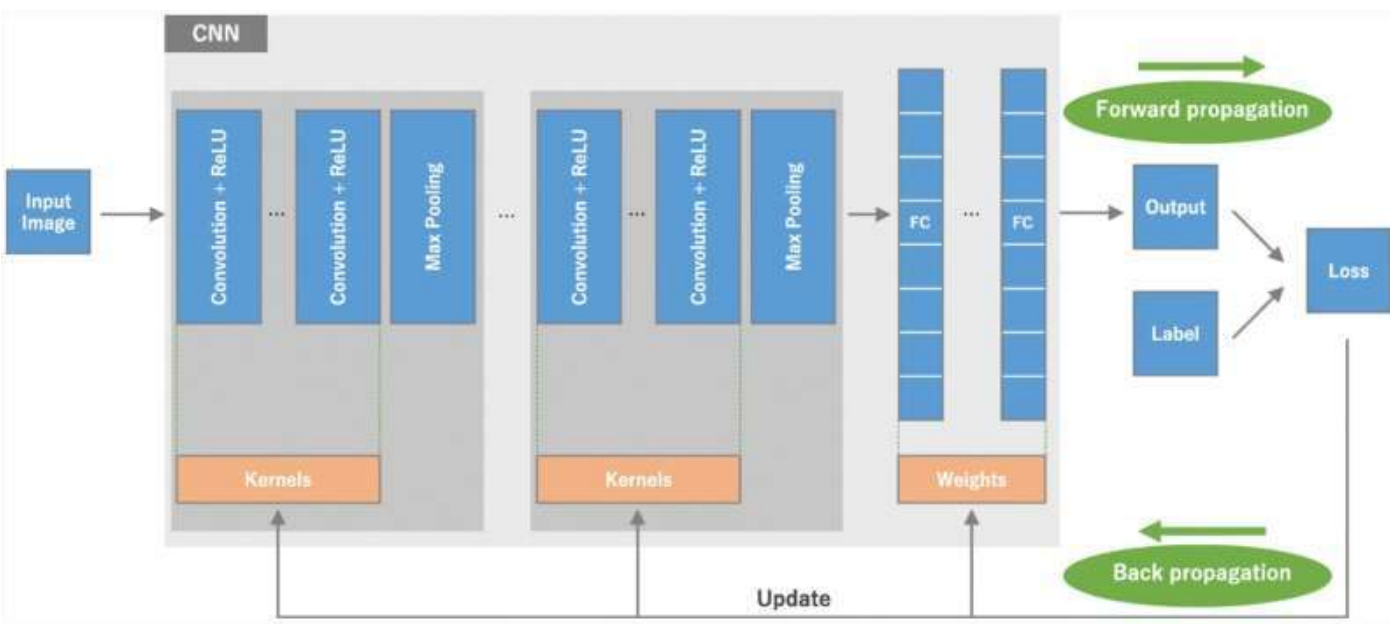
The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

Building blocks



A “hyperparameter” refers to a variable that needs to be set before the training process starts.

CNN Principle

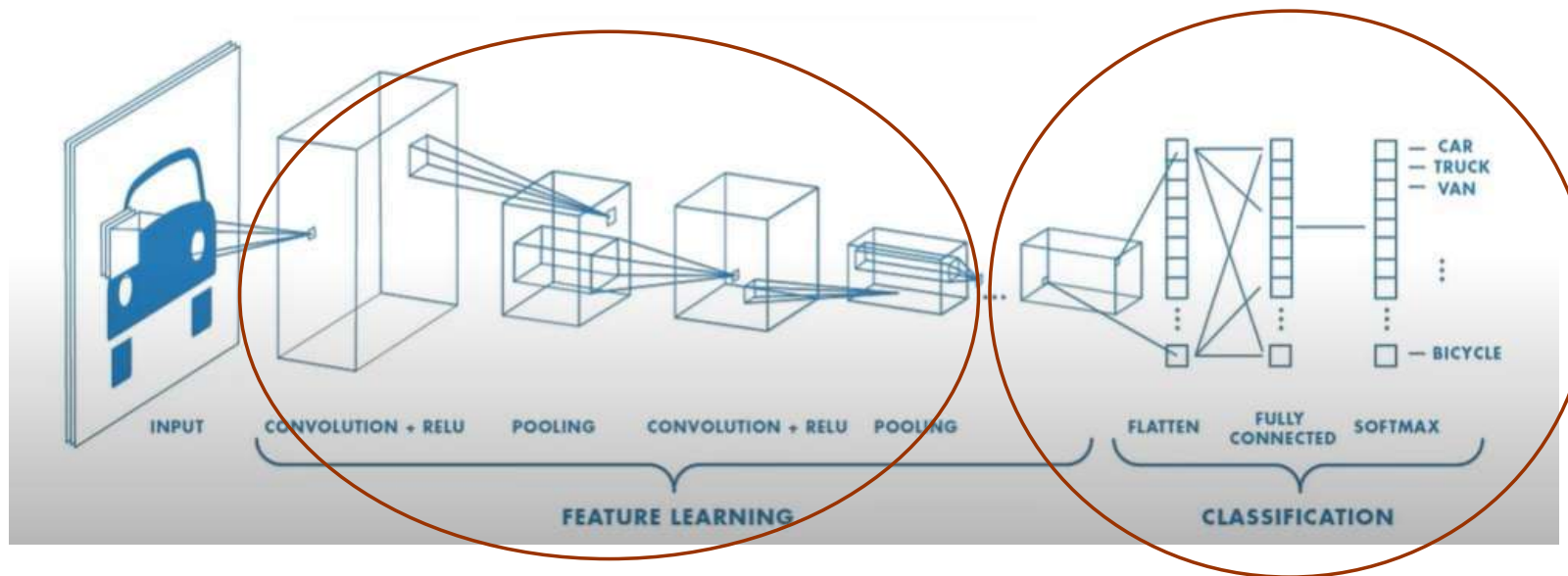


Using multiple convolution layers in a CNN allows the network to learn increasingly **complex features from the input image or video**.

The **first convolution** layers learn simple features, such as **edges and corners**.

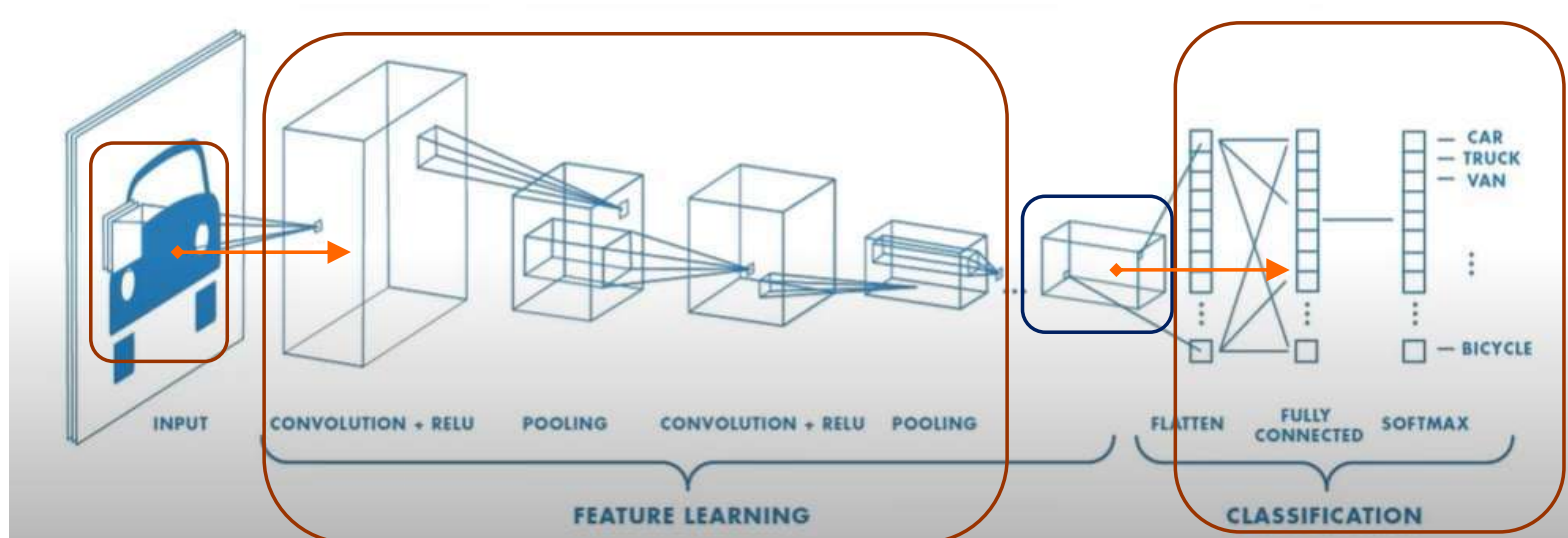
The **deeper convolution layers** learn more complex features, such as **shapes and objects**.

Convolution neural network (ConvNet)

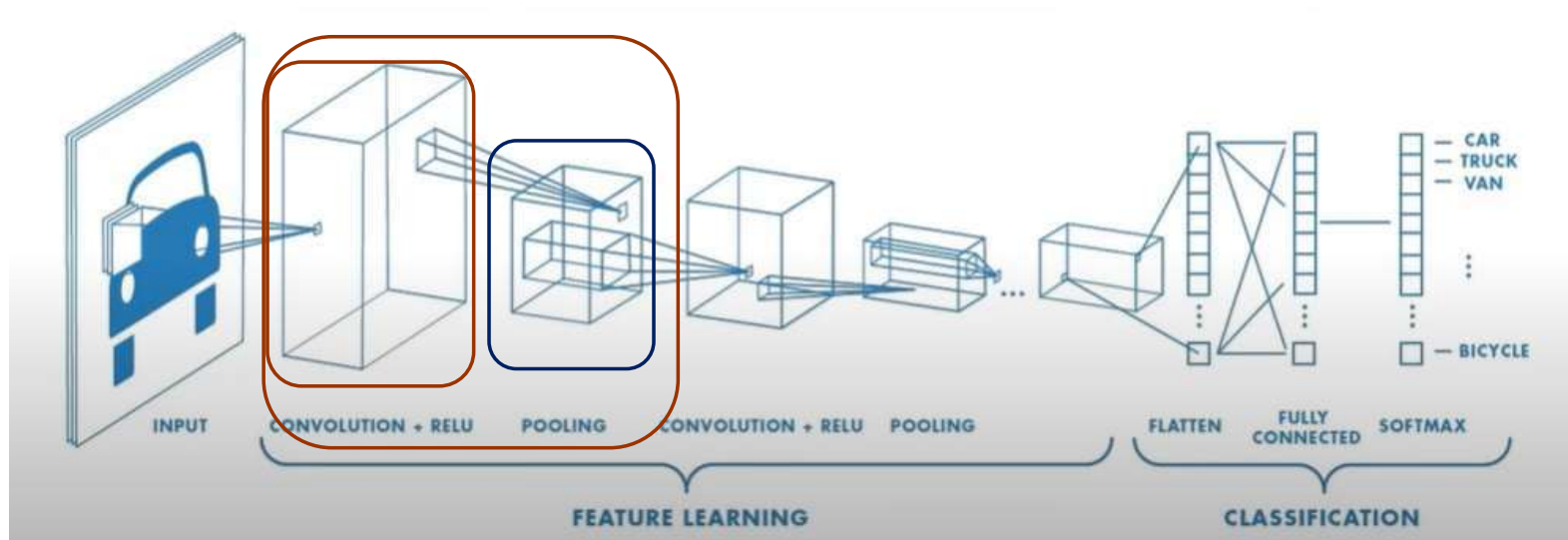




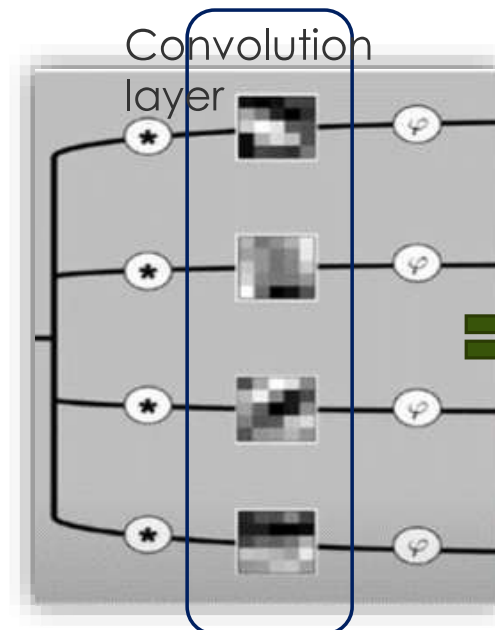
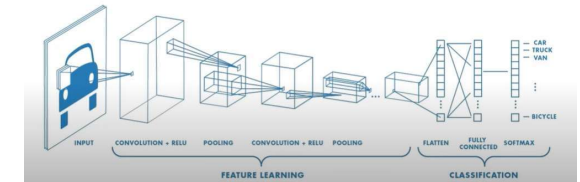
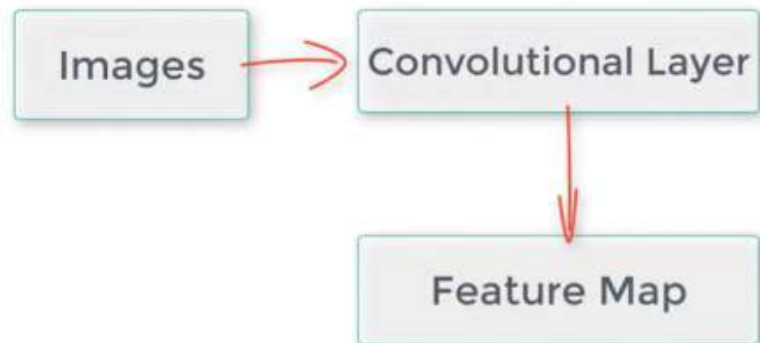
Convolution neural network (ConvNet)



Convolution neural network (ConvNet)

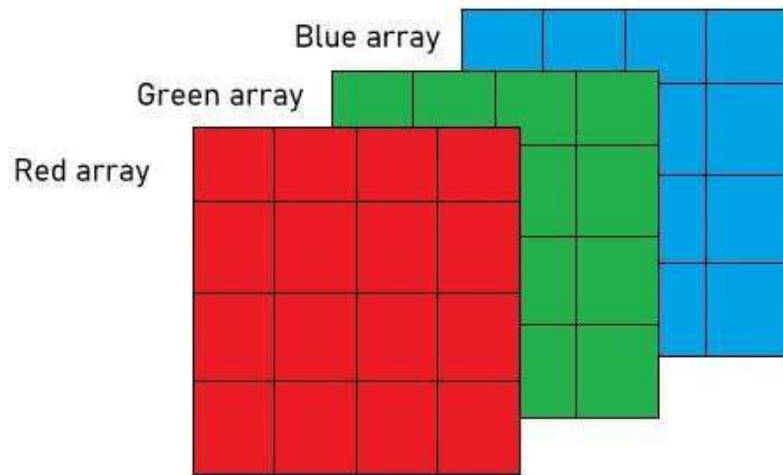


Convolution layer:

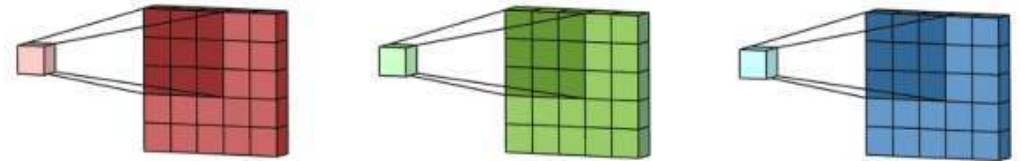


ConvNet

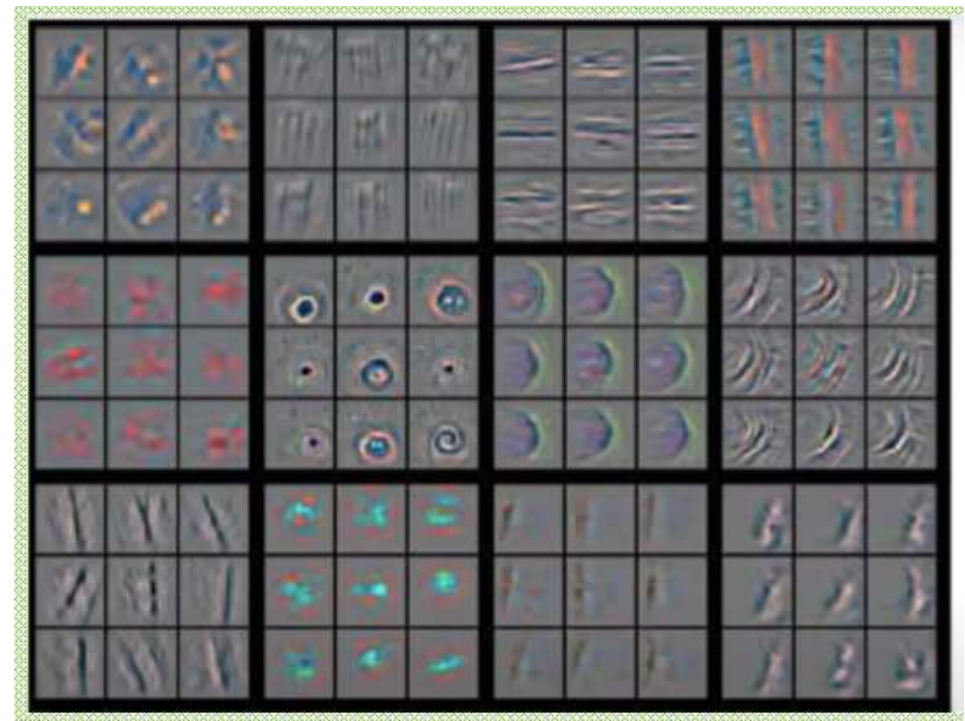
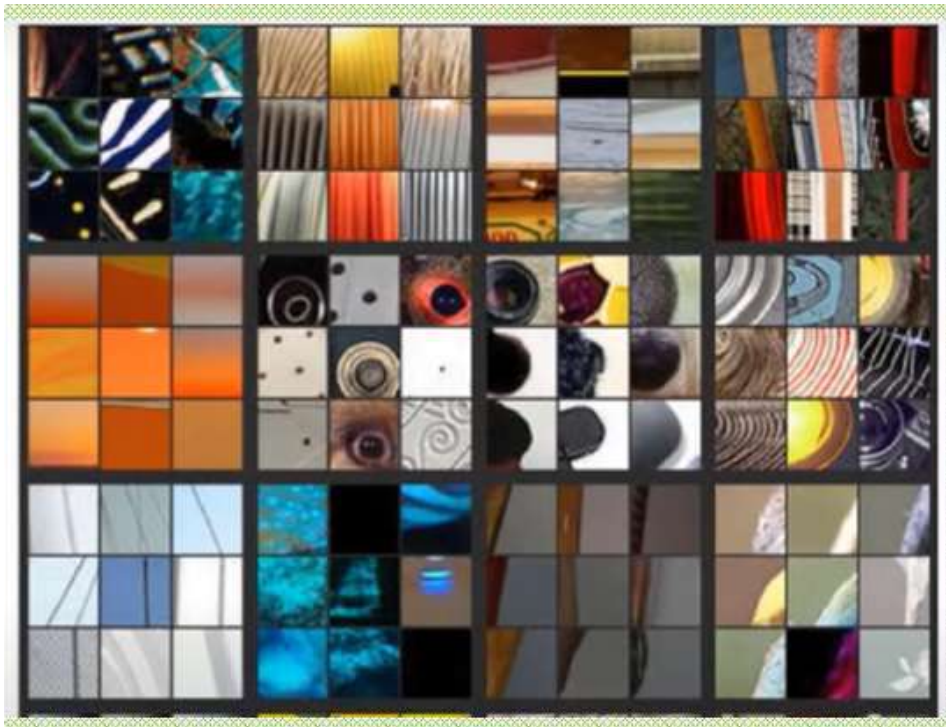
digital images are often composed of **three channels (RGB)** which are represented in three different matrices.



For an RGB image, there are typically **separate kernels for each color channel** because different features might be more visible or relevant in one channel compared to the others.

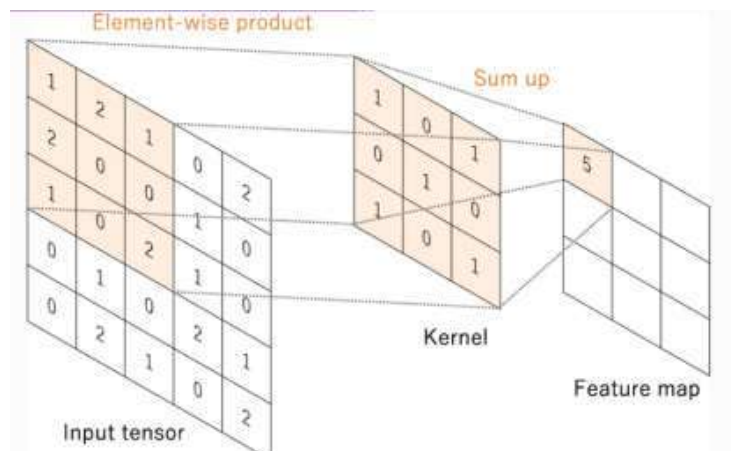


Kernel: Output feature map - Depth



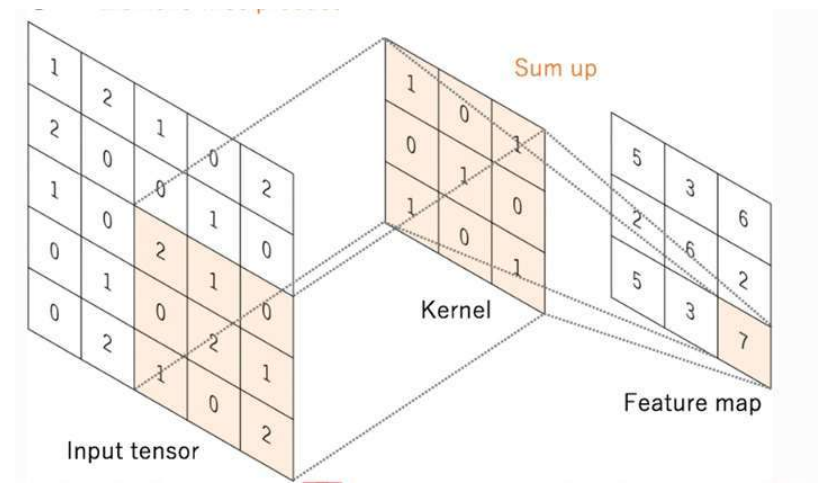
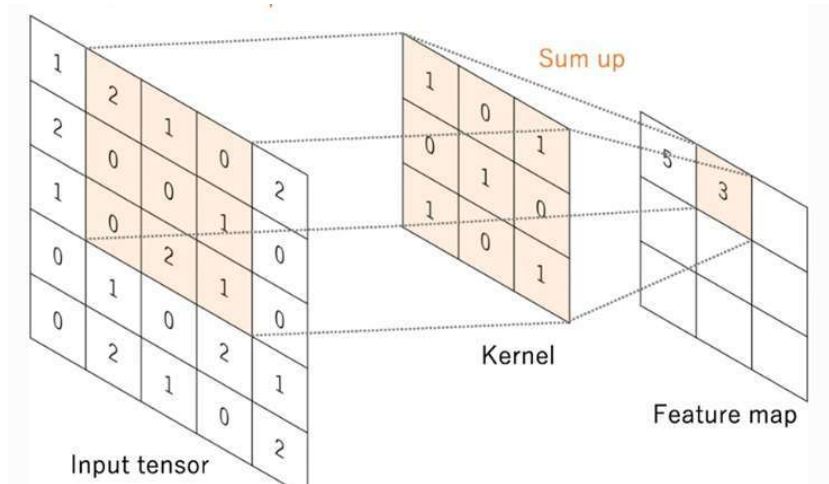
Convolution Layer

A convolution layer is a **fundamental** component of the **CNN architecture** that performs **feature extraction**, which typically consists of a **combination** of **linear** and **nonlinear** operations, i.e., **convolution operation** and **activation function**.



- An element-wise product between each element of the kernel and the input tensor is calculated at each location of the tensor.
- Summed to obtain the output value

CNN Layer

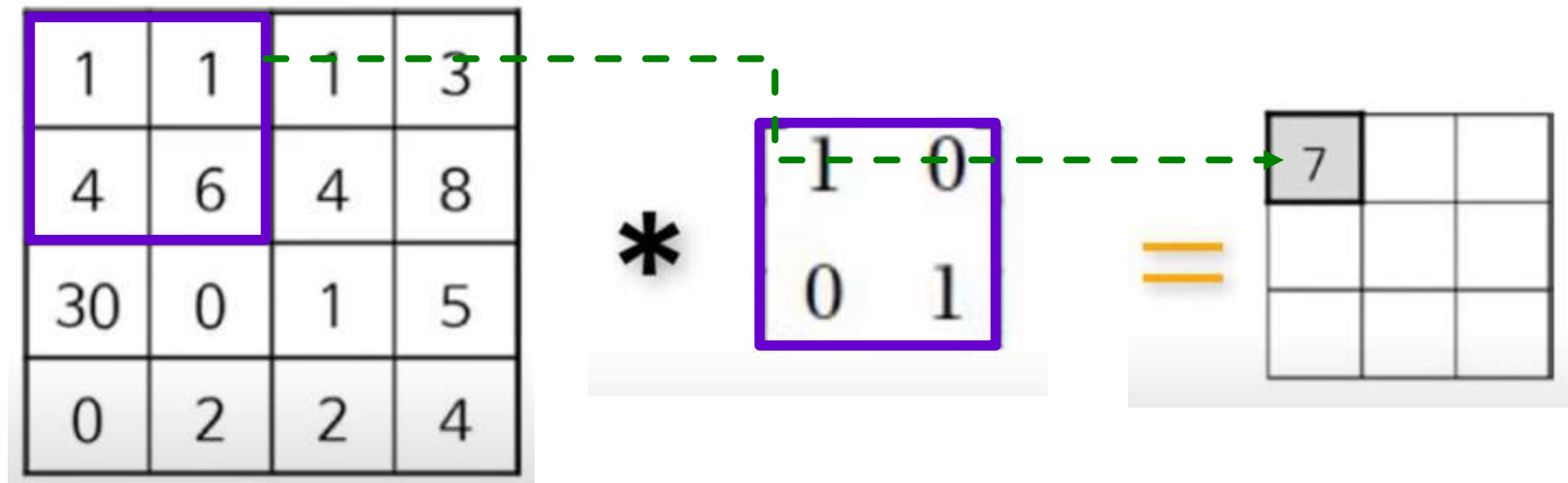


Two key hyperparameters that define the convolution operation are **size and number of kernels**.

The former is typically 3×3 , but sometimes 5×5 or 7×7 . The latter is arbitrary and determines the depth of output feature maps.



Convolution layer:



$$1 \times 1 + 1 \times 0 + 4 \times 0 + 6 \times 1 = 7$$

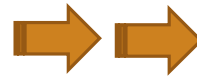


Convolution layer:

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 4 & 6 & 4 & 8 \\ 30 & 0 & 1 & 5 \\ 0 & 2 & 2 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 5 & & \\ & & & \\ & & & \\ & & & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 4 & 6 & 4 & 8 \\ 30 & 0 & 1 & 5 \\ 0 & 2 & 2 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 5 & 9 & \\ & & & \\ & & & \\ & & & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 4 & 6 & 4 & 8 \\ 30 & 0 & 1 & 5 \\ 0 & 2 & 2 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 5 & 9 & \\ 4 & & & \\ & & & \\ & & & \end{bmatrix}$$



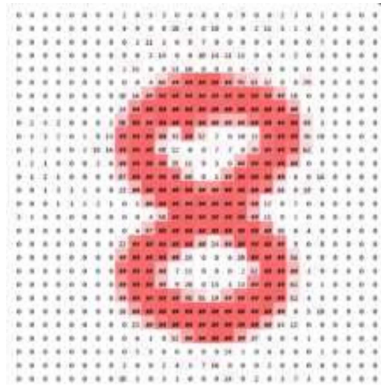
$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 4 & 6 & 4 & 8 \\ 30 & 0 & 1 & 5 \\ 0 & 2 & 2 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 5 & 9 & \\ 4 & 7 & 9 & \\ 32 & 2 & 5 & \end{bmatrix}$$



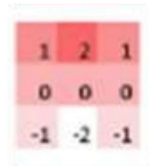
Convolution layer:

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 4 & 6 & 4 & 8 \\ 30 & 0 & 1 & 5 \\ 0 & 2 & 2 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 5 & 9 \\ 4 & 7 & 9 \\ 32 & 2 & 5 \end{bmatrix}$$

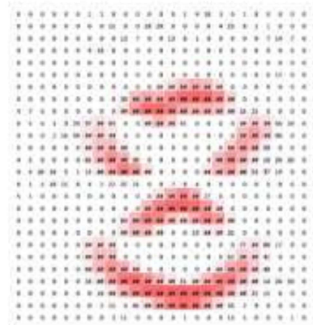
Kernel: depth of output feature maps



Input

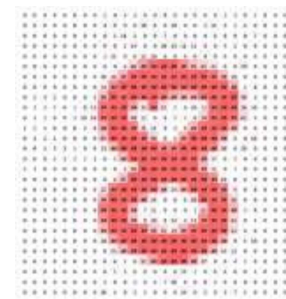


Kernel

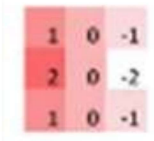


Output

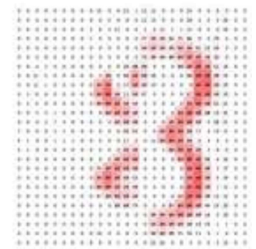
Horizontal edge detector



Input

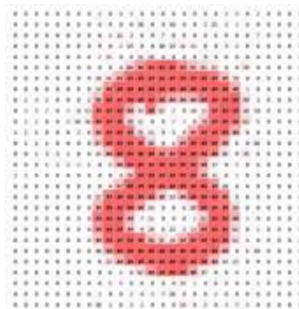


Kernel

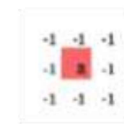


Output

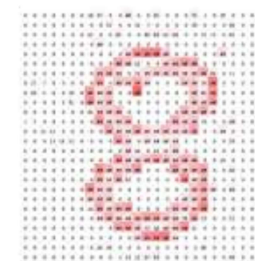
Vertical edge detector



Input



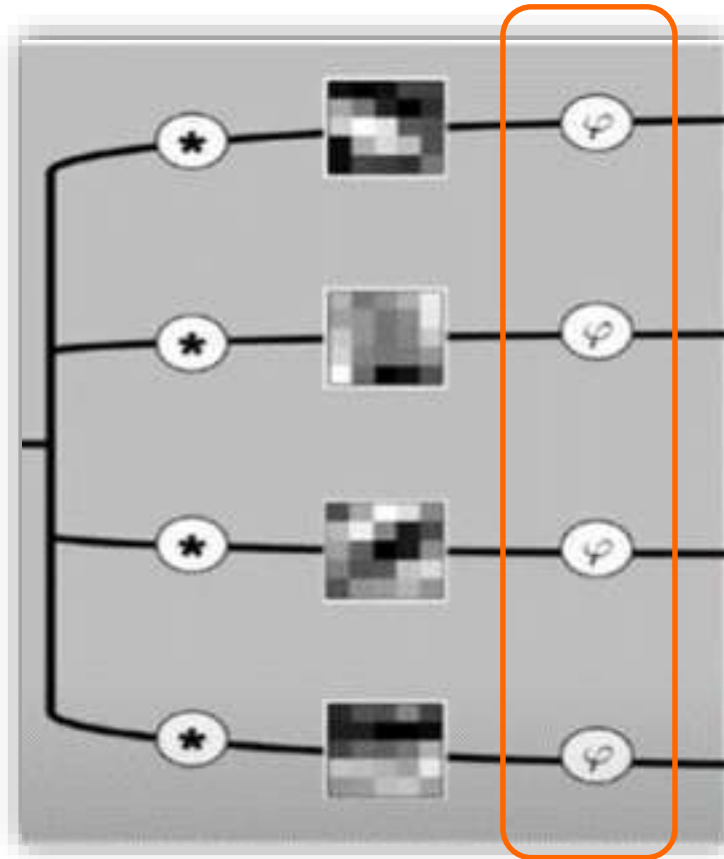
Kernel



Output

Outline detector

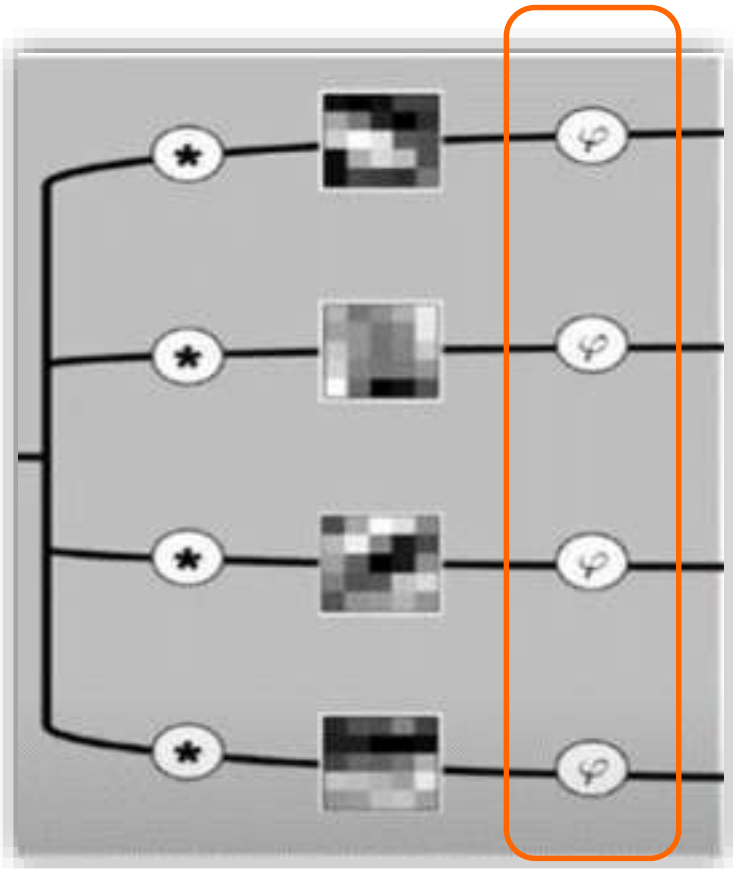
Convolution layer:



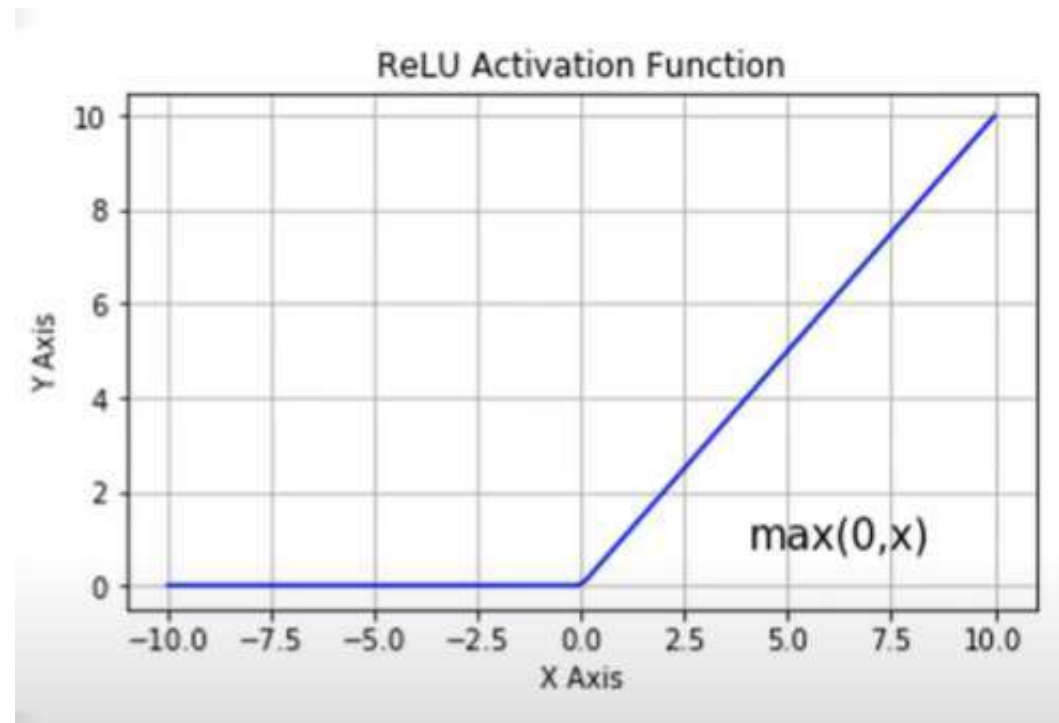
Activation function

Convolution layer:

Sigmoid or tanh functions



Activation function

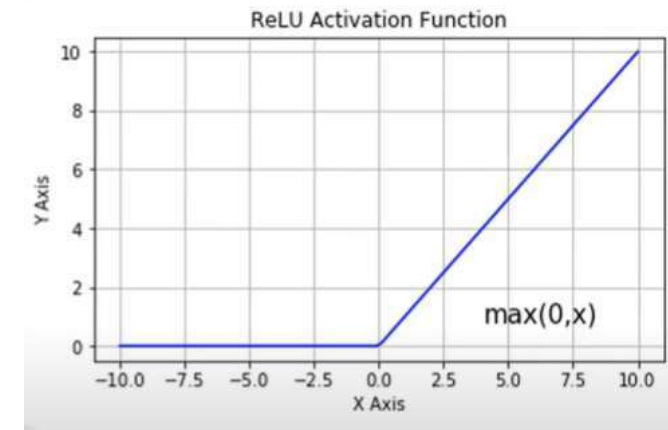




Rectified linear activation (ReLU) function

The **rectified linear activation function** or **ReLU** for short is a **piecewise linear function** that will **output the input directly** if it is **positive**, otherwise, it will **output zero**.

It has become the **default activation function** for many types of **neural networks** because a model that uses it is **easier to train** and often achieves **better performance**.



0	1	-4
0	2	0
-1	4	3

Filter output



0	1	0
0	2	0
0	4	3

Filter output after ReLU

ReLU leaves pixels with positive values in filter output as is, and replacing negative values with zero (or a small number in case of Leaky ReLU).

ReLU activation function to filter
output



What is Pooling?

A pooling layer is another essential building block of CNN. It tries to figure out whether a particular region in the image has the feature we are interested in or not.

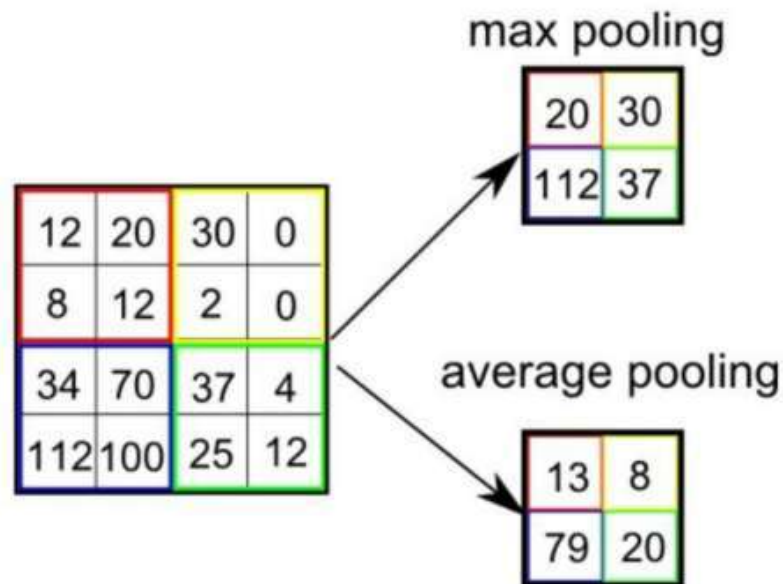
It summarizes the featured map so that the model will not need to be trained on precisely positioned features, making a model more reliable and robust.

The pooling layer looks at more significant regions (having multiple patches) of the image & captures aggregate statistics (min, max, average). In other words, it makes the network invariant to local transformations.



Max and Average Pooling

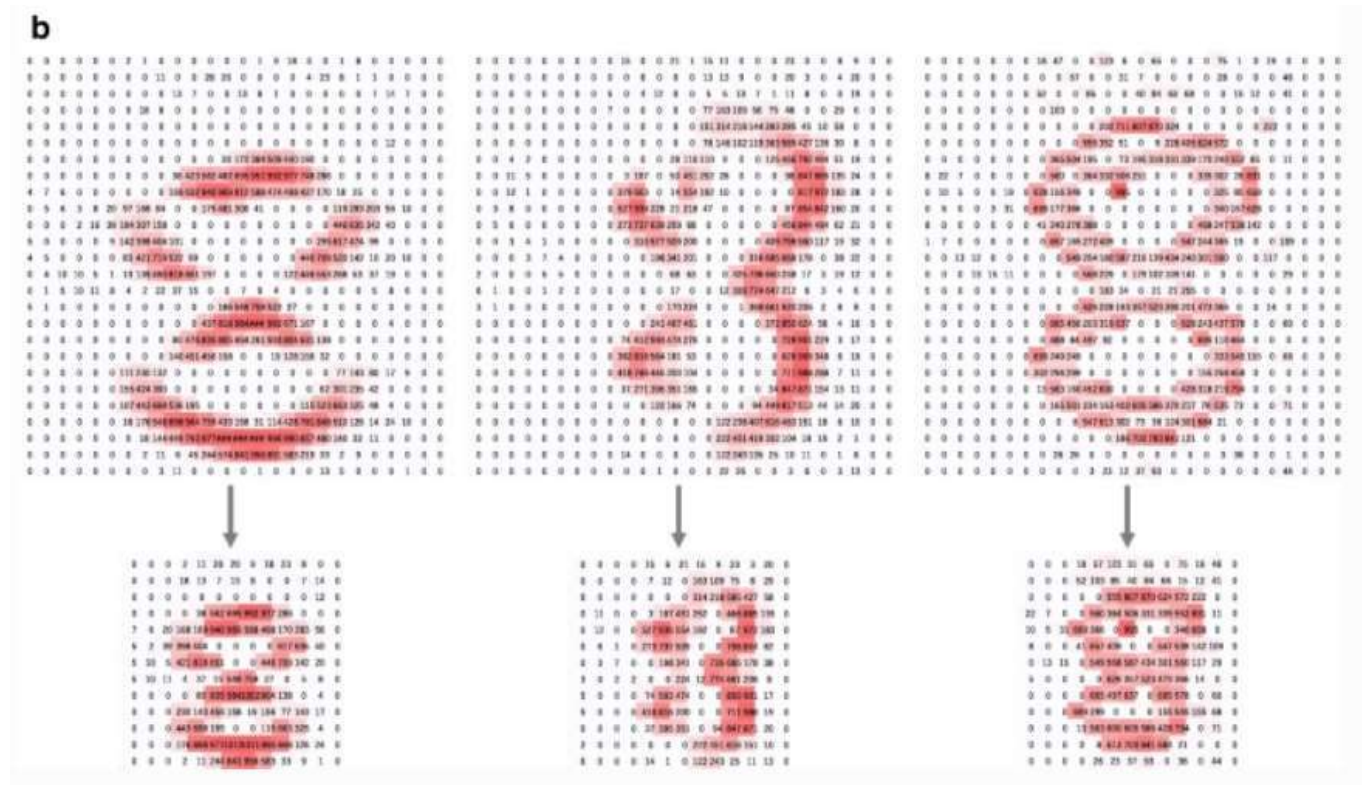
Image denoising is to remove noise from a noisy image, so as to restore the true image



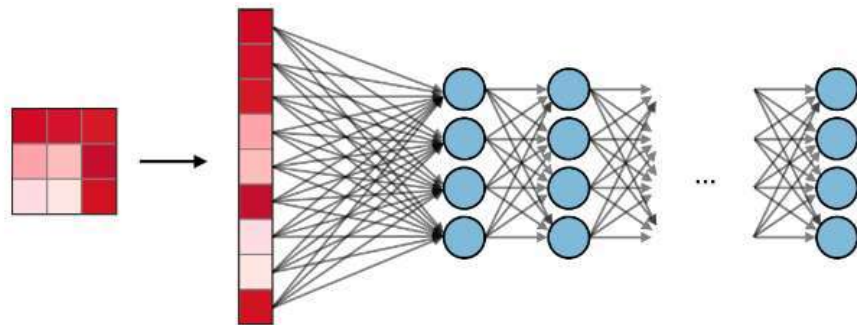
Max Pooling also performs as a Noise Suppression. It performs de-noising along with dimensionality reduction.

Average Pooling simply performs dimensionality reduction. Hence, we can say that Max Pooling performs a lot better than Average Pooling.

Max Pooling

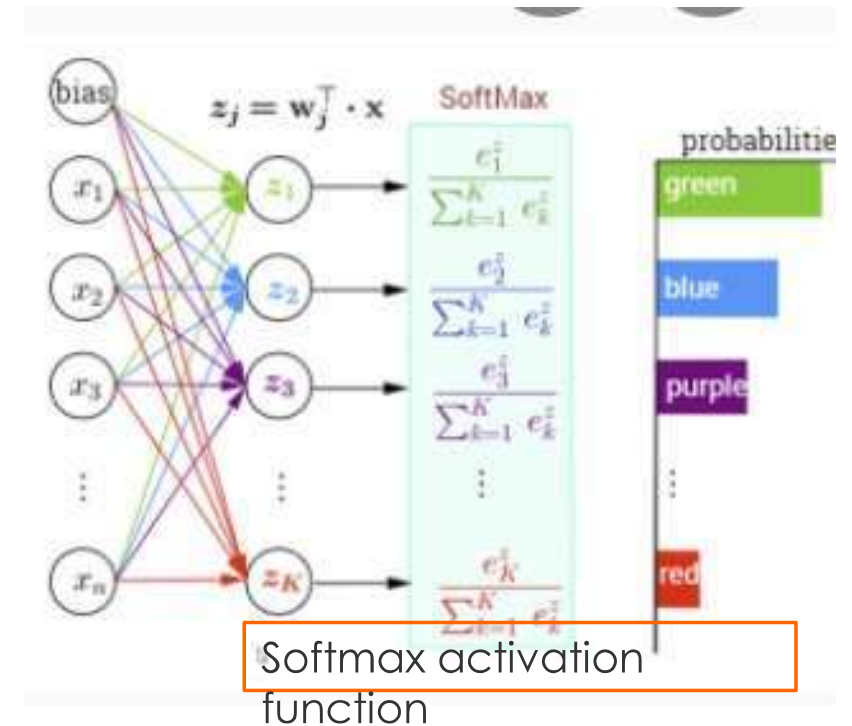


Fully connected layer



The output feature maps of the final convolution or pooling layer is typically flattened, i.e., transformed into a one-dimensional (1D) array of numbers (or vector), and connected to one or more fully connected layers, also known as dense layers

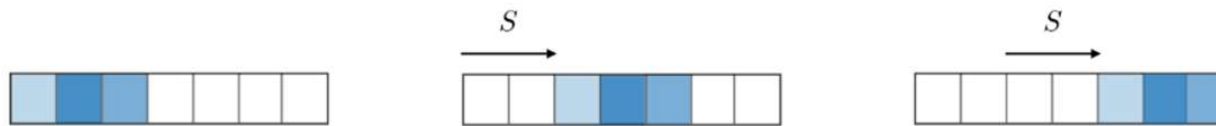
The final fully connected layer typically has the same number of output nodes as the number of classes.



Stride and Zero padding

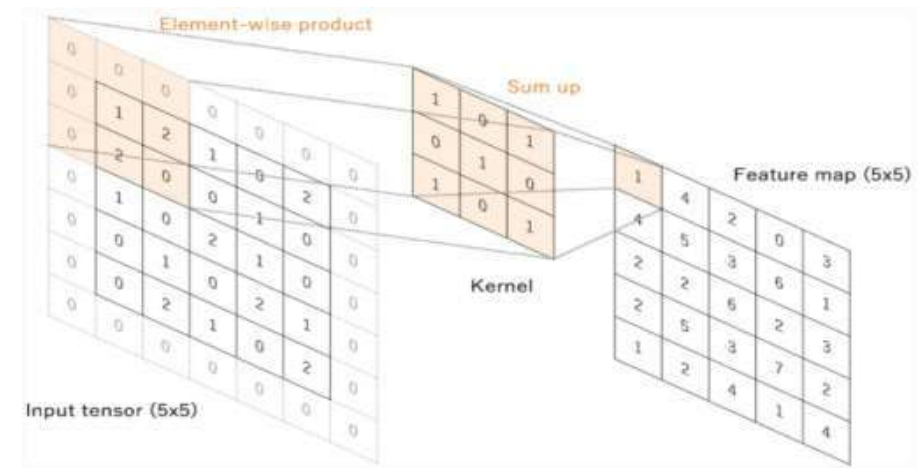
Stride

The **stride (S)** denotes the number of pixels by which the window moves after each operation.



Zero padding

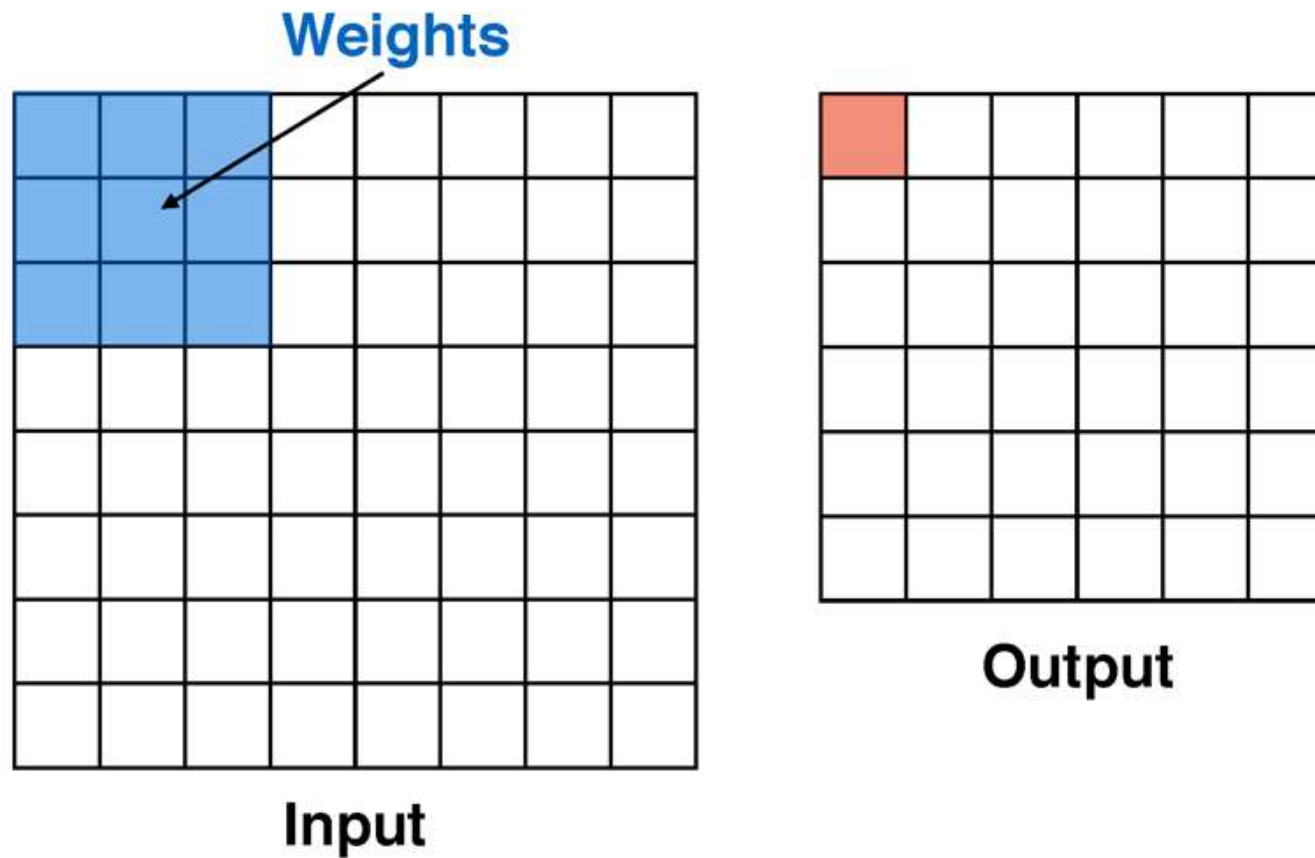
Rows and columns of zeros are added on each side of the input tensor, so as to fit the center of a kernel on the outermost element.





Convolution: Stride

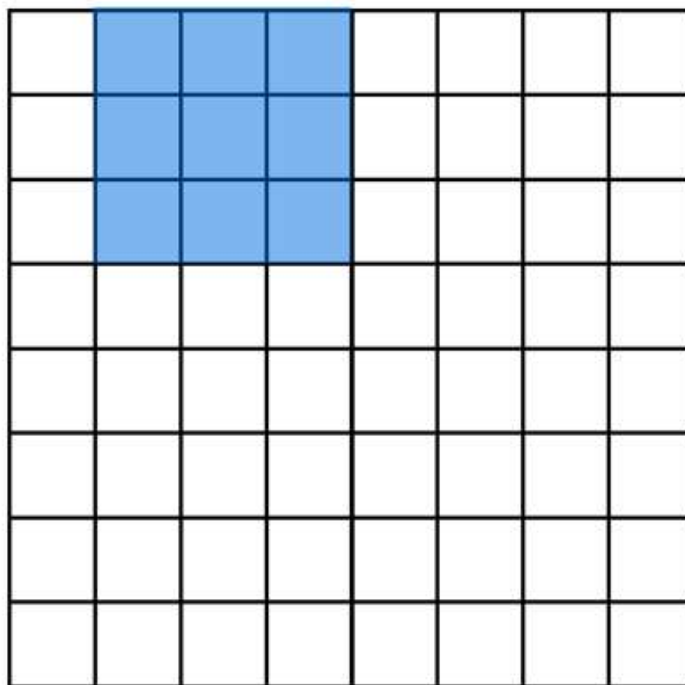
During convolution, the weights „slide“ along the input to generate each output.



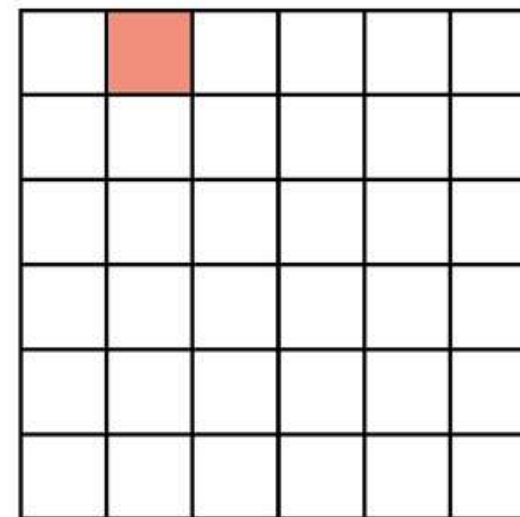


Convolution: Stride

During convolution, the weights „slide“ along the input to generate each output.



Input

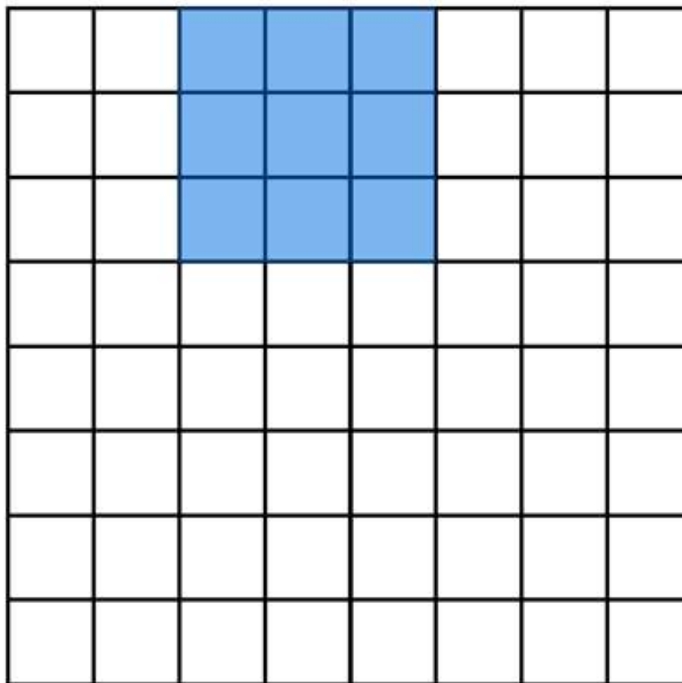


Output

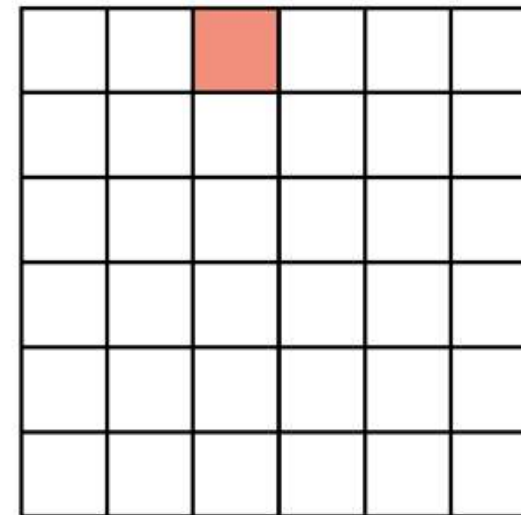


Convolution: Stride

During convolution, the weights „slide“ along the input to generate each output.



Input

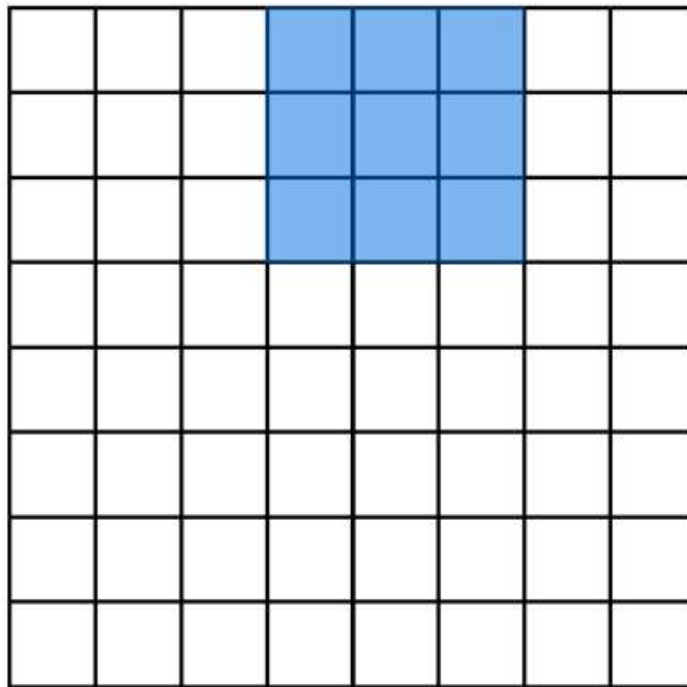


Output

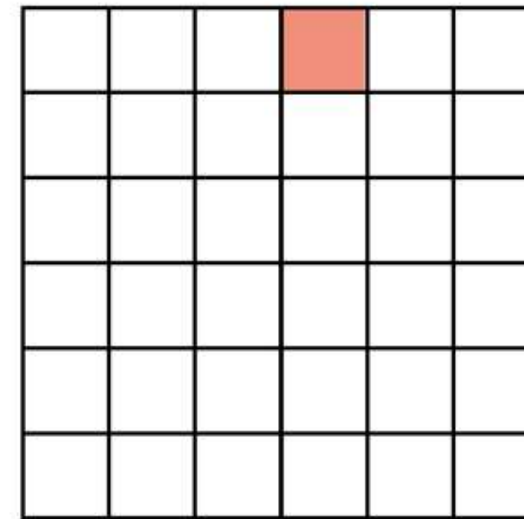


Convolution: Stride

During convolution, the weights „slide“ along the input to generate each output.



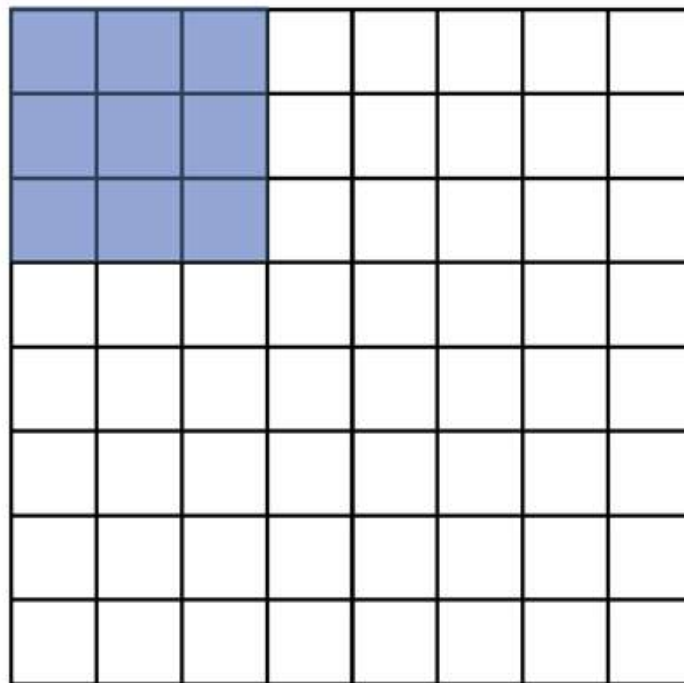
Input



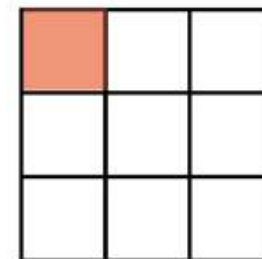
Output



Convolution: Stride = 2

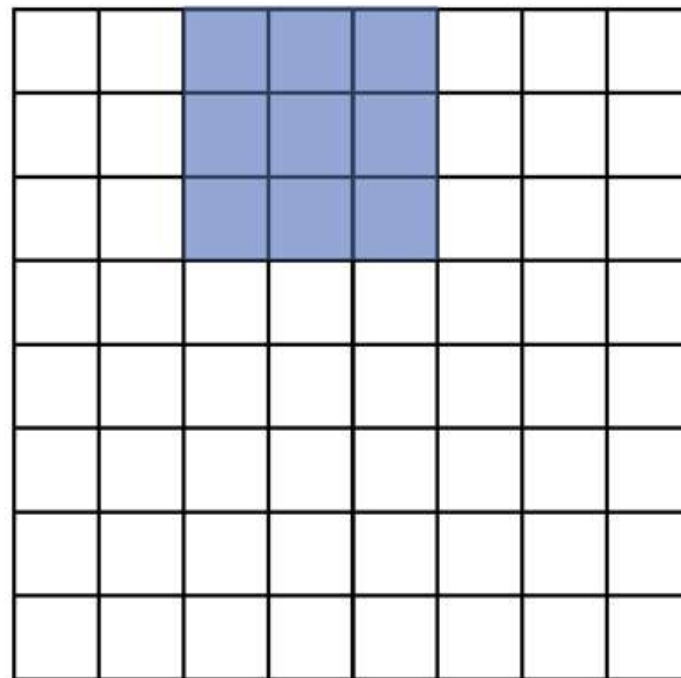


Input

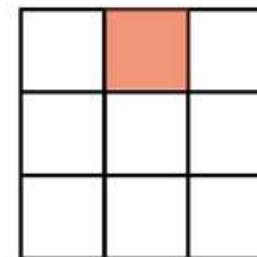


Output

Convolution: Stride

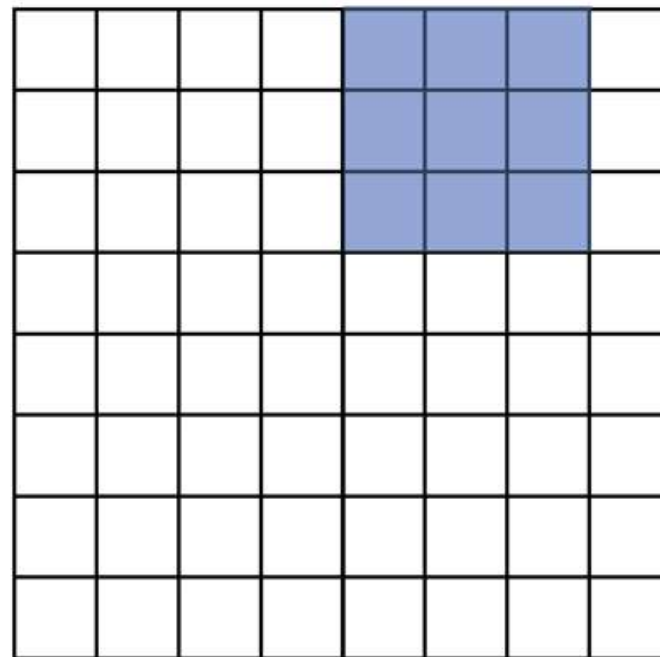


Input

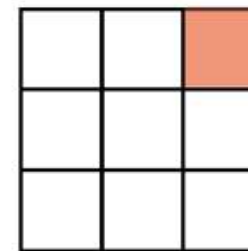


Output

Convolution: Stride



Input



Output

- With certain strides, we may not be able to cover all of the inputs.
- The output is also half the input size

Convolution: Padding, Pad = 1, Stride = 2



0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output

Convolution: Padding, Pad = 1, Stride = 2



0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output

Convolution: Padding, Pad = 1, Stride = 2



0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output

Convolution: Padding, Pad = 1, Stride = 2



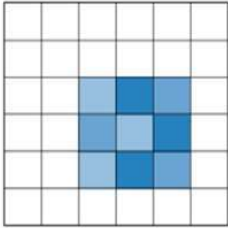
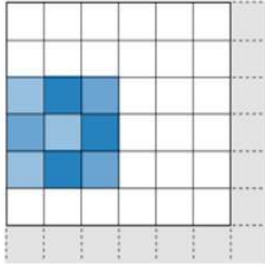
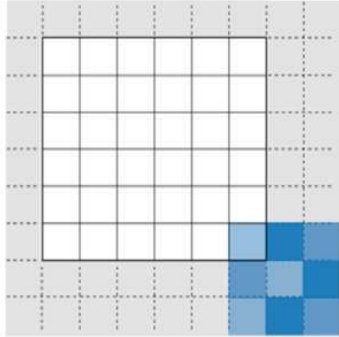
0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

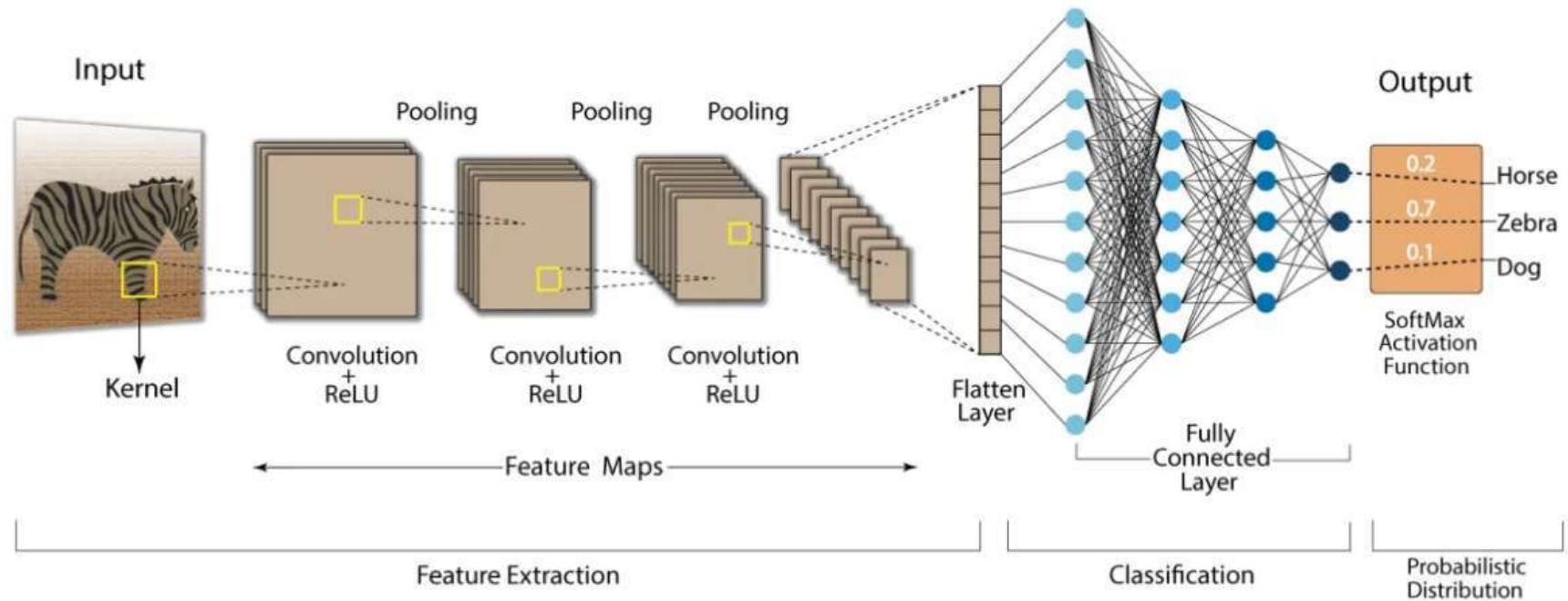
Output

Padding



Mode	Valid	Same	Full
Value	$P = 0$	$P_{\text{start}} = \left\lfloor \frac{S \lceil \frac{I}{S} \rceil - I + F - S}{2} \right\rfloor$ $P_{\text{end}} = \left\lceil \frac{S \lceil \frac{I}{S} \rceil - I + F - S}{2} \right\rceil$	$P_{\text{start}} \in \llbracket 0, F - 1 \rrbracket$ $P_{\text{end}} = F - 1$
Illustration			
Purpose	• No padding • Drops last convolution if dimensions do not match	• Padding such that feature map size has size $\lceil \frac{I}{S} \rceil$ • Output size is mathematically convenient • Also called 'half' padding	• Maximum padding such that end convolutions are applied on the limits of the input • Filter 'sees' the input end-to-end

Example of ConvNet



Complete CNN representation | [Source](#)